

Table of Contents

1. [Code Workspaces • Overview • Palantir](#)
2. [Code Workspaces • Overview • Palantir](#)
3. [Code Workspaces • Getting started • Palantir](#)
4. [Code Workspaces • Interact with data • Palantir](#)
5. [Code Workspaces • Interact with external systems • Palantir](#)
6. [Code Workspaces • Interact with the Ontology • Palantir](#)
7. [Code Workspaces • Train and interact with models • Palantir](#)
8. [Code Workspaces • JupyterLab® • Palantir](#)
9. [Code Workspaces • RStudio® Workbench • Palantir](#)
10. [Code Workspaces • Palantir-provided models • Palantir](#)
11. [Code Workspaces • Security • Palantir](#)
12. [Code Workspaces • FAQ • Palantir](#)
13. [Code Workspaces • Troubleshooting • Palantir](#)
14. [Code Workspaces • Compute usage • Palantir](#)

Code Workspaces

Code Workspaces brings the JupyterLab®, RStudio® Workbench, and VS Code third-party IDEs to Palantir Foundry, enabling users to boost their productivity and accelerate their data science and statistics workflows by using their preferred tools on the high-quality data of the Foundry Ontology. Code Workspaces containers are natively integrated with the rest of the Foundry ecosystem to combine familiar IDEs with the benefits of the Foundry platform, such as data security, branching, build scheduling, and resource management.

Code Workspaces gives platform administrators an easily-deployed, fully-managed, secure, and production-ready way to provide JupyterLab®, RStudio® Workbench, and VS Code to users with Foundry's data governance and compliance with FedRAMP, GxP, and other standards built-in. With Code Workspaces, users can securely connect to existing internal systems and build analyses, transforms, models, applications, or entire workflows on data with Foundry's access controls and data permissioning.

Key features

Key features of Code Workspaces include:

- **Security:** Code Workspaces is built on the core components of [Foundry security](#) that underpin the platform as a whole, like robust permissions and granular access controls. This provides Foundry's security model to the third-party IDEs available in Code Workspaces. For example, restricting access to a dataset in Foundry will restrict it for Code Workspaces IDEs as well, ensuring consistent permissions across tools.

- **Customizable environments:** Code Workspaces allows users to define custom [workspace](#) files and increase or decrease the compute resources of their [workspace](#) as desired.

[API Reference](#) ↗

Workflow support: Code Workspaces are backed by the [Code](#)

infrastructure, which provides industry-standard version control features like branching,

[Send feedback](#)

merging, and commit history. These features enable multiple users to operate in the same workspace more easily and safely.

- **Applications:** Code Workspaces currently supports [Dash ↗](#) and [Streamlit ↗](#) for Python applications and [Shiny® ↗](#) for R applications. Users can create application workflows directly in Code Workspaces with Foundry's version control, branching, and data governance features built-in.
- **Model integration:** Users can create model assets from within a Code Workspace and track these assets with [modeling objectives](#). Multiple models can be created from the same workspace.
- **Transforms/build integration:** Code Workspaces serves as a development environment for transforms. Logic written in Code Workspaces can be published as data transformation pipelines and seamlessly integrates with Foundry's [data integration](#) toolkit, including builds, schedules, data lineage, and health checks. Code Workspaces supports both R transforms and Python/Jupyter® transforms.

When to use Code Workspaces

Foundry has a variety of applications you can use for analytical or coding purposes. For example, if you are an analyst, you may be best served by Contour, Foundry's point-and-click low-code interface for dataset analysis.

If you need to write large-scale data pipelines, set up data connections, or work with streaming data, other Foundry tools have more functionality than Code Workspaces; for these use cases, we recommend using [Pipeline Builder](#), [Data Connection](#), and [Foundry Streaming](#), respectively.

Specifically, Code Workspaces runs on a single node, while other Foundry applications leverage a Spark infrastructure. Thus, we recommend that users performing large-scale data transformations choose [Pipeline Builder](#) or [Code Repositories](#) instead of Code Workspaces.

Code Workspaces is geared for building machine learning models or those familiar with working in JupyterLab® or RStudio® Workbench.

API Reference ↗

Learn more

Send feedback

Code Workspaces currently supports three environments: [JupyterLab®](#), [RStudio®](#), and [VS Code](#).

More information about Code Workspaces can be found in the [FAQ](#).

[Get started using Code Workspaces with this tutorial](#).

RStudio® and Shiny® are trademarks of Posit™.

Jupyter®, JupyterLab®, and the Jupyter® logos are trademarks or registered trademarks of NumFOCUS.

All third-party trademarks (including logos and icons) referenced remain the property of their respective owners. No affiliation or endorsement is implied.

PREVIOUS

← Custom application building / Custom Endpoints / Use custom endpoints in your applications

NEXT

Getting started →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

API Reference ↗

Send feedback

Code Workspaces

Code Workspaces brings the JupyterLab®, RStudio® Workbench, and VS Code third-party IDEs to Palantir Foundry, enabling users to boost their productivity and accelerate their data science and statistics workflows by using their preferred tools on the high-quality data of the Foundry Ontology. Code Workspaces containers are natively integrated with the rest of the Foundry ecosystem to combine familiar IDEs with the benefits of the Foundry platform, such as data security, branching, build scheduling, and resource management.

Code Workspaces gives platform administrators an easily-deployed, fully-managed, secure, and production-ready way to provide JupyterLab®, RStudio® Workbench, and VS Code to users with Foundry's data governance and compliance with FedRAMP, GxP, and other standards built-in. With Code Workspaces, users can securely connect to existing internal systems and build analyses, transforms, models, applications, or entire workflows on data with Foundry's access controls and data permissioning.

Key features

Key features of Code Workspaces include:

- **Security:** Code Workspaces is built on the core components of [Foundry security](#) that underpin the platform as a whole, like robust permissions and granular access controls. This provides Foundry's security model to the third-party IDEs available in Code Workspaces. For example, restricting access to a dataset in Foundry will restrict it for Code Workspaces IDEs as well, ensuring consistent permissions across tools.

- **Customizable environments:** Code Workspaces allows users to define custom [workspace](#) files and increase or decrease the compute resources of their [workspace](#) as desired.

API Reference ↗



flow support: Code Workspaces are backed by the [Code](#)

Send feedback

infrastructure, which provides industry-standard version control features like branching,

merging, and commit history. These features enable multiple users to operate in the same workspace more easily and safely.

- **Applications:** Code Workspaces currently supports [Dash ↗](#) and [Streamlit ↗](#) for Python applications and [Shiny® ↗](#) for R applications. Users can create application workflows directly in Code Workspaces with Foundry's version control, branching, and data governance features built-in.
- **Model integration:** Users can create model assets from within a Code Workspace and track these assets with [modeling objectives](#). Multiple models can be created from the same workspace.
- **Transforms/build integration:** Code Workspaces serves as a development environment for transforms. Logic written in Code Workspaces can be published as data transformation pipelines and seamlessly integrates with Foundry's [data integration](#) toolkit, including builds, schedules, data lineage, and health checks. Code Workspaces supports both R transforms and Python/Jupyter® transforms.

When to use Code Workspaces

Foundry has a variety of applications you can use for analytical or coding purposes. For example, if you are an analyst, you may be best served by Contour, Foundry's point-and-click low-code interface for dataset analysis.

If you need to write large-scale data pipelines, set up data connections, or work with streaming data, other Foundry tools have more functionality than Code Workspaces; for these use cases, we recommend using [Pipeline Builder](#), [Data Connection](#), and [Foundry Streaming](#), respectively.

Specifically, Code Workspaces runs on a single node, while other Foundry applications leverage a Spark infrastructure. Thus, we recommend that users performing large-scale data transformations choose [Pipeline Builder](#) or [Code Repositories](#) instead of Code Workspaces.

Code Workspaces is geared for building machine learning models or those familiar with working in JupyterLab® or RStudio® Workbench.

API Reference ↗

Learn more

Send feedback

Code Workspaces currently supports three environments: [JupyterLab®](#), [RStudio®](#), and [VS Code](#).

More information about Code Workspaces can be found in the [FAQ](#).

[Get started using Code Workspaces with this tutorial](#).

RStudio® and Shiny® are trademarks of Posit™.

Jupyter®, JupyterLab®, and the Jupyter® logos are trademarks or registered trademarks of NumFOCUS.

All third-party trademarks (including logos and icons) referenced remain the property of their respective owners. No affiliation or endorsement is implied.

Developer toolchain / Code Workspaces / Getting started

Getting started with Code Workspaces

This tutorial provides an introduction to Code Workspaces by setting up a workspace for either JupyterLab® or RStudio® Workbench.

Backing code repository

All code workspaces are backed by a Palantir platform [code repository](#). This enables code workspaces to have industry-standard version control features like branching, merging, and commit history, allowing other users to view the code and safely operate in the same workspace.

⚠ Previously, code workspaces and associated code repositories were represented by separate resources. Now, code workspaces and associated code repositories are represented by a single resource; the code repository. Selecting the code repository from any folder in Foundry will automatically open the associated Jupyter or RStudio® workspace.

This change applies to new code workspaces, and does not affect existing code workspaces and code repositories, which will continue to be separate resources..

Launch a code workspace

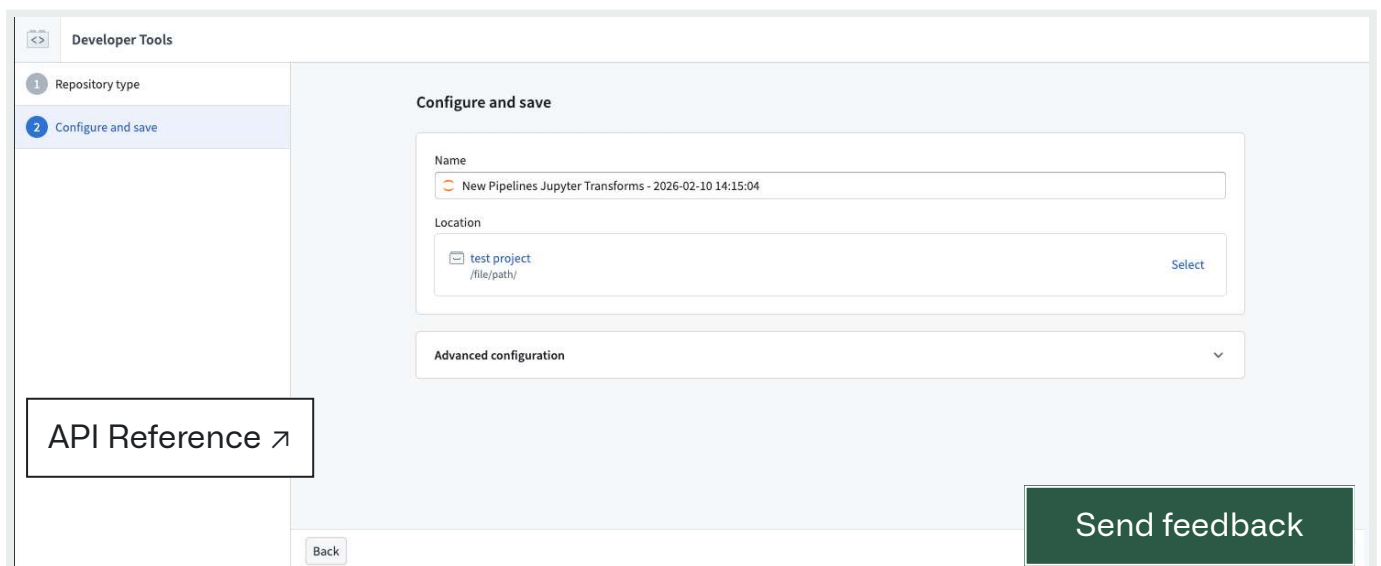
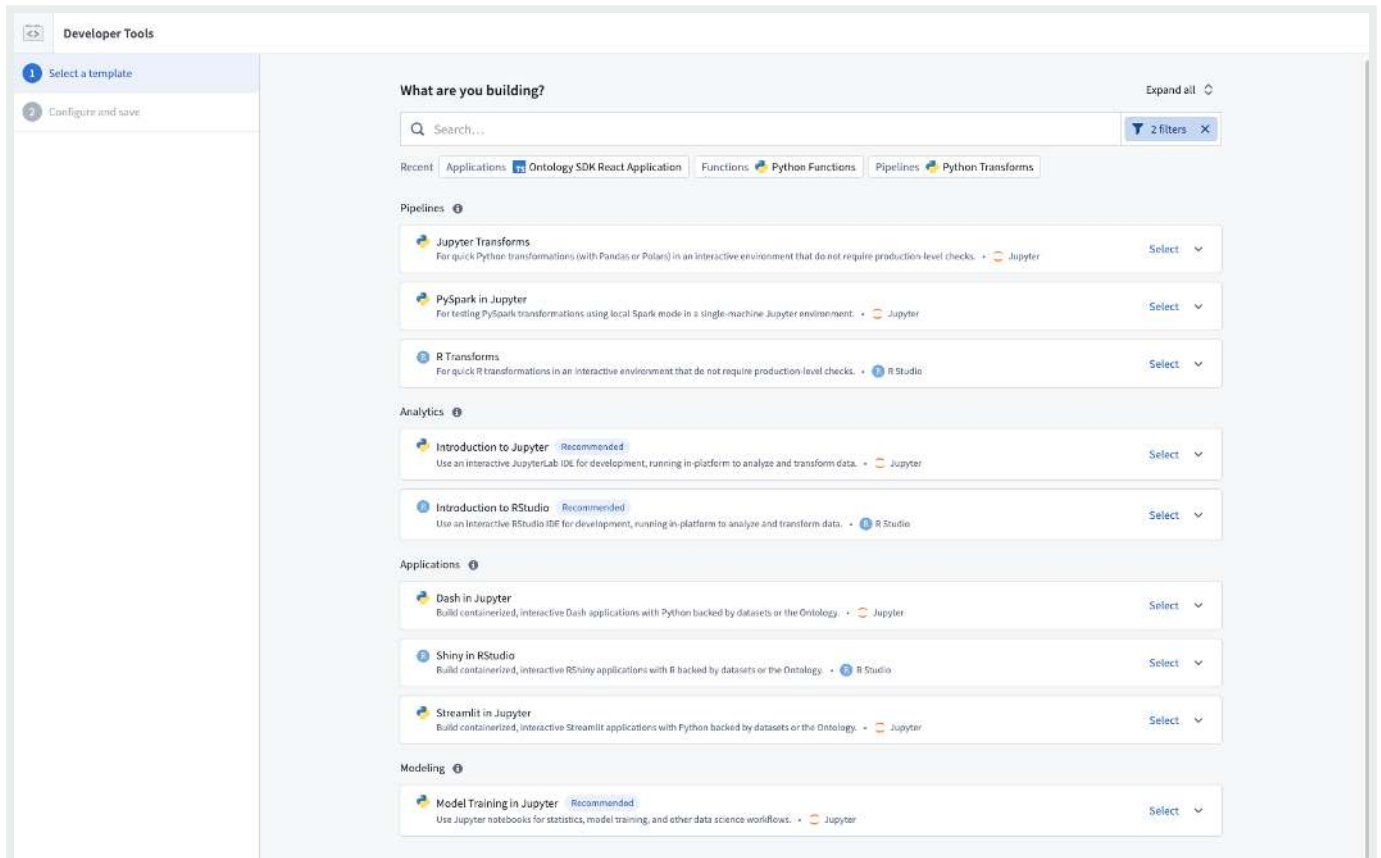
API Reference ↗

To launch a code workspace, open the Code Workspaces application . Then, select a folder in which to create your code workspace.

Send feedback

i Users with **Editor** or **Owner** roles on the project will be able to modify the settings of the code workspace. Users with **Viewer** roles on the project will be able to launch the workspace. You can select your home folder as a location, but this may cause your code workspace and data to be restricted to a project.

Choose the type of workspace to launch. Currently, the supported workspaces are JupyterLab and RStudio® Workbench.



Then, set the name and location of the backing code repository for the workspace you want to create. If you are creating an RStudio® workspace, you will be able to select a license. In most cases, the license will be automatically selected.

The screenshot shows the 'Configure and save' step in a workspace creation process. On the left, a sidebar under 'Developer Tools' has two steps: '1 Repository type' and '2 Configure and save'. The main area is titled 'Configure and save' and contains several configuration sections:

- Name:** A text input field containing 'New Pipelines Jupyter Transforms - 2026-02-10 14:15:04'.
- Location:** A dropdown menu showing 'test project' with a 'Select' button to the right.
- Advanced configuration:** A section with an expandable arrow. It contains:
 - A blue information bar: 'These settings can be modified at any point in the Settings tab of the workspace.'
 - Maximum enabled profile:** A dropdown menu set to 'Medium' with the description 'Up to 2 CPUs • Up to 16 GB memory'.
 - CPU and memory allocation:** A text block explaining that the ratio of memory to CPU is fixed at 8 GB per CPU.
 - Compute resources:** A slider bar with four positions: '1 CPU / 8GB' (selected), '2 CPUs', '4 CPUs', and '8 CPUs'.
 - GPU configuration:** A dropdown menu set to 'No GPU'.
 - Idle timeout:** A text block explaining the auto-shutdown feature, followed by a dropdown menu set to '30 minutes'.

At the bottom of the interface, there is a 'Back' button on the left and an 'Initialize repository' button on the right. The bottom left corner shows 'space/compass'.

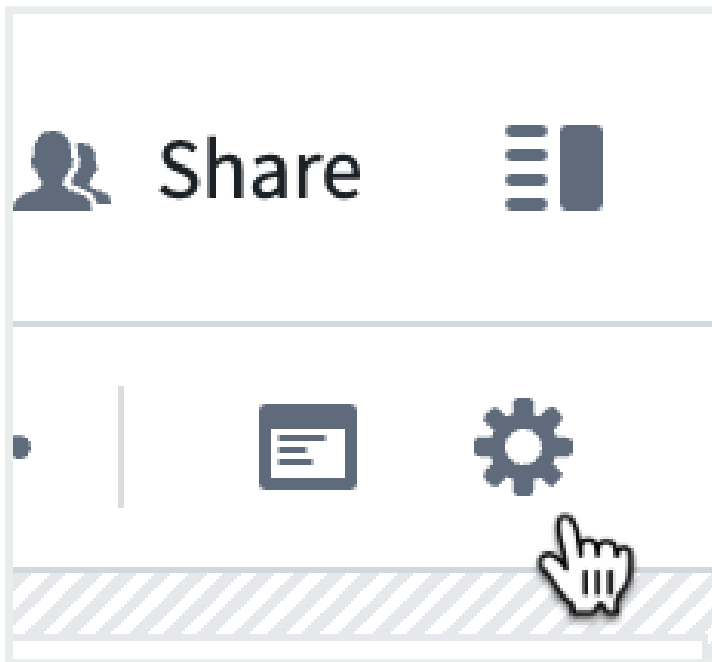
Each user who launches the workspace will run their own isolated session. If two or more users will be using the same workspace, ensure that all users have **Viewer** permission or higher for the underlying code repository. See above for more information about a code workspace's [underlying code repository](#).

Advanced settings

Advanced settings allow you to define idle time and the resource profile for your code workspace. These settings can be modified at any time in the **Settings** panel of the workspace accessible by selecting the gear icon in the top right corner.

API Reference ↗

Send feedback



- **Auto-shutdown**

- The auto-shutdown setting defines the period of time that your container will remain active. The workspace will automatically be shut down after this period of inactivity in order to reduce resource usage. After an auto-shutdown, packages and datasets will need to be redeployed to your container. Note that your code will persist after the auto-shutdown.
- The default auto-shutdown period is 30 minutes. The maximum auto-shutdown setting is 6 hours, and the maximum session length is 24 hours.
- Longer auto-shutdown settings can incur higher costs. We recommend setting a shorter auto-shutdown period to reduce cost, and to rely on code committed to Foundry rather than uncommitted changes which would be restored across sessions (see the [code workspace lifecycle documentation](#) below).

- **Compute Resources**

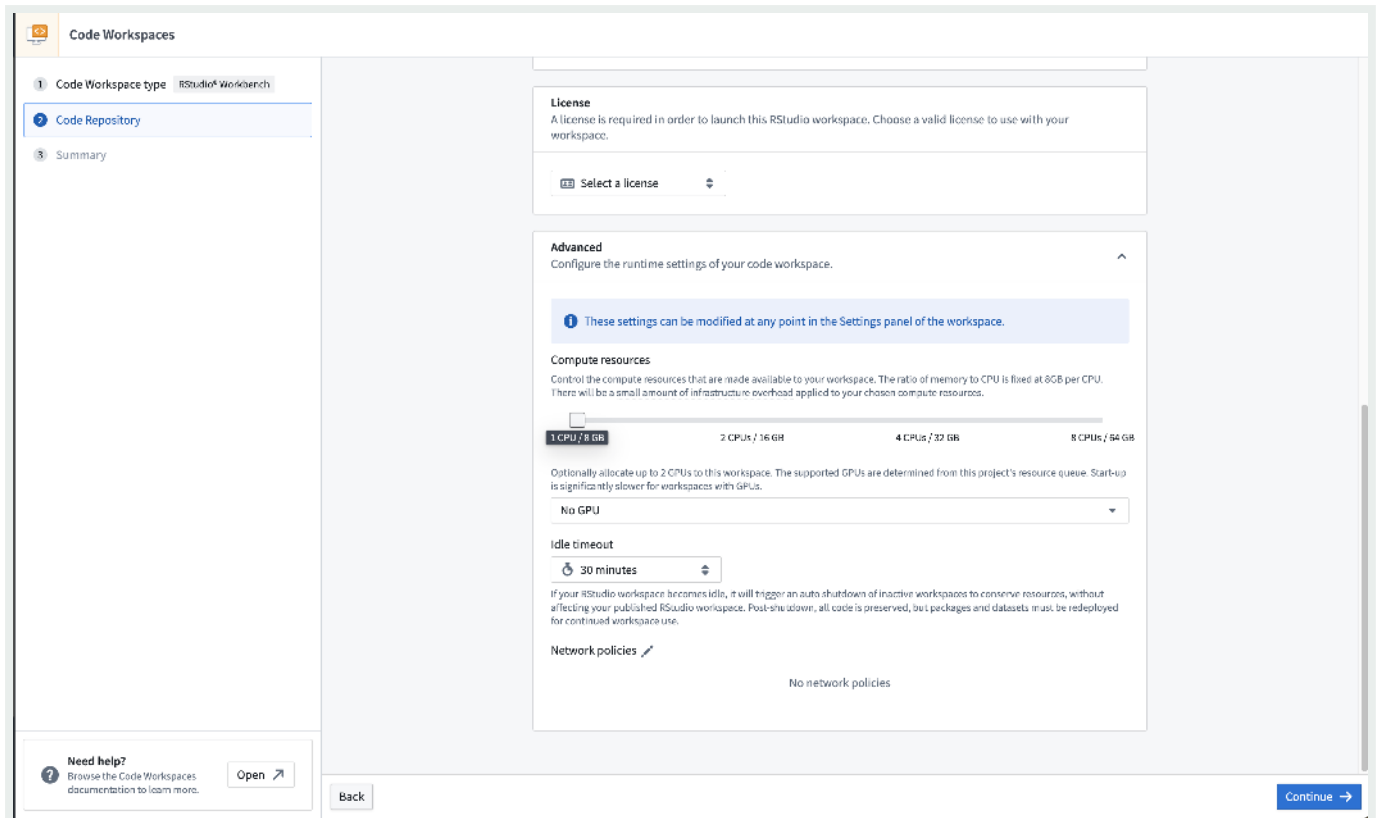
- Larger resource profiles can incur higher costs.

- **Network Policies**

- If enabled by your administrator, you will be able to select network policies to attach to your workspace in order to make external API calls from your workspace.

API Reference ↗

Send feedback



Transforms

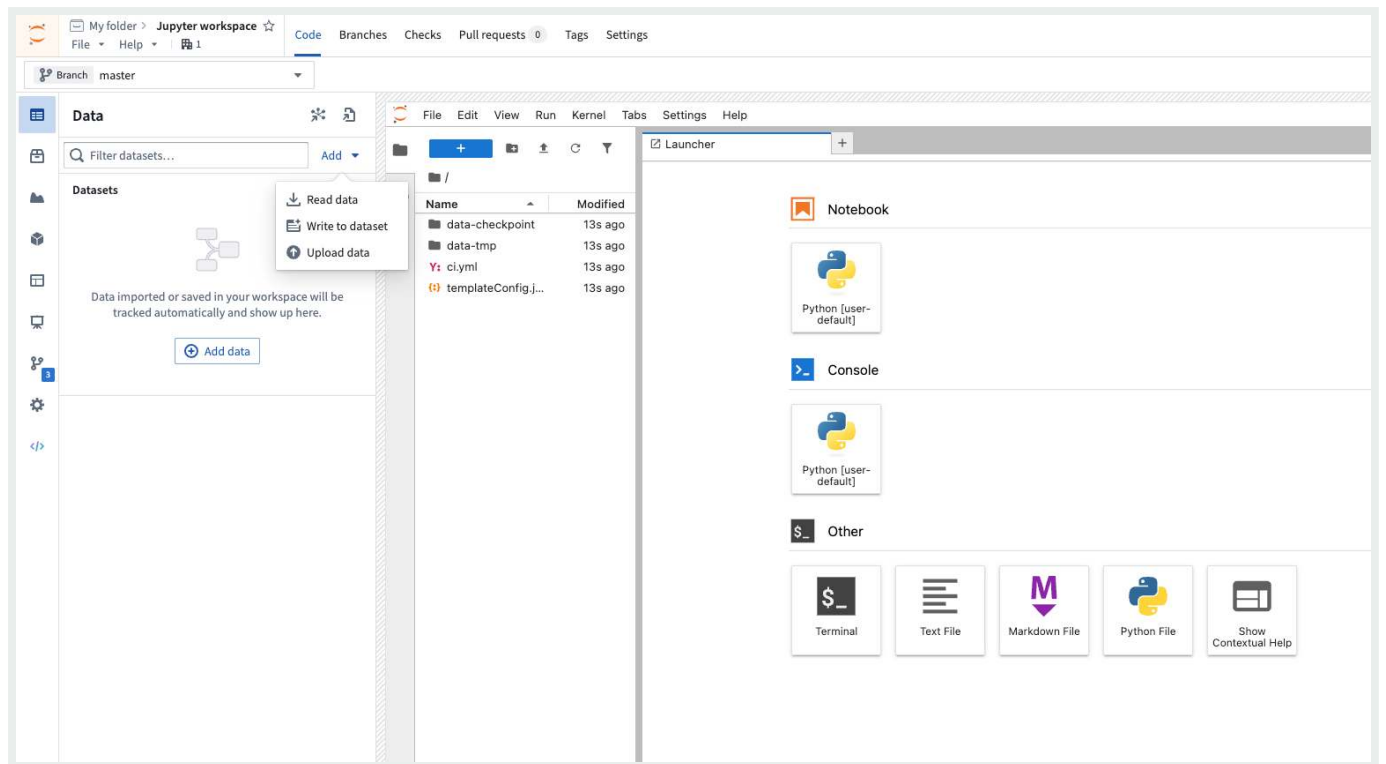
- i** Users cannot publish transforms directly in an underlying code repository, since the repository is customized for use as infrastructure for Code Workspaces. All code must be published from the code workspace.

Import data

You can import data from sources in Foundry directly by navigating to the **Data** tab and then selecting **Add > Read data**. This includes Foundry datasets, [Iceberg tables](#), and [virtual tables](#). Any data that is imported from outside the code workspace's current project (including the user's home folder) will require a reference to be added.

Alternatively, for data that has not yet been ingested into Foundry, select **Upload data** to in [API Reference ↗](#) from your device.

Send feedback



Dataset views cannot be used as imported datasets in a code workspace. Instead, copy the dataset view to a new or existing dataset where necessary.

Import packages

You can import packages into your code workspace by navigating to the **Packages** tab in the left panel, or you can install packages directly using Conda or PyPI for Jupyter® and CRAN for RStudio®.

JupyterLab® Packages

JupyterLab® workspaces support installation of packages from Conda or PyPI. By default, Code Workspaces exposes `conda-forge` and `pypi.org`. You can add other channels to a code workspace by adding them to the backing code repository and restarting your code workspace.

API Reference ↗

Components in JupyterLab® can be consistently restored across workflows in JupyterLab®, applications, or transforms using managed environments.

Send feedback

You can browse packages from the **Packages** tab and select the package you wish to install. This will automatically execute an installation command for you in a new JupyterLab® terminal.

To install packages manually, you can use `maestro env conda install <packages>` for Conda packages and `maestro env pip install <packages>` for PyPI packages in the JupyterLab® terminal.

Review [managed Conda/PyPI environments in Jupyter® Notebooks](#) for more information.

 Installing packages directly with `mamba` and `pip` is now deprecated.

CRAN

RStudio® Workspaces support installation of packages from CRAN, Posit™ Package Manager, or Bioconductor. By default, Code Workspaces exposes all of [Posit™ Public Package Manager ↗](#) as well as [Bioconductor ↗](#). You can add other channels (such as MRAN) by adding them to the backing code repository and restarting your code workspace.

To install packages, you can use `renv` since `renv` supports authenticated CRAN channels.

You can browse packages from the **Packages** tab and select the package you wish to install. This will generate an installation command for you to copy-paste in RStudio® Workbench and execute. Generally, CRAN packages are installed with `renv::install("<package>")` and Bioconductor packages with `renv::install("bioc::<package>")`.

Publish your code

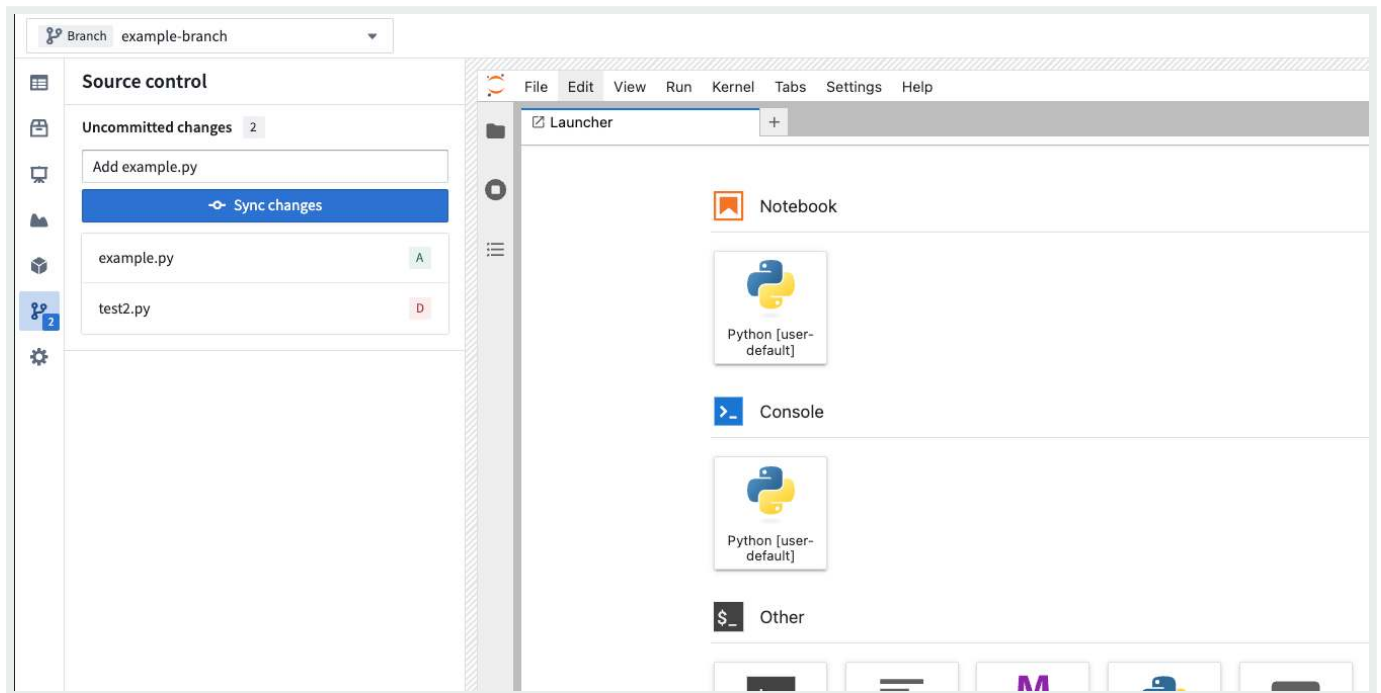
In order for other users of the same workspace to view your code, you must publish your code to the branch you are working on.

Now that you have [API Reference ↗](#) your code will cause CI checks to run, which will publish a new version of the Code Workspace application. As a best practice, we recommend keeping code workspaces focused on a single type of task (such as building a

[Send feedback](#)

analyzing data, writing data transformations, or building models), rather than attempting to accomplish multiple independent workflows in a single code workspace.

Unpublished changes can be viewed in the **Source control** side panel. Optionally supply a commit message and then select **Sync changes** to publish your code. When collapsed, the number of file changes is displayed over the source control icon.



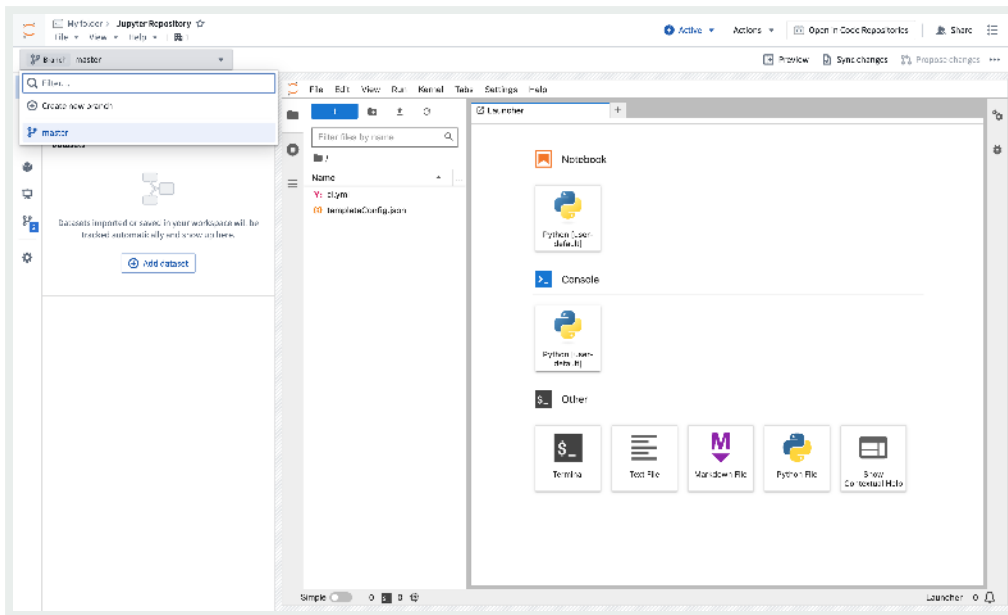
Branch your code workspace

You can branch your code workspace, check out branches, and perform other version control activities with the branch manager.

To create or find a branch, use the branch selector at the top of the screen. Selecting the branch selector opens a menu displaying the list of existing branches. Alternatively, a new branch can be created by selecting **Create new branch**. If you have unpublished code changes, you will be prompted to sync before checking out the selected branch.

[API Reference ↗](#)

[Send feedback](#)



In the same way as [branching elsewhere in Foundry](#), the data available in a code workspace comes from the branch you are currently on (with [fallbacks](#)). If you write data to an output dataset, the data will be written to your current branch.

You can merge your branch into master by selecting **Propose changes**. After proposing changes, a pull request (PR) preview will appear.

Share your code workspace

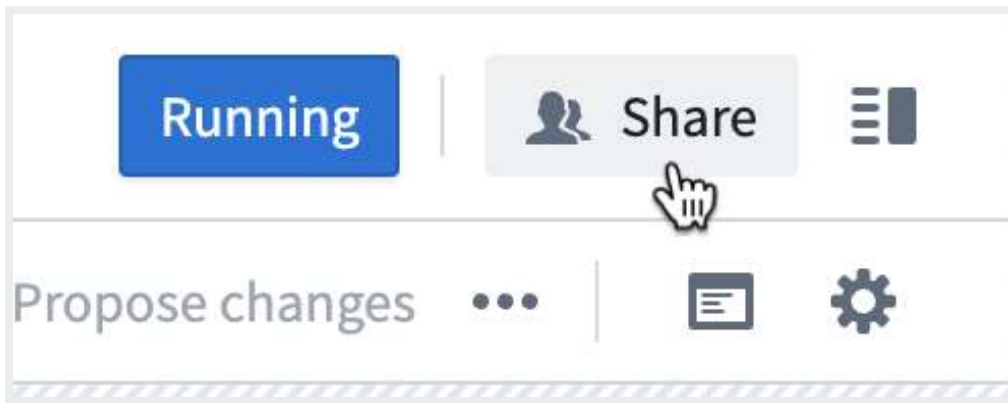
You can share your code workspace by selecting **Share** in the upper-right corner of the interface.

⚠ If the code workspace and code repository are separate resources, you must also share the permissions on the underlying repository for others to see the code in your workspace.

Users with **Editor** or **Owner** roles on the project will be able to modify the settings of the code workspace. Users with **Viewer** roles on the project will be able to launch the

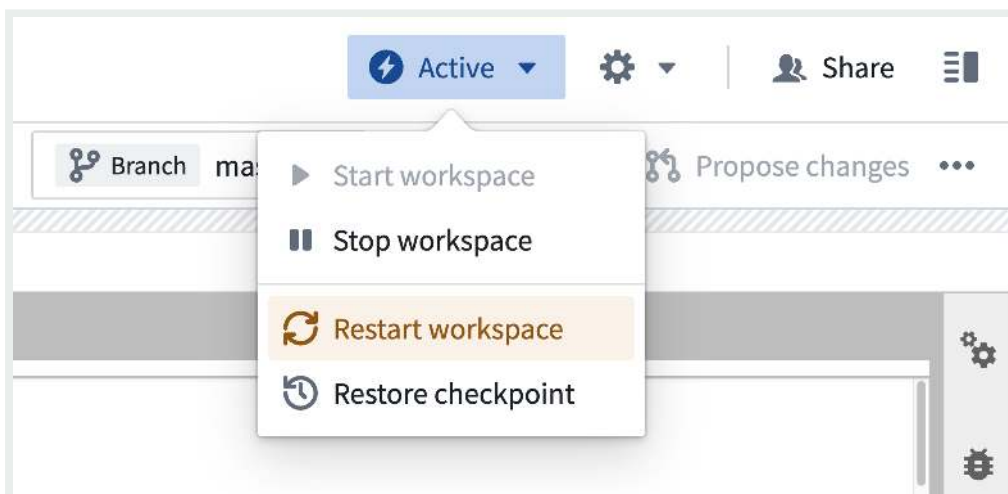
[API Reference](#) ↗

[Send feedback](#)



Restart a code workspace

You can manually restart a code workspace (for instance, to pick up configuration changes) by selecting **Active > Restart workspace** in the upper-right corner of the interface.



This will result in a new session in the initial state. The workspace will automatically restart with all the changes that were committed to the backing code repository. However, the workspace will not contain everything from the state present before the restart, including imported data, variables stored in memory, installed packages, and more.

Before restarting a workspace, the application will automatically create a checkpoint and push it to the code repository. For more information, see the [API Reference](#) for our latest changes.

You can select **Active > Restore checkpoint** to restore the workspace to its previous checkpoint. See the [checkpoints](#) section below for more information. Code

Send feedback

Workspaces will not automatically restore the workspace to a previous checkpoint following a workspace restart.

Code workspace lifecycle

Users who open Code Workspaces for the first time will receive a blank workspace with the backing repository cloned in `/home/user/repo`.

Code Workspaces sessions are automatically shut down when there is no user activity, and can last at most 24 hours (see more about automatic shutdowns in the [advanced settings](#) documentation).

When a session shuts down, all the relevant state is persisted in Foundry in a [checkpoint](#) and restored the next time the same user opens the code workspace. This allows users to pick up where they were at the time of shutdown.

Save work in a code workspace

Code Workspaces has two distinct ways to save existing work: manual code syncing and automated checkpointing.

i We recommend relying on code synced to the underlying code repository (with the required package installations as part of the code) rather than relying on workspace checkpoints. Code syncing will help ensure a resilient workflow and facilitate collaboration with other users who may not have access to a given checkpoint.

Code syncs

Code syncing is a manual action and the recommended way to save existing work in a code workspace. Syncing code means committing your files to the backing code repository of the workspace. Code saved through code syncing is permanently saved in the repository, and everything in the `/home/user/repo` folder is stored as part of that sync. Code syncing does not save existing states such as variable contents, installed packages, environment, data, or code cell outputs. The sync history of the workspace can be consulted

API Reference ↗

Send feedback

through the backing code repository of the workspace. We generally recommend to frequently sync your code to both maintain a history of changes and ensure all relevant work is properly and permanently stored.

To save all of your existing changes in a workspace, select **Sync changes** in the top right corner of the workspace. Once that succeeds, the changes are committed to the backing repository of the workspace, which you can access by selecting the gear icon in the top right corner then choosing **Open code repository**.

Checkpoints

Checkpointing is a feature that automatically takes snapshots of the existing state of a workspace to be able to restore it across sessions. There are two types of checkpoints: data checkpoints and code checkpoints.

Data checkpoints

Unlike code syncs that only store committed files, data checkpoints store all contents of `/home/user/repo` (including uncommitted code changes), `/home/user/libs` (including installed packages), `/home/user/envs` (Conda environments), and `/home/user/data` (extra data downloaded by the user). In the case of RStudio®, variables stored in memory are stored in `/home/user/repo/.RData`.

By default, Code Workspaces saves a data checkpoint of a given user's session every ten minutes. Data checkpoints are not shared across users. Code Workspaces also creates a data checkpoint before the workspace shuts down (through a [manual restart](#), a manual shutdown, or an automatic shutdown, for example). An inactive code workspace does not generate any new data checkpoints, and any given data checkpoint lasts up to a maximum of 30 days. As a result, since no valid data checkpoints will be available, a workspace left untouched for more than 30 days will only be able to restore work from what was synced to the backing repository or from a [code checkpoint](#). Data checkpoints are designed to be a safety net to persist the workspaces's state across sessions; they should not be relied on as a primary form of saving. We strongly recommend frequently syncing existing work in the workspace.

API Reference ↗

Restoring a data checkpoint will restart the workspace and restore the home folder to how it was at the time the checkpoint was created.

Send feedback

Code checkpoints

Code checkpoints store *only* the workspace code (not data) by saving the contents of `/home/user/repo` (including uncommitted code changes) while excluding files that are either above a certain size or have extensions commonly associated with data. This filtering is done on a best-effort basis. Code checkpoints are created only before the workspace is shut down. Unlike data checkpoints, there is no retention period for code checkpoints.

Code checkpoints are not subject to filtering that would prevent you from committing common data files to your workspace's backing repository. You must ensure that code files do not contain data. For example, a `.py` code file should never contain tabular data that exists in a Foundry dataset.

Manage data files

Code Workspaces automatically scans your `/home/user/repo/` folder for files with large sizes or common data file extensions, such as `.csv`, `.gz`, `.parquet`, `.pdf`, or `.xlsx`. Moving these files to either the `data-checkpoint` or `data-tmp` directory optimizes workspace startup times and improves security, acting as a safeguard in case you or another user unintentionally commit these files to the workspace's backing repository.

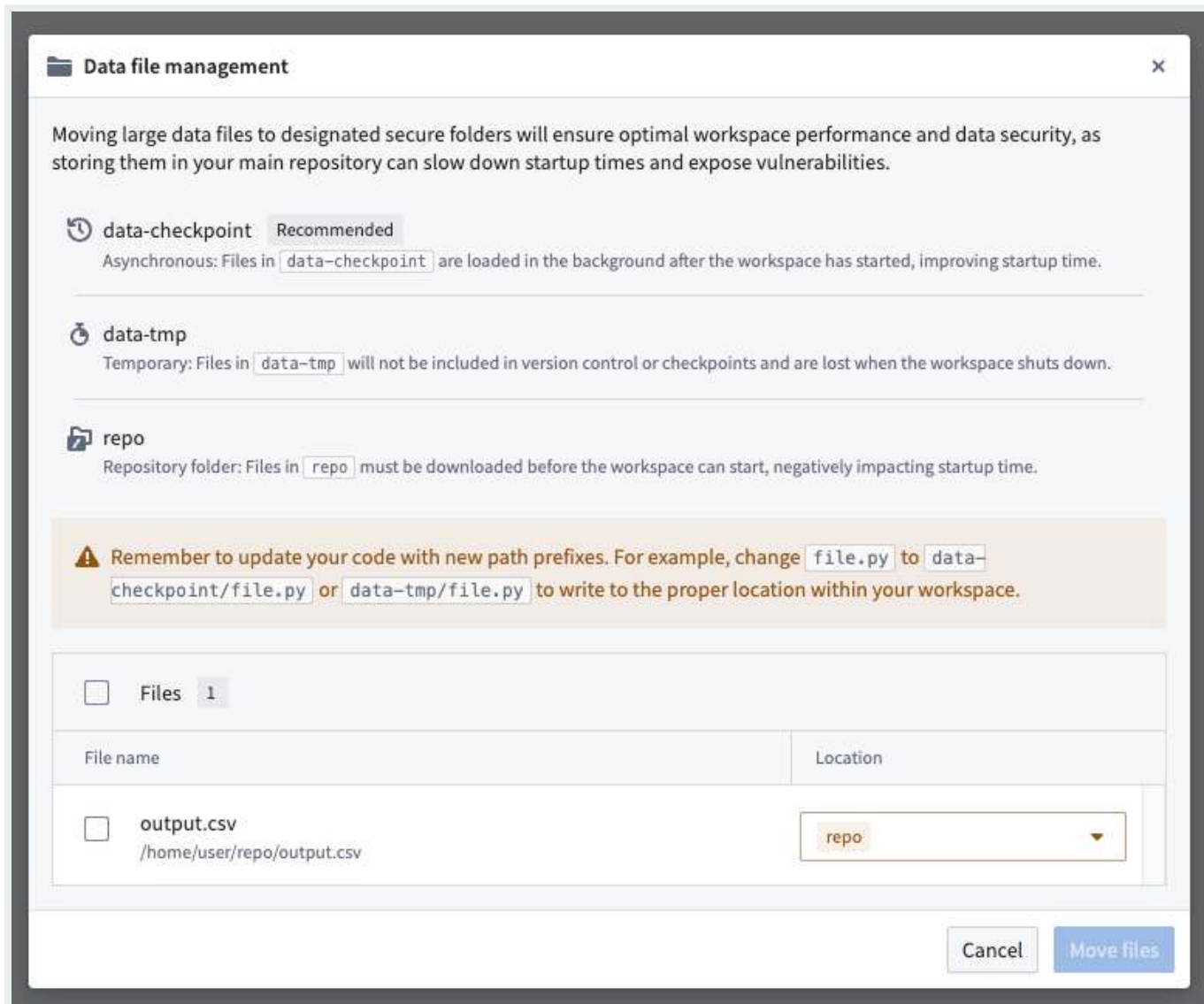
1. **data-checkpoint**: The recommended place to store data files that you want to save and restore across sessions. These files will be saved to checkpoints and restored in the background without blocking workspace startup. Additionally, this folder will be excluded from syncs to prevent committing large, unsecured data to the backing code repository.
2. **data-tmp**: The recommended place to store temporary files that you do not want to save across sessions. These files will not be saved to checkpoints or the backing code repository; they will instead be discarded when your workspace shuts down.

If you want data to be available to published dashboards or transforms, or to other users, we recommend that you [persist data to a Foundry dataset](#) instead of saving it in your workspace.

When the workspace detects data files, you will see a warning that prompts you to move them to the `data-checkpoint` or `data-tmp` directory. Select the files you want to relocate and follow the dialog to move them. Remember to update your code with the new path prefixes, or else the warning will appear again as soon as you write to the `/home/user/repo/` folder.

API Reference ↗

Send feedback



RStudio® and Shiny® are trademarks of Posit™.

Jupyter®, JupyterLab®, and the Jupyter® logos are trademarks or registered trademarks of NumFOCUS.

All third-party trademarks (including logos and icons) referenced remain the property of their respective owners. No affiliation or endorsement is implied.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Developer toolchain / Code Workspaces / Interact with data

Interact with data

Jupyter® and RStudio® Code Workspaces allow you to read, analyze, transform, and write back to Foundry datasets, [iceberg tables](#), and [virtual tables](#). They also allow you to read and analyze [restricted views](#) and [time series properties](#).

Security

Code Workspaces respects [Foundry markings](#) applied to datasets, and a workspace will inherit the markings of all datasets loaded into it. This is referred to as the **workspace lineage**. This means that to access a workspace, you must also have the required permissions for all of the datasets and other inputs contained in the workspace. If you lose access to a single input of the workspace, you will lose access to the entire workspace.

Resource considerations

Jupyter® and RStudio® workspaces are designed for interactive, analytical workflows and application development. They run on a single virtual machine and do not process datasets with a distributed Spark environment. As a result, the data you work with must fit into your machine's resources, which default to a maximum of 8 CPUs and 64 GB of memory. Using [filters](#) or [SQL](#) to reduce the amount of data you need before loading large datasets can help you avoid exceeding these limits. [Filtering dataset files](#) is another strategy to stay within your workspace's limits.

To develop large-scale data pipelines, use [Python transforms with Spark](#) or [Pipeline Builder](#).

API Reference ↗

  branching

Send feedback

By default, Code Workspaces will load the data of the dataset from the same branch as the workspace itself, and will otherwise fall back to the `master` branch. For example, a code workspace currently on branch `my-branch` will try to read the `my-branch` version of the dataset, and will fall back to the `master` branch if `my-branch` does not exist on the dataset.

To pin a dataset branch for a specific imported dataset, select the options **••** icon next to the dataset in the **Data** tab of the left sidebar. Then, select **Pin dataset branch for reads** and choose the desired dataset branch for use within your code workspace.

Read data

Code Workspaces requires you to select an **alias** for every Foundry dataset or restricted view that you import into a workspace. The alias acts as a reference that allows you to read from a dataset or restricted view and write to a dataset within your code. When registering a dataset in the **Data** tab, Code Workspaces creates a mapping between the chosen dataset alias and the Foundry resource's unique identifier in a hidden file located under the `/home/user/repo/.foundry` folder of the workspace.

Code Workspaces allows you to load tabular datasets, non-tabular datasets, [Iceberg tables](#), [virtual tables](#), and [restricted views](#). You can add a new data source to your workspace by using the **Add > Read data** button in the **Data** tab.

After selecting a dataset to add to your workspace, you must define an alias for that resource to serve as a unique identifier and reference in your code. Choosing a read strategy for the dataset (such as pandas DataFrame, Polars LazyFrame, or raw file access) generates a code snippet that loads the dataset as the specified input type.

- You can modify this read strategy and generate a new snippet at any time, even after registering the dataset.
- By default, code workspaces will suggest a dataset alias with the same name as the dataset itself.

Selecting **Done** will complete the dataset registration process and allow you to use the data in your code with the **Run snippet** or **Copy to clipboard** options.

API Reference ↗

This page provides examples for [tabular datasets](#), [non-tabular datasets](#), and [restricted views](#).

Send feedback

i Reading a dataset does not require you to know its file names or file count. A single Foundry dataset is often backed by many underlying files, and those files may be named dynamically or vary in number between runs. This is common when the dataset is produced by a pipeline that partitions its output or includes variables in file names.

- If the dataset has a schema, read it as a table with `read_table` or `SQL`. These methods read the entire logical table regardless of how many files back the dataset or how those files are named, so you do not need to enumerate files individually.
- If the dataset has no schema, or you need the raw file contents, read every file without hardcoding names by `iterating over all files in the dataset`.

You do not need to treat structured, schema-backed data as unstructured files simply because the backing files are numerous or dynamically named.

Tabular datasets

The following snippets are generated for a `Cats` tabular dataset with a dataset alias `kittens`. Notice that `Cats` is not referenced anywhere in the code snippet; Code Workspaces implicitly registers it under your chosen alias.

In Jupyter®:

```
1 from foundry.transforms import Dataset
2
3 kittens = Dataset.get("kittens")\
4     .read_table(format="arrow")\
5     .to_pandas()
```

For the full list of methods and properties available on the `Dataset` class used in Jupyter® workspaces, see the [foundry.transforms.Dataset API reference](#).

API Reference ↗

Note that the `read_table` method shown above supports the following parameters:

Send feedback

- **arrow (recommended)**: Converts the dataset to an Apache Arrow table on which efficient filtering can be performed. You can then convert this table to a pandas dataframe using `.to_pandas()`.
- **pandas or dataframe**: Converts the dataset to a pandas dataframe.
- **polars**: Converts the dataset to a Polars dataframe. You can then convert it to a pandas dataframe using `.to_pandas()`.
- **lazy-polars**: The lazy variant of a Polar dataframe. [Filters](#) cannot be executed on lazy polars.
- **path**: Outputs the local path under which the dataset is stored.

i For a given format to be available as part of a `read_table` operation, the corresponding packages `pyarrow`, `pandas` and `polars` must be present in the environment. These are included automatically in the default Code Workspaces environment, but may need to be manually added to your custom environment.

In RStudio®:

```
1 kittens <- datasets.read_table("kittens")
```

The syntax above loads the dataset and automatically collects the data into an R dataframe.

If the data exceeds the workspace's memory capacity, you can apply push-down [filters](#) to only load a subset of rows or columns using the following syntax:

```
1 library(dplyr) # should be imported by default in .Rprofile
2
3 kittens_df <- Dataset.get("kittens") %>%
4   # (optional) apply other transformations before collecting
5   collect()
```

Query datasets with SQL

API Reference ↗

You may query any tabular dataset, [Iceberg table](#), or [virtual table](#) that using SQL in Python.

Send feedback

The following snippets are generated for a `Ducks` tabular dataset with a dataset alias `ducklings`. Notice that `Ducks` is not referenced anywhere in the code snippet; Code Workspaces implicitly registers it under your chosen alias.

You may only query one dataset per `SELECT` Spark SQL statement using the approaches below. Queries return the results as a [PyArrow table](#) that you can then convert to another format, such as a pandas DataFrame.

- Note that the default limit for SQL queries is one million rows. If you need to query more than one million rows at a time, you can parallelize separate queries and combine their results. Ensure that your data can fit within your workspace's memory.

In Jupyter®:

After installing the `containers-sql` and `foundry-platform-sdk` packages, you may use the `FoundrySdkSqlExecutor` to write Spark SQL:

```
1 from containers_sql import FoundrySdkSqlExecutor
2 sql = FoundrySdkSqlExecutor()
3 query = sql.execute("SELECT * from `ducklings`")
4 df = query.fetch_results() # optionally specify timeout arg: query.fetch_r
```

You may also use a [magic command](#) by first running the following at the top of your notebook:

```
1 import containers_sql
2 %reload_ext containers_sql.foundry_sdk.magics
```

Then, use either the `%sql` magic command to run a single-line query or `%%sql` to run a multi-line query. Using `%%sql -o df` will assign the result of your SQL query to the python object `df` as a PyArrow table:

```
API Reference ↗ = %sql SELECT hatch_date, bill_color, * from `ducklings`
2 ducklings_pandas_df = ducklings_df.to_pandas()
```

Send feedback

```

1 %%sql -o ducklings_df
2 SELECT
3     hatch_date, bill_color, *
4 FROM
5     `ducklings`
6 LIMIT 10;

```

In RStudio®:

First install `containers-sql`, `foundry-platform-sdk`, `pyarrow`, and `reticulate`. Then use the `reticulate` package with the Python SQL executor described above to query a dataset:

```

1 library(reticulate)
2
3 FoundrySdkSqlExecutor <- import("containers_sql")$FoundrySdkSqlExecutor
4 sql <- FoundrySdkSqlExecutor()
5 query <- sql$execute("SELECT * from `ducklings`")
6 results_df <- query$fetch_results()
7
8 # For smaller data, creating an intermediate R object is acceptable.
9 r_list <- py_to_r(results_df$to_pydict())
10 tibble <- as_tibble(r_list)
11
12 # For larger data, you may want to write your data to disk
13 # to avoid exceeding your container's memory limits.
14 library(arrow)
15 pa <- import("pyarrow")
16 pa$feather$write_feather(results_df, "tmp.arrow")
17 tibble <- read_feather("tmp.arrow") |> as_tibble()

```

Non-tabular datasets

The following snippets are generated for a `Dogs` non-tabular dataset with dataset alias `puppies`. Notice that `Dogs` is not referenced anywhere in the code snippet; Code

Wd [API Reference ↗](#) registers it under your chosen alias. Contrary to reading tabular data, which gives you access to the files from the dataset in a `puppies_files`

variable instead of inserting values inside a dataframe.

[Send feedback](#)

In Jupyter®:

```
1 from foundry.transforms import Dataset
2
3 puppies = Dataset.get("puppies")
4 puppies_files = puppies.files().download()
5 with open(puppies_files["puppy_names.txt"], "rb") as f:
6     puppy_names_text = f.read().decode("utf-8")
```

In RStudio®:

```
1 puppies_files <- datasets.list_files("puppies")
2 puppies_local_files <- datasets.download_files("puppies", puppies_files)
3 puppy_names_text <- readLines(raw_files_local_files[["puppy_names.txt"]],
```

See [Filter dataset files](#) for details on how to target a certain subset of files to download into your workspace.

Read all files in a dataset

The examples above access a file by its exact name. When a dataset contains files whose names you do not know in advance, such as files named dynamically by a pipeline or a variable number of output files, download every file and iterate over the returned map instead of referencing names directly. The `download()` method returns a map from each file's name (as defined in the current dataset view) to the local path where it was downloaded, so the code works regardless of how many files the dataset contains or what they are named.

In Jupyter®:

```
1 from foundry.transforms import Dataset
2
3 # Iterate over every file in the dataset, whatever its name or count
4 puppies_files = Dataset.get("puppies").files().download()
5
6 # Iterate over all files without referencing any name
7 for file_name, local_path in downloaded_files.items():
```

API Reference ↗

Send feedback

```

8     with open(local_path, "rb") as f:
9         contents = f.read()
10        # Process each file's contents here

```

To restrict which files are downloaded, such as loading only the files in a given directory prefix, apply a [filter](#) on the file metadata before downloading:

```

1  from foundry.transforms import Dataset
2
3  # Download only the files under the "2024/" directory prefix
4  downloaded_files = Dataset.get("puppies").files()\
5      .filter(lambda f: f.path.startswith("2024/"))\
6      .download()
7
8  for file_name, local_path in downloaded_files.items():
9      # Process each file's contents here
10     ...

```

In RStudio®:

```

1  # List and download every file in the dataset, whatever its name or count
2  all_files <- datasets.list_files("puppies")
3  downloaded_files <- datasets.download_files("puppies", all_files)
4
5  # Iterate over all files without referencing any name explicitly
6  for (file_name in names(downloaded_files)) {
7      local_path <- downloaded_files[[file_name]]
8      contents <- readLines(local_path, warn = FALSE)
9      # Process each file's contents here
10 }

```

i If the dataset has a schema and you intend to work with its tabular contents, prefer reading it as a table with [read_table](#) or [SQL](#). These methods combine all backing files into a single logical table for you, so you do not need to download and parse

[API Reference ↗](#)

[Send feedback](#)

Restricted views

i Querying restricted views is only supported in Jupyter® workspaces.

You can query restricted views using the [SQL approach described above](#) using Python. To do this, import a restricted view into your workspace and query it by its alias as with any other tabular dataset.

Note that importing a restricted view into your code workspace requires enabling [restricted outputs mode](#) to enforce a higher level of data security. With restricted outputs mode enabled, you can perform analysis on data contained in a restricted view but cannot publish outputs from your workspace that use the data.

When adding a restricted view to your workspace for the first time, a prompt will instruct you to install specific Python dependencies. You can then define an alias (as with a dataset) and add the restricted view to your workspace. You must enable restricted outputs mode and restart your workspace to query the restricted view.

i A restricted view's [policy](#) determines what data a user can query. For example, if you import a restricted view into a workspace and another user opens that workspace, they may see different data than you. If your access changes while a restricted view is loaded in a workspace, you will lose access to that workspace and will need to restart the workspace.

Filter dataset files

Code Workspaces enables you to download the files backing any dataset, whether they are tabular (i.e., they have a schema) or non-tabular. It is possible to select a subset of files to download, either by name, or by applying filtering logic on the file metadata (path, size in bytes, transaction RID, and updated time).

In [API Reference ↗](#)

[Send feedback](#)

```

1 from foundry.transforms import Dataset
2
3 # Download all files in the dataset
4 downloaded_files = Dataset.get("my_alias").files().download()
5 local_file = downloaded_files["file.pdf"]
6
7 # Download a single file
8 local_file = Dataset.get("my_alias").files().get("file.pdf").download()
9
10 # Download all PDF files of less than 1MB in the dataset
11 downloaded_files = my_dataset.files()\
12     .filter(lambda f: f.path.endswith(".pdf") and f.size_bytes < 1024 * 1
13     .download()

```

When downloading multiple files, you should use the filter syntax rather than downloading files individually by name to leverage parallel downloads.

In RStudio®:

```

1 # Download all files in the dataset
2 all_files <- datasets.list_files("my_alias")
3 downloaded_files <- datasets.download_files("my_alias", all_files)
4 local_file <- downloaded_files$`file.pdf`
5
6 # Download files by name
7 downloaded_files <- datasets.download_files("my_alias", c("file1.pdf", "f
8
9 # Download all PDF files of less than 1MB in the dataset
10 all_files <- datasets.list_files("my_alias")
11 pdf_files <- all_files[sapply(all_files, function(f) f.endswith(".pdf")) &
12 downloaded_files <- datasets.download_files("my_alias", pdf_files)

```

In both cases, `downloaded_files` will be a map from file name as defined in the current dataset view (which may contain slashes) to the local path where the file was downloaded. Note that this local path may change, so it is recommended to rely on the map keys.

[API Reference ↗](#)

Filter tabular datasets

[Send feedback](#)

Code Workspaces enables you to apply filters to datasets prior to loading them into memory. This reduces the memory consumption of the dataframes imported into the workspace, enabling you to focus on the subset of data relevant for your analysis. Code Workspaces provides the flexibility to work with a selection of columns, rows, or both.

When to use filters for tabular datasets

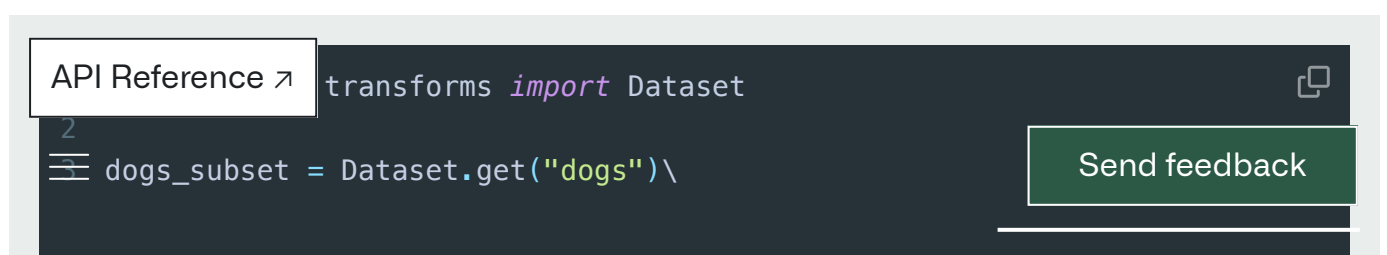
To ensure that you stay within your workspace's [available resources](#), we recommend the following:

- When your uncompressed data fits within your workspace's memory, you can load datasets without filters and apply transformations in-memory for maximum efficiency.
- When your uncompressed data exceeds your workspace's memory, you can use [column](#) and [row](#) filters to load a subset of the data into memory. These push-down filters are applied before data is loaded into the workspace, reducing the memory footprint of the imported data. The speed of these filter operations depends on the scale and partitioning of the data. You can also use [SQL with Python](#) to query tabular datasets of any size as long as the resulting data fits within your machine's memory limits.
 - If this approach does not fit your use case, you can also use a [Spark-based application](#) to process your data and produce a smaller dataset that you can work with in Code Workspaces. This often speeds up interactive workflows that depend on using the same subset of data frequently or across multiple workspaces, because Parquet files from datasets are downloaded only once while filtering occurs each time data is loaded into the workspace.
- For more complex data loading requirements, you can download the files backing a dataset using [non-tabular dataset syntax](#) and use native `Python` or `R` packages to process the file contents.

Row limit

It is possible to only load a limited number of rows from a dataset.

In Jupyter®:



The screenshot shows a Jupyter notebook interface. On the left, there is a sidebar with a button labeled "API Reference" and a magnifying glass icon. The main area is a code editor with a dark background. It contains a code cell with the following text: `transforms import Dataset` on the first line, and `dogs_subset = Dataset.get("dogs")\` on the second line. A line number "2" is visible to the left of the second line. In the top right corner of the code editor, there is a copy icon. In the bottom right corner, there is a green button labeled "Send feedback".

```
4 .limit(1000)\n5 .read_table(format="pandas")
```

In RStudio®:

```
1 library(dplyr) # should be imported by default in .Rprofile\n2\n3 dogs_subset <- Dataset.get("dogs") %>%\n4   head(1000) %>%\n5   collect()
```

Column filters

All tabular datasets can be loaded into a workspace with a subset of columns. Consider the following dataset as an example:

name	weight	color	age	breed
Bella	60	Brown	4	Labrador
Max	75	Black	7	German Shepherd
Daisy	30	White	2	Poodle

You may want to load this dataset only with the `breed` and `age` columns using the syntax below, assuming a `dogs` dataset was correctly registered into the workspace:

In Jupyter®:

```
1 from foundry.transforms import Dataset, Column\n2\n3 # Only load the "breed" and "age" columns\n4 columns_to_load = ["breed", "age"]\n5 breed_and_age_only = Dataset.get("dogs")\n6   .select(*columns_to_load)\n7   .read_table(format="pandas")
```

API Reference ↗

```
9 # Only load the "weight" and "color" columns\n10 weight_and_color_only = Dataset.get("dogs")\n   .select(*columns_to_load)\n   .read_table(format="pandas")
```

Send feedback

```
11 .select("weight", "color")\  
12 .read_table(format="pandas")
```

In RStudio®:

```
1 library(dplyr) # should be imported by default in .Rprofile  
2  
3 # Only load the "weight" and "color" columns  
4 weight_and_color_only <- Dataset.get("dogs") %>%  
5   select(weight, color) %>%  
6   collect()
```

Row filters

Tabular datasets can also be loaded into a workspace with a subset of rows that meet certain conditions.

i Row filters are applicable only to datasets in [Parquet](#) format. Other formats, such as CSV, allow for [column filters](#) but not row filters. With the help of a Foundry transform, most tabular datasets can be easily converted to Parquet format.

Recall the `dogs` dataset mentioned earlier:

name	weight	color	age	breed
Bella	60	Brown	4	Labrador
Max	75	Black	7	German Shepherd
Daisy	30	White	2	Poodle
Buddy	65	Yellow	3	Labrador
Gizmo	18	Brown	1	Pug

API Reference ↗ in Jupyter®

The syntax below can be used to filter datasets in Jupyter® at the row

Send feedback

You may only load brown-colored dogs from the `dogs` dataset using the following syntax:

```
1 from foundry.transforms import Dataset, Column
2
3 # Only load dogs of color "Brown"
4 brown_dogs = Dataset.get("dogs")\
5     .where(Column.get("color") == "Brown")\
6     .read_table(format="pandas")
```

Notice the use of `.where`, `select`, or `.limit` to pre-filter the dataset before it gets loaded into the workspace. These statements can be chained to apply several conditions at once:

```
1 # Only load dogs of color "Brown" and of breed "Labrador"
2 golden_dogs = Dataset.get("dogs")\
3     .where(Column.get("color") == "Brown")\
4     .where(Column.get("breed") == "Labrador")\
5     .read_table(format="pandas")
```

Below, you can find more examples of acceptable row filtering syntax supported in Jupyter® Code Workspaces:

```
1 # only retain rows equal to a certain value
2 .where(Column.get("column_name") == value)
3
4 # only retain rows not equal to a certain value
5 .where(Column.get("column_name") != value)
6
7 # inequality using the ~ operator
8 .where(~(Column.get("column_name") == value))
9
10 # only retain rows whose value is comparable to another value
11 .where(Column.get("column_name") > value)
12 .where(Column.get("column_name") >= value)
13 .where(Column.get("column_name") < value)
14 .where(Column.get("column_name") <= value)
```

[API Reference ↗](#)

```
15 # OR / AND operators
16 .where((Column.get("column_name") == value1) | (Column.get("column_name") == value2))
17 .where((Column.get("column_name1") == value1) & (Column.get("column_name2") == value2))
18
19
```

[Send feedback](#)

```

20 # only retain rows whose value is not null
21 .where(~Column.get("column_name").isnull())
22
23 # only retain rows whose value is part of a given list
24 .where(Column.get("column_name").isin([value1, value2, value3]))
25
26 # only retain rows whose date is between two given inclusive bounds
27 .where(Column.get("date_column_name").between('lower_bound_incl', 'upper_
28
29 # only retain the first N rows, where N is a number. This will be applied
30 .limit(N)
31
32 # select a subset of columns
33 .select("column_name1", "column_name2", "column_name3")

```

Row filter syntax in RStudio®

The syntax below can be used to filter datasets in RStudio® at the row level.

Rstudio filters are implemented through the use of the `dplyr` library and implement the standard methods `filter`, `select`, and `head`. These filters are **pushed down**, which means they are applied before the data gets loaded into the memory of the workspace.

You may load only brown-colored dogs from the `dogs` dataset using the following syntax:

```

1 library(dplyr) # should be imported by default in .Rprofile
2
3 # Only load dogs of color "Brown"
4 brown_dogs <- Dataset.get("dogs") %>%
5   foundry::filter(color == "Brown") %>%
6   collect()

```

Notice the use of `foundry::filter` to pre-filter the dataset before it gets loaded into the workspace. Technically, the `foundry::` prefix is not required, but we recommend to use it in order to avoid potential conflicts with other similarly named `filter` functions from other packages in your environment. These `filter` statements can be chained to apply several c

[API Reference ↗](#) using the `%>%` operator from the `dplyr` library. This library should be imported by default in the `.Rprofile` file of your RStudio workspace

[Send feedback](#)

```

1 library(dplyr) # should be imported by default in .Rprofile
2
3 # Only load dogs of color "Brown" and of breed "Labrador"
4 brown_labradors <- Dataset.get("dogs") %>%
5   foundry::filter(color == "Brown") %>%
6   foundry::filter(breed == "Labrador") %>%
7   collect()

```

Below, you can find more examples of acceptable row filtering syntax supported in RStudio® Code Workspaces. Column names must be passed to the `foundry::filter` function without wrapping them with quotation marks.

```

1 # only retain rows equal to a certain value
2 foundry::filter(column_name == "string_value") %>%
3 foundry::filter(integer_column_name == 4) %>%
4
5 # only retain rows not equal to a certain value
6 foundry::filter(column_name != value) %>%
7
8 # inequality using the ! operator
9 foundry::filter(!(column_name == value)) %>%
10
11 # only retain rows whose value is comparable to another value
12 foundry::filter(column_name > value) %>%
13 foundry::filter(column_name >= value) %>%
14 foundry::filter(column_name < value) %>%
15 foundry::filter(column_name <= value) %>%
16
17 # OR / AND operators
18 foundry::filter(column_name == value1 | column_name == value2) %>%
19 foundry::filter(column_name == value1 & column_name == value2) %>%
20
21 # only retain rows whose value is part of a given list
22 foundry::filter(column_name %in% c("value1", "value2")) %>%
23
24 # only retain rows whose value is not null
25 foundry::filter(!is.na(column_name)) %>%
26

```

API Reference ↗

```

27 # only retain rows whose value is between two given inclusive bounds
28 foundry::filter(between(age, 2, 4)) %>%

```

```

29
30 # select a subset of columns
31 select(column_name1, column_name2) %>% # if set, must include all columns

```

Send feedback

```
32
33 # only retain the first N rows, where N is a number. This will be applied
34 head(N) %>%
```

Additionally, you may perform advanced data transformations, such as `group_by`, by temporarily collecting the data as an `Arrow` table:

```
1 library(dplyr) # should be imported by default in .Rprofile
2
3 grouped_dogs <- Dataset.get("alias") %>%
4   # Simple filters can be pushed down
5   foundry::filter(age > 2) %>%
6   collect(as_data_frame = FALSE) %>% # temporarily collect the data as an
7   # Advanced transformations need to be applied on the arrow Table
8   group_by(breed) %>%
9   collect()
```

Column and row filters together

Column filters and row filters can be used together in order to load in a dataset that has both a subset of its columns and a subset of its rows. Using the `dogs` dataset mentioned earlier:

name	weight	color	age	breed
Bella	60	Brown	4	Labrador
Max	75	Black	7	German Shepherd
Daisy	30	White	2	Poodle
Buddy	65	Yellow	3	Labrador
Gizmo	18	Brown	1	Pug

The syntax below can be used to get a dataset with the name, breed, and color of brown dogs with a weight greater than 50.

[API Reference ↗](#)

In Jupyter®:

[Send feedback](#)

```

1 # Only load dogs whose color is "Brown" and whose weight is above 62
2 # Only load the columns "name", "breed", and "color"
3 heavy_brown_dogs = Dataset.get("dogs")\
4   .where(Column.get("weight") > 62)\
5   .where(Column.get("color") == "Brown")\
6   .select("name", "breed", "color")\
7   .read_table(format="arrow")\
8   .to_pandas()

```

In RStudio®:

```

1 library(dplyr) # should be imported by default in .Rprofile
2
3 # Only load dogs whose color is "Brown" and whose weight is above 62
4 # Only load the columns "name", "breed", and "color"
5 heavy_brown_dogs <- Dataset.get("dogs") %>%
6   foundry::filter(weight > 62) %>%
7   foundry::filter(color == "Brown") %>%
8   select("name", "breed", "color") %>%
9   collect()

```

Write data

i To schedule your data transformation, view the data lineage, or write incrementally. You can convert your code to a [Jupyter®](#) or [RStudio](#) transform.

You can interactively write Foundry datasets with Code Workspaces by following the steps below.

1. Create a target output Dataset by opening the **Data** tab and selecting the **Save to dataset** option, which can be found to the right of **Import Dataset**.

API Reference ↗

the output dataset as well as a location to save the dataset.

4. A new dataset will appear in the **Data** tab. By default, the **Save to** selected, which should be left as such for output datasets.

Send feedback

5. You will also be prompted to specify a dataset alias, which will become the name of the output dataset within the workspace, similarly to how aliases work when importing data.
 - For tabular output datasets, you will also be prompted to specify the dataframe variable which will populate the dataset.
 - For non-tabular datasets, you need instead to specify a local file or folder path to upload to the dataset.
6. Once the dataset type, the dataset alias, and the dataframe variable are set, select **Copy and register dataset** to register the dataset in the workspace, which will also save the code snippet to your clipboard.
7. Paste the code snippet in your workspace, replacing the variable as necessary, and execute the code to write to the output dataset.

Transaction types

When writing back interactively, each SDK function call will correspond to one transaction, by default:

- A **SNAPSHOT** transaction will be created when writing back tabular data (`output_dataset_tabular.write_table(df_variable)` in Python or `datasets.write_table(df_variable, "output_dataset_tabular")` in R).
- An **UPDATE** transaction will be created when writing back files (`output_dataset_non_tabular.upload_directory(path_to_file_variable)` in Python or `datasets.upload_files(path_to_file_variable, "output_dataset_non_tabular")` in R).

Once the script has been registered as a transform, interactive calls will start writing to a branch prefixed by `code-workspace-sandbox/`, while the current branch will be updated when the transform runs. In this case, a single transaction will be created for the full script execution, even if there are multiple SDK function calls:

- By default, the transaction will be of type **SNAPSHOT**.
- If incremental settings have been configured, transaction will be of type **APPEND**.

[API Reference ↗](#)

Example code snippets

[Send feedback](#)

Following the instructions above, assume that two datasets named `output_dataset_tabular` and `output_dataset_non_tabular` were created with variables of the same name, and registered in the workspace. Code Workspaces will generate the following code snippets for each dataset based on your chosen variables:

```
1 # tabular snippet
2 from foundry.transforms import Dataset
3
4 output_dataset_tabular = Dataset.get("output_dataset_tabular")
5 output_dataset_tabular.write_table(df_variable)
6
7 # non-tabular snippet
8 from foundry.transforms import Dataset
9
10 output_dataset_non_tabular = Dataset.get("output_dataset_non_tabular")
11 output_dataset_non_tabular.upload_directory(path_to_file_variable)
```

And in R:

```
1 # tabular snippet
2 datasets.write_table(df_variable, "output_dataset_tabular")
3
4 # non-tabular snippet
5 datasets.upload_files(path_to_file_variable, "output_dataset_non_tabular")
```

PREVIOUS

← Getting started

NEXT

Interact with external systems →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[API Reference ↗](#)

[Send feedback](#)

Interact with external systems

Code Workspaces allows you to use Python to interact with external systems that exist outside of the Foundry platform using [sources](#).

Sources allow you to store and access secrets, configure multiple egress policies at once, monitor usage in code across the platform, manage data export controls, and more. Because it is possible to egress Foundry data from within a code workspace to an external system, you must consider some [additional security considerations](#) when using sources with Foundry data.

⚠ Sources have replaced [network egress policies](#) as the preferred method of interacting with external systems for interactive workflows and publishing applications, except for [CBAC-enabled environments](#). **Sources are not yet supported for transforms published from a Code Workspace.**

Setting up a source

To interact with an external system in a Jupyter or RStudio® workspace, first create a source in the **Data Connection** application, provide an API name, and toggle the setting to allow the source to be imported into code repositories. After you add the source to your workspace and restart your workspace once, you can then interact with the source using Palantir's [external-systems Python library](#).

[API Reference ↗](#) [a Jupyter workspace](#)



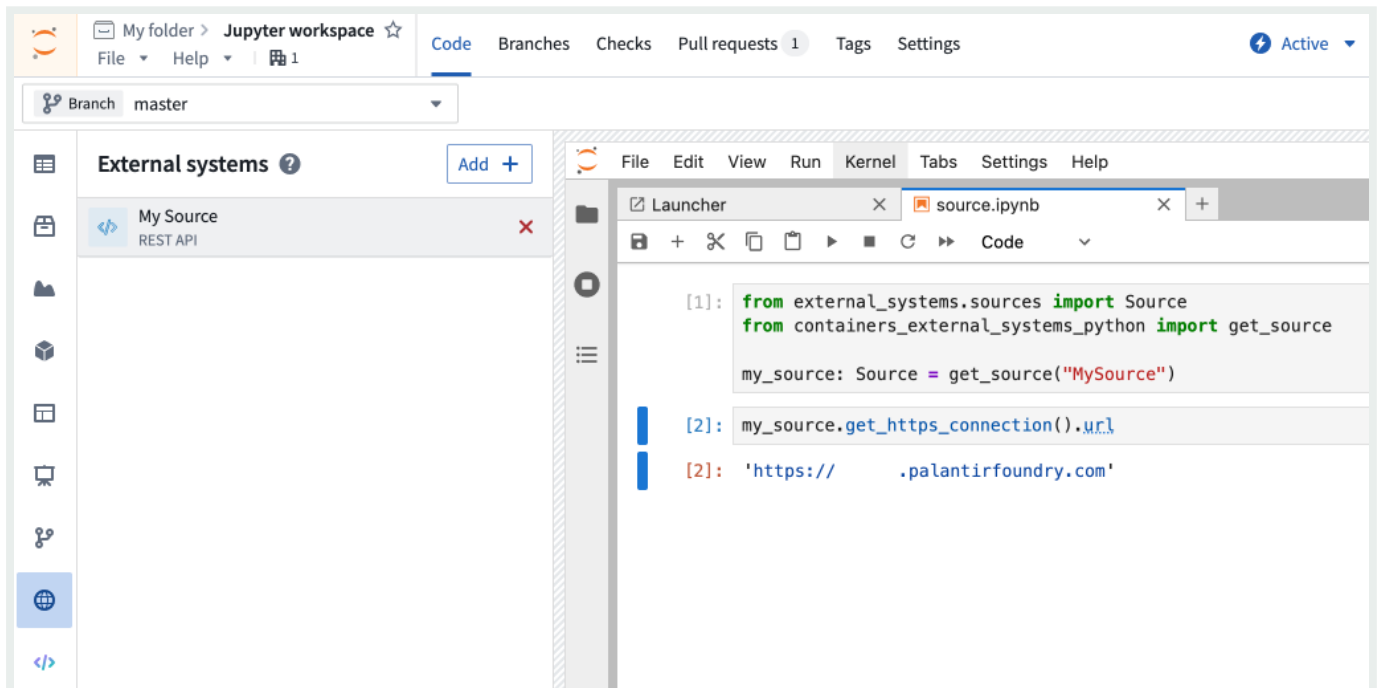
Send feedback

To use a source in a Jupyter workspace, ensure that the `containers-external-systems-python` library is installed. Then use the code snippets in the external systems side panel to interact with the source, starting with the following code:

```
1 from containers_external_systems_python import get_source
2
3 my_source = get_source("SourceApiName")
```

To retrieve credentials from a source in a Jupyter workspace, use the following approach:

```
1 some_secret: str = my_source.get_secret("secretName")
```



Sources in an RStudio® workspace

To use a source in an RStudio® workspace, ensure that the `containers-external-systems-python` library is installed along with `reticulate` and `python`. Then use the provided code

s for the `reticulate` adaptation of the Python syntax), such as in the

[API Reference ↗](#)

Send feedback

```

1 library(reticulate)
2
3 source <- import("containers_external_systems_python")$get_source("SourceA")
4 https_connection <- source$get_https_connection()
5 client <- https_connection$get_client()
6 response <- client$get(https_connection$url, timeout=10)
7
8 response$ok

```

To retrieve credentials from a source in an RStudio® workspace, use the following approach:

```

1 some_secret <- source$get_secret("secretName")

```

Sources in published applications

When publishing an application (e.g., [Streamlit](#)) that requires a source, you must include that source in the application's "advanced" configuration settings. Users must have access to all included sources or they will not be able to open the application.

Create new Streamlit application

Name

 Choose a name for the application.

Location
 [Change](#)
 Choose a location for the application.

Streamlit app path

 Path to the Streamlit app Python script, relative to the root of the

Advanced
 Configure the runtime settings of the application.

Sources
 Configure the sources this application can use when published.

Managed Conda environments
 Manage Conda environments for this application using specfiles and lockfiles.
☒ **Enable managed Conda environments**
 To learn more about using managed environments in this workspace, [view the documentation](#).

Faster reads for large tabular datasets and Views support
 Faster reads via Polars for large datasets backed by `.parquet` or `.csv` files, and enables loading of View datasets for this

API Reference ↗

Send feedback

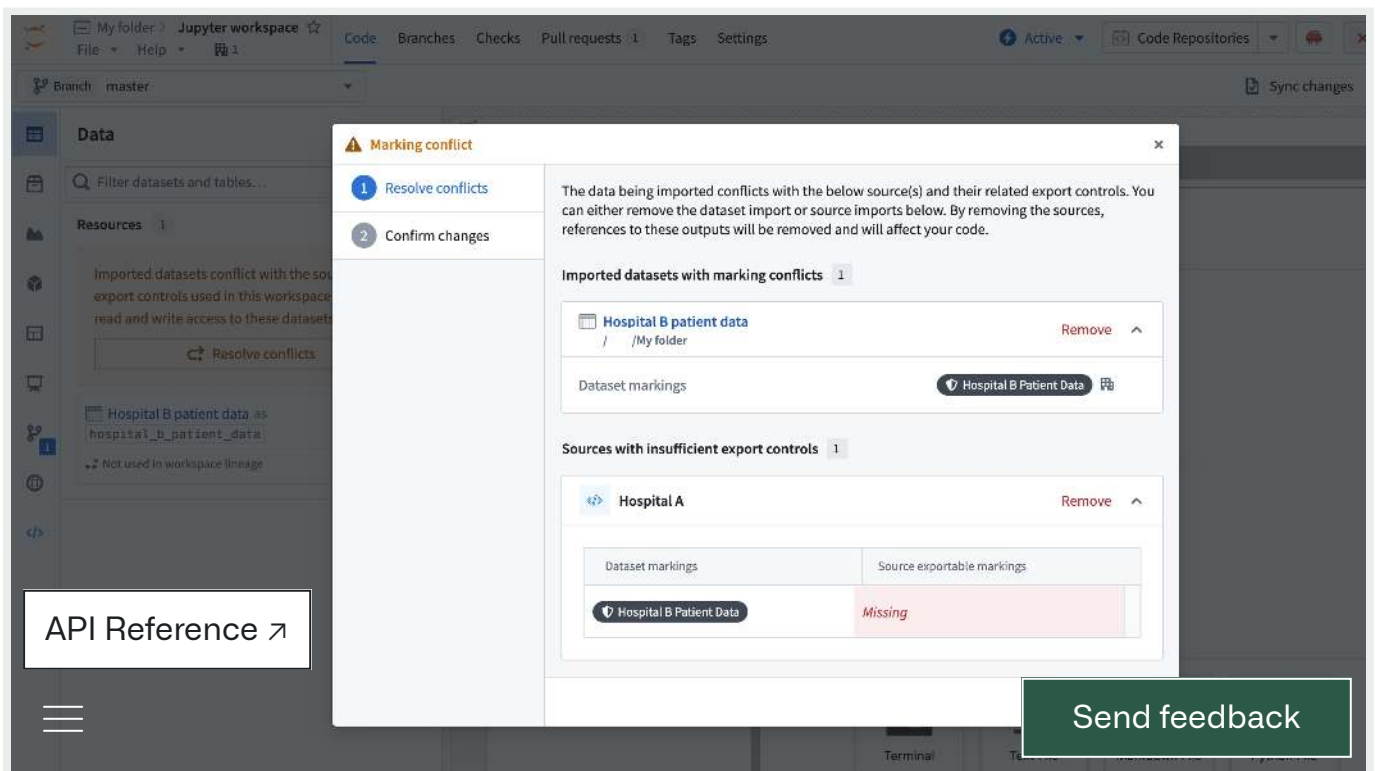
Sources and workspace security

Importing a source with exports enabled enforces additional checks on your workspace's security. Most importantly, **a dataset with a marking that is not on a source's exportable markings list cannot be used with that source in the same workspace.**

For example, if you had a source for an electronic health record system named `Hospital A`, you might be permitted to use it to export data with the marking `Hospital A Patient Data`. But suppose that you also had a dataset with the marking `Hospital B Patient Data` loaded into your workspace: if you were also using the `Hospital A` source, without checks you would be able to export disallowed `Hospital B Patient Data` to the `Hospital A` external system. Code Workspaces enforces these checks on the **level of the entire workspace**, also known as the *workspace lineage*, during workspace startup. That is, the incompatible source and dataset cannot be accessed in the workspace simultaneously.

If you have a source loaded into your workspace with an enforced exportable markings list and add a dataset with a non-exportable marking, the workspace will prevent you from accessing that dataset; for example, `Dataset.get()` will throw an error.

If you initialize or restart a workspace that includes an incompatible source and dataset, you will be prompted to remove either a source or dataset until the conflict resolves. You must then restart the workspace without a checkpoint to resolve this conflict.



© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Developer toolchain / Code Workspaces / Interact with the Ontology

Interact with the Ontology

⚠ [Restricted View-backed Ontology entities](#) and [query functions](#) are not yet supported in Code Workspaces.

Code Workspaces supports interacting with the [Ontology](#) using object, link, and action types in JupyterLab® and RStudio®.

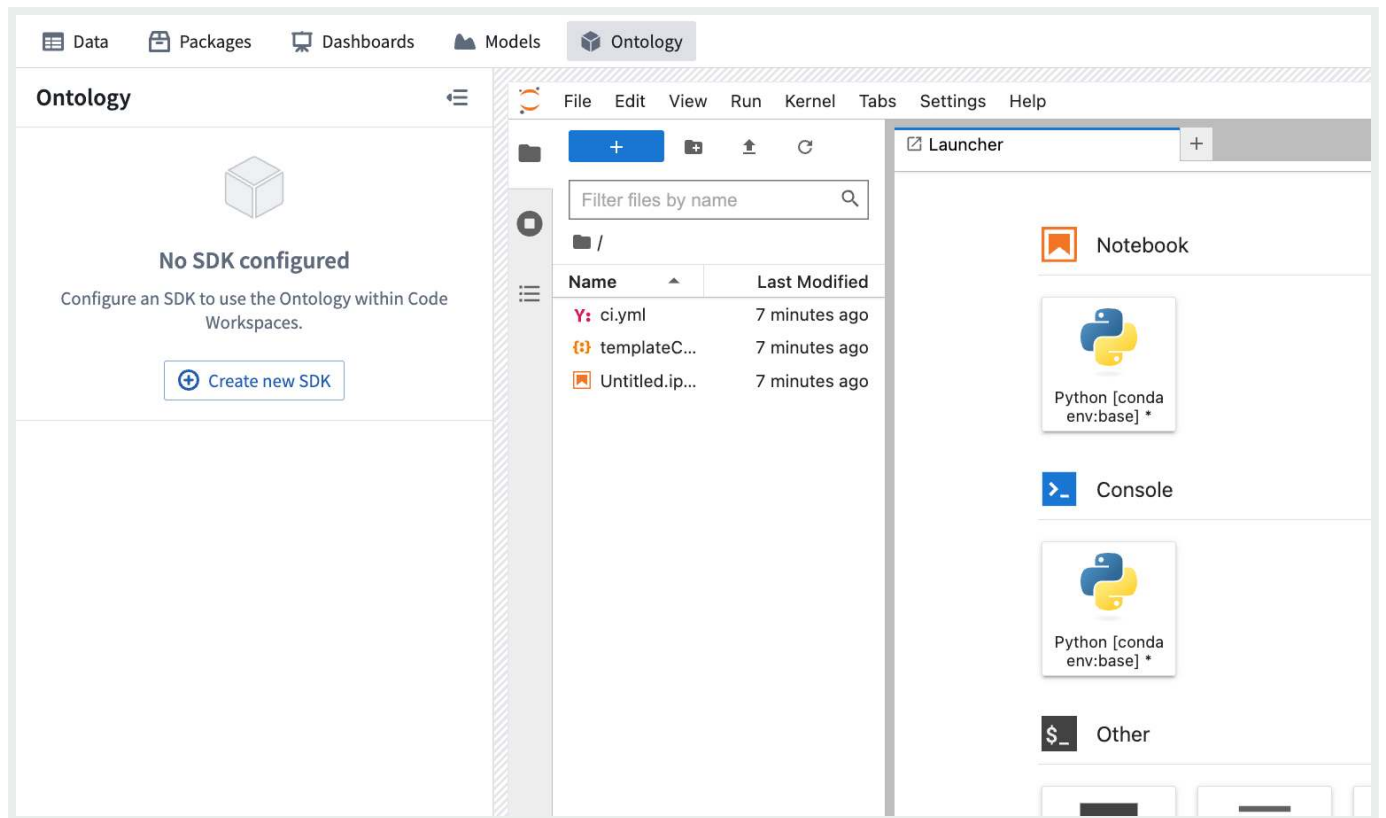
Create a new Ontology SDK

To create a new Ontology SDK in Code Workspaces, first navigate to the **Ontology** side panel. Each Code Workspace can have one SDK, and multiple versions of each SDK can be created. From the **Ontology** side panel, select **Create new SDK** to open the SDK configuration form.

API Reference ↗



Send feedback

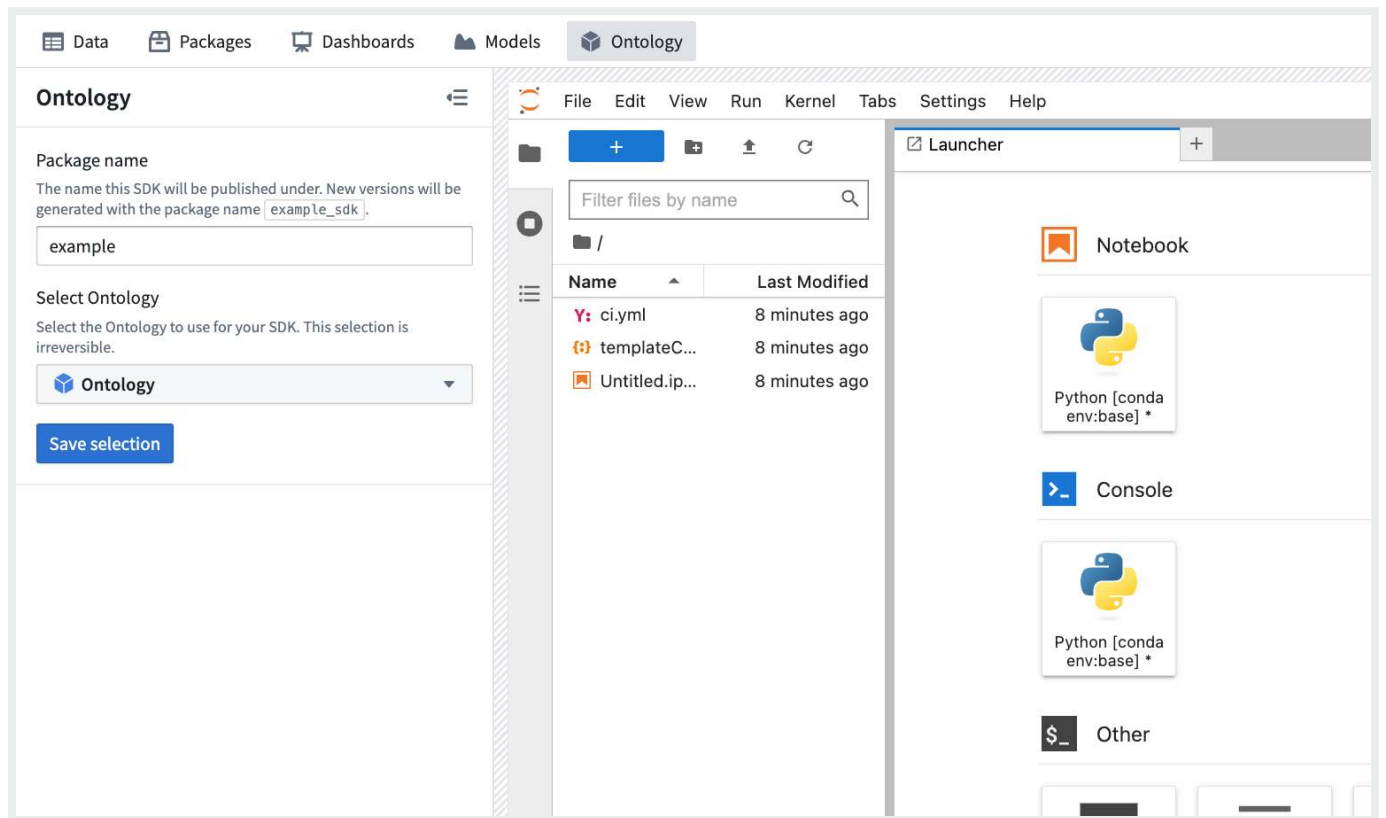


New Ontology SDKs require a package name and an associated Ontology. The package name is used to determine how your SDK is accessed in code: versions of an SDK with a package name of `example` would be importable under the library name `example_sdk`. SDK package names can only contain letters, numbers and hyphens, and cannot end with a hyphen. Once published, SDK package names cannot be changed.

After configuring your SDK, select **Save selection** to continue.

[API Reference ↗](#)

[Send feedback](#)



Select data entities

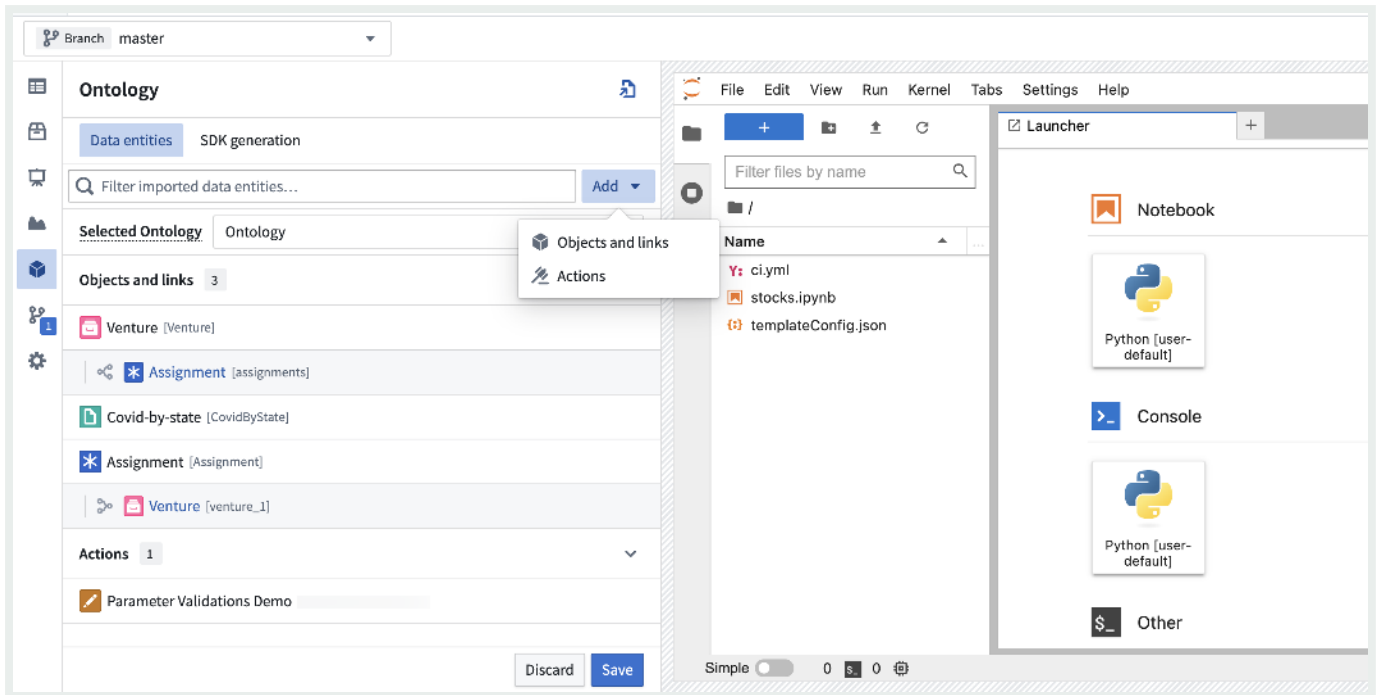
The **Ontology** side panel contains two tabs: **Data entities** and **SDK generation**.

- The **Data entities** tab is used to import object and action types from your Ontology.
- The **SDK generation** tab is used to inspect and install SDK versions.

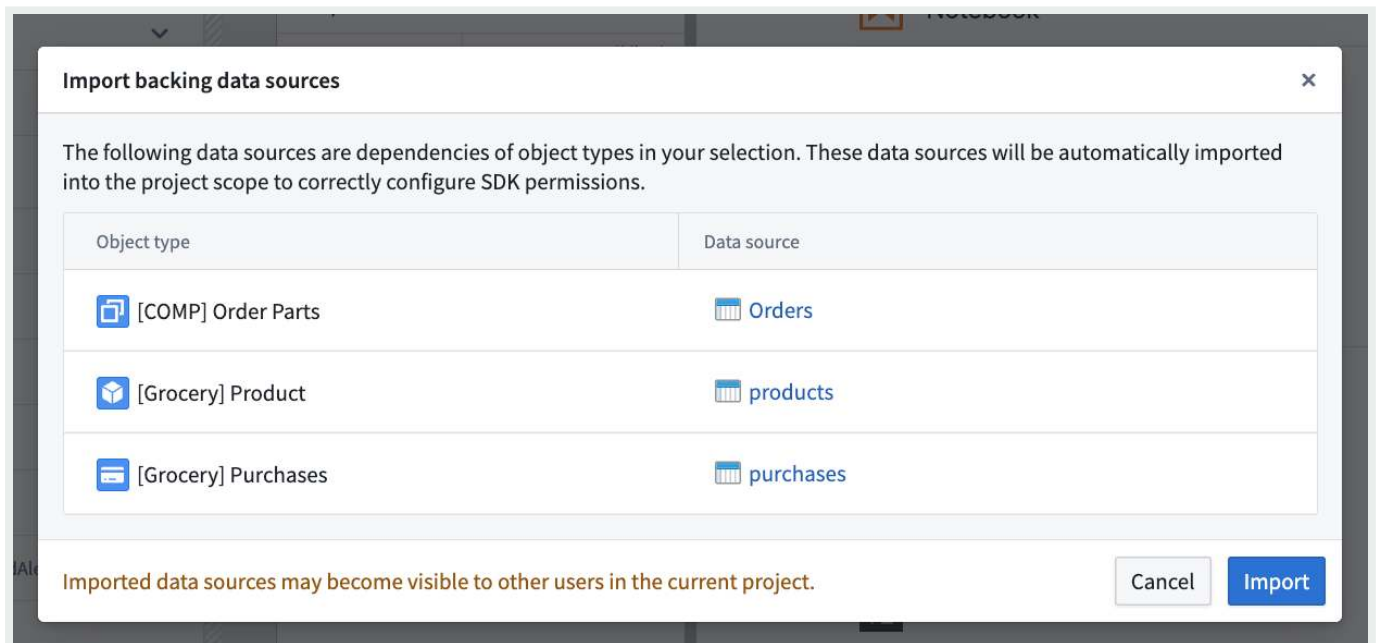
To select data entities, navigate to the **Data entities** tab. Select **Add** and choose the desired data entity type to open a resource selector dialog which enables browsing for and selecting data entities. After selecting data entities, your selection can be saved using the **Save** button at the bottom of the side panel. After selecting **Save**, a new SDK version will be generated with access to the selected data entities.

[API Reference ↗](#)

[Send feedback](#)



When selecting action types that affect object types or have object types as parameters, those object types will also be added to the selection automatically. If any selected object types have backing data sources which have not yet been imported into the project scope of your Code Workspace, you will also be prompted to confirm the automatic import of these data sources.

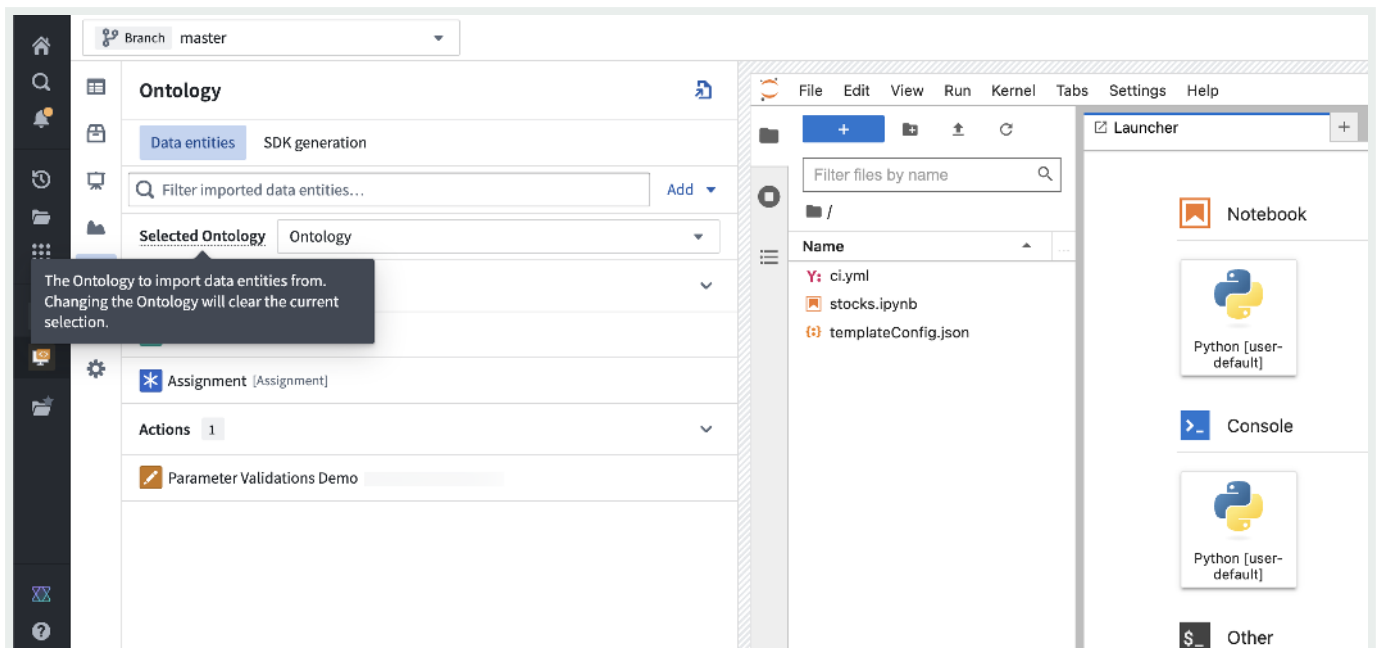


[API Reference](#) selected Ontology

To import Ontology entities from a different Ontology, use the Ontology **Data entities** tab. After selecting a different Ontology, select **Save** to apply changes and

[Send feedback](#)

generate a new version. Changing the selected Ontology will reset the data scope of the SDK, clearing any selected object, action or link types.

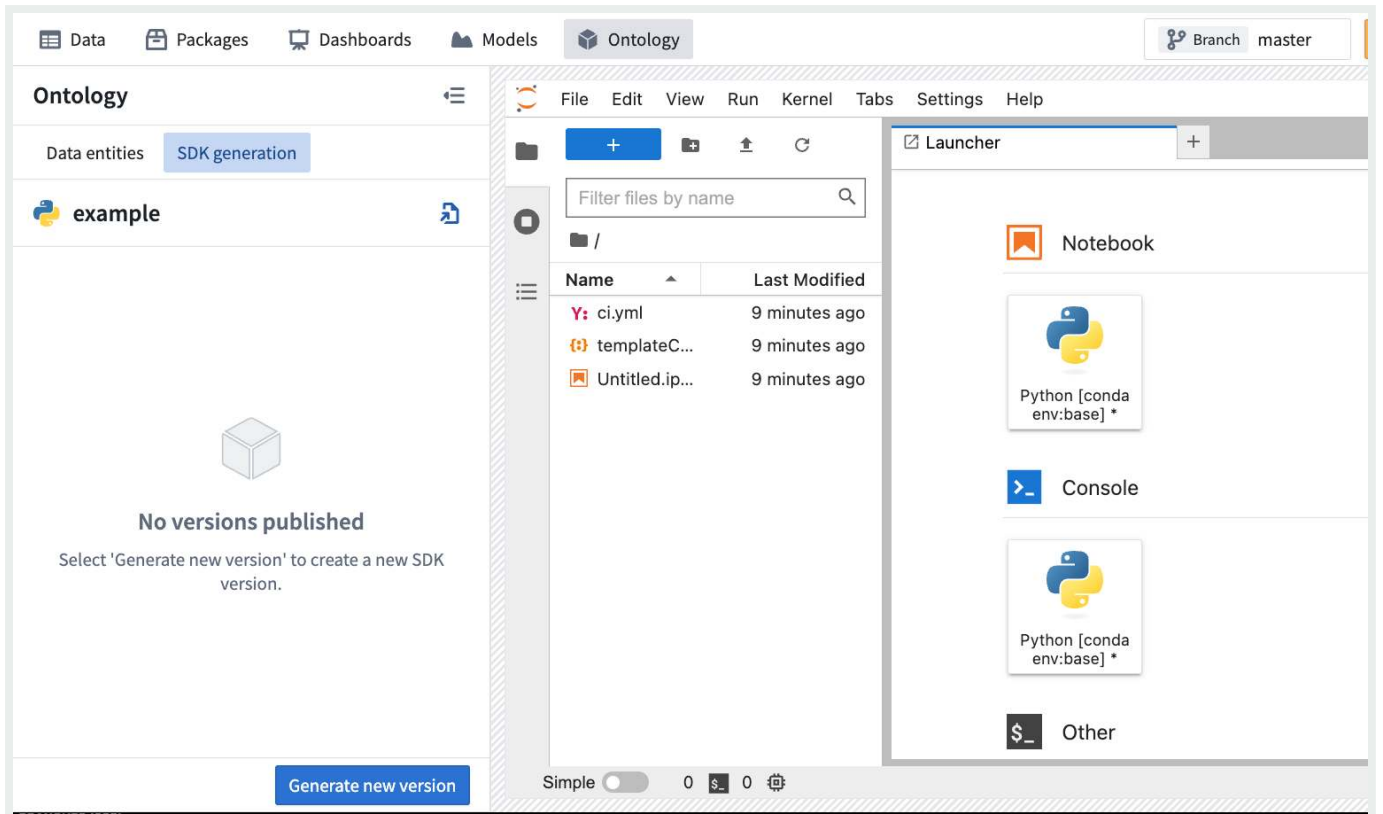


Publish new Ontology SDK versions

Once configured, new Ontology SDK versions can be published using the **Generate new version** button found at the bottom of the **Ontology** side panel. Published and pending versions can be inspected through the **SDK generation** tab.

[API Reference ↗](#)

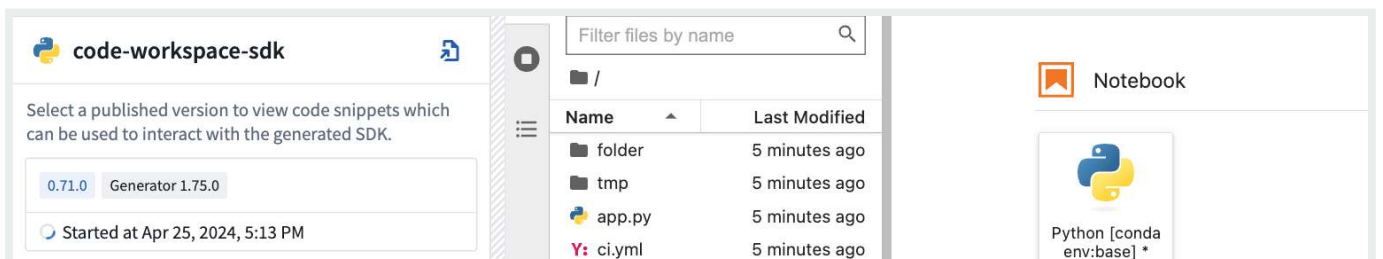
[Send feedback](#)



After selecting **Generate new version**, the newly created version will be visible in the **SDK generation tab**. SDK versions are not usable until the status shows as “Published”.

Common reasons for publishing to fail include:

- Using action types without importing dependent object types.
- Using object types that have backing data sources which are not present in the project scope.



Use Ontology SDK versions in code

API Reference ↗ ed versions can be imported and used in code from within your editor. First, click on the card for the version you want to use within the **SDK generation tab** of the **Ontology** side panel. This will expand the card with several code snippets

Send feedback

editor's language. Clicking on a snippet will add it to the clipboard so that it can be pasted into your editor.

The visible snippets will vary depending on your editor's language. All supported languages have the “Install package” snippet which is used to install a version of your SDK and the “Initialize client” snippet which is used to import the SDK and initialize a Foundry client to interact with the Ontology.

The screenshot shows the Foundry Ontology SDK interface on the left and a code editor on the right. The Ontology SDK interface has tabs for 'Data entities' and 'SDK generation'. Under 'SDK generation', it shows the 'code-workspace-sdk' with a version of 0.71.0 and a generator version of 1.75.0. It is published at Apr 25, 2024, 5:15 PM. There are three code snippets: 'Install package' with the command `!mamba install -y -q code_workspace_sdk_sdk==0.71.0`, 'Reload (optional)' with `%load_ext autoreload` and `%autoreload 2`, and 'Initialize client' with `from code_workspace_sdk_sdk import FoundryClient` and `client = FoundryClient()`. The code editor on the right shows a file explorer with a list of files and folders, including 'folder', 'tmp', 'app.py', 'ci.yml', 'my_dashb...', 'my_new_t...', 'my_third_...', 'templateC...', and several 'Untitled' files. The 'Launcher' panel on the right shows a 'Notebook' and a 'Console' tab, both with a Python [conda env:base] * environment.

Use the Ontology SDK

After the Ontology SDK has been successfully published, follow the instructions to install and import the SDK. You can run the following the command:

```
!mamba install -y -q my_sdk_package
```

Next, the following code snippet is used to initialize the FoundryClient, which will enable you to interact with the Ontology SDK.

[API Reference ↗](#)

[Send feedback](#)

```
1 from my_sdk_package import FoundryClient
2 client = FoundryClient()
```



Basic usage examples

Any imported object or link type can be selected in the **Data entities** tab to view usage examples for that specific resource. The following examples are longer, more general examples that demonstrate end-to-end OSDK usage in Code Workspaces.

Python OSDK usage example

The following example demonstrates how to interact with the OSDK using Python. This example assumes that the last successfully published version of your SDK includes object type "Aircraft" and action type "ActionMode". The term `my_sdk_package` in this example should be replaced with the package name of your SDK.

```
1 AircraftObject = client.ontology.objects.Aircraft
2
3 # Retrieve an object and view properties
4 my_aircraft = AircraftObject.get(1)
5 my_aircraft.date_of_manufacture
6
7 # Iterate over objects
8 for aircraft in AircraftObject.take(2):
9     print(aircraft.current_location)
10
11 # Aggregations
12 AircraftObject.aggregate({"min_id": Aircraft.id.min(),
13                           "max_id": Aircraft.id.max(),
14                           "aircraft_count": Aircraft.count()}).compute()
15
16 # Filter
17 my_a330s = AircraftObject.where((Aircraft.type == "A330") | (Aircraft.id
```



API Reference ↗

validate and apply

```
20 import datetime
21 from my_sdk_package.types import ActionMode
22
```

Send feedback


```

23 action_validation = client.ontology.actions.change_manufacture_date({"mod
24                                     airc
25                                     date
26 if action_validation.result == "VALID":
27     client.ontology.actions.change_manufacture_date(aircraft=1, date_of_m

```

Time series properties

To access [time series properties](#) in a workspace, follow the preceding steps to create, publish, and import an Ontology SDK. Then access time series properties and manipulate them with the [foundryTS library](#). This example shows how to retrieve the `Denver` object from a set of `City` objects and access its electricity price as a time series:

```

1 from my_sdk_package import FoundryClient
2 from foundryts import TimeSeries
3
4 client = FoundryClient()
5 city_objects = client.ontology.objects.City
6
7 denver = city_objects.get("Denver") # This is the object's primary key
8 denver_electricity_price_tsp = TimeSeries.from_osdk(denver.electricity_pr
9
10 # Convert the series into a foundryts node
11 node = denver_electricity_price_tsp.node()
12
13 # Convert the series into a pandas dataframe
14 df = denver_electricity_price_tsp.to_pandas()

```

R OSDK usage example

Code Workspaces supports OSDK in R via the [reticulate](#) package, which allows you to call Python from R.

This example demonstrates how to import a Python OSDK version into R and interact with

an example assumes that your SDK includes an object type called `"API Reference ↗"`. `my_sdk_package` should be replaced with the package name of your SDK.

Send feedback

```
1 library(reticulate)
2
3 osdk <- import("my_sdk_package")
4 client <- osdk$FoundryClient()
5
6 aircraft_object = client$ontology$objects$Aircraft
7
8 # Retrieve 1 object
9 aircraft_object$take(1L)
10
11 # Retrieve object by ID
12 aircraft_object$get("1")
```

As R and Python have different default numeric types, the L suffix has to be used within R when a Python API expects an integer. [Learn more at the official documentation for reticulate.](#) ↗

Jupyter®, *JupyterLab®*, and the *Jupyter®* logos are trademarks or registered trademarks of NumFOCUS.

RStudio® and *Shiny®* are trademarks of Posit™.

All third-party trademarks (including logos and icons) referenced remain the property of their respective owners. No affiliation or endorsement is implied.

PREVIOUS
← [Interact with external systems](#)

NEXT
[Train and interact with models](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[API Reference](#) ↗

[Send feedback](#)

Train and interact with models

i Interacting with models is only supported in [JupyterLab® code workspaces](#).

Code Workspaces allows you to train and publish models, as well as import them to interact with previously published models.

Code Workspaces requires you to create a **model alias** for every Foundry [model asset](#) referenced in the workspace. The model alias acts as a reference to a specific model, allowing you to interact with models to publish new versions or download the saved model files.

When registering a model in the **Models** tab, Code Workspaces creates a mapping between the chosen model alias and the Foundry model's unique identifier in a hidden file located under the `/home/user/repo/.foundry` folder of the workspace.

To learn how to publish a model from Jupyter® notebooks, review the [model integration documentation](#).

Add models to the workspace

Models can be created or imported into the workspace all from the **Models** sidebar panel.

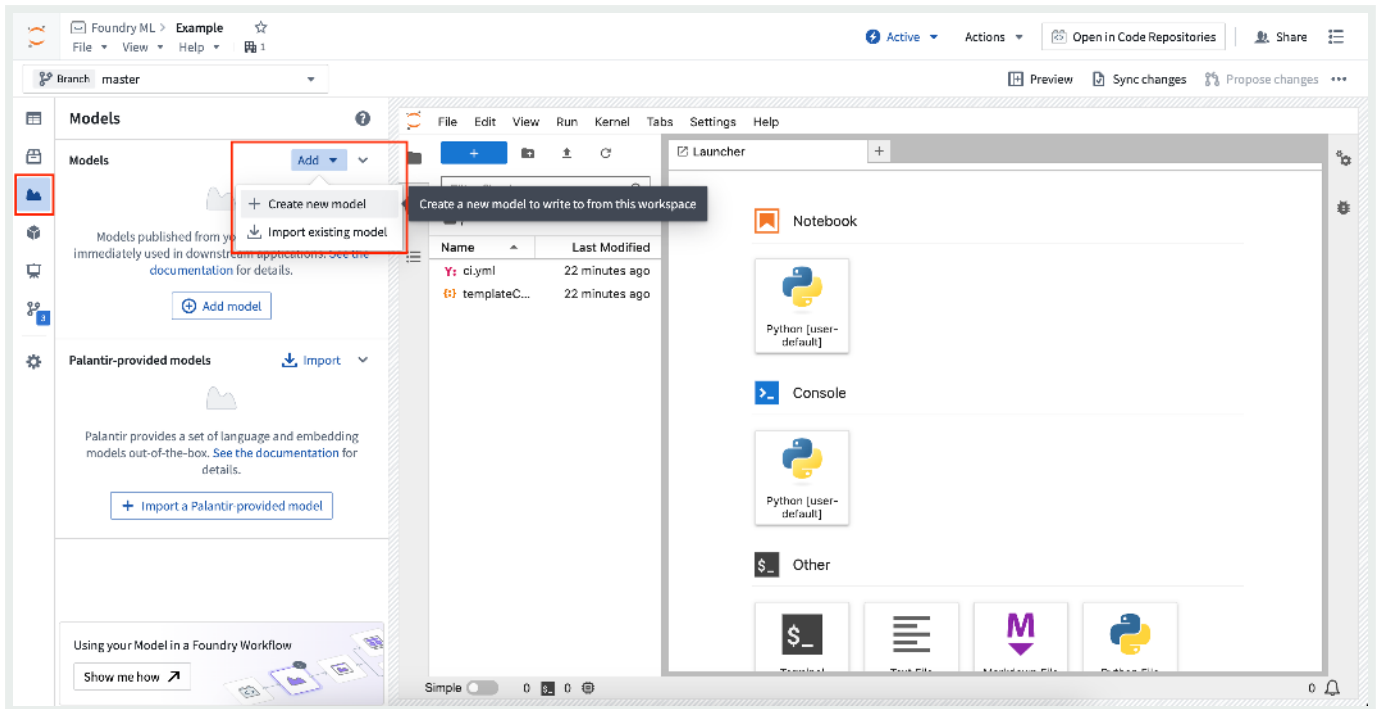
To create a new model, open the **Models** sidebar panel and select **Add > Create new model**.

This process involves creating a new model, registering the alias, and will provide a skeleton [model adapter](#) to the model. This will also generate a skeleton [model adapter](#)

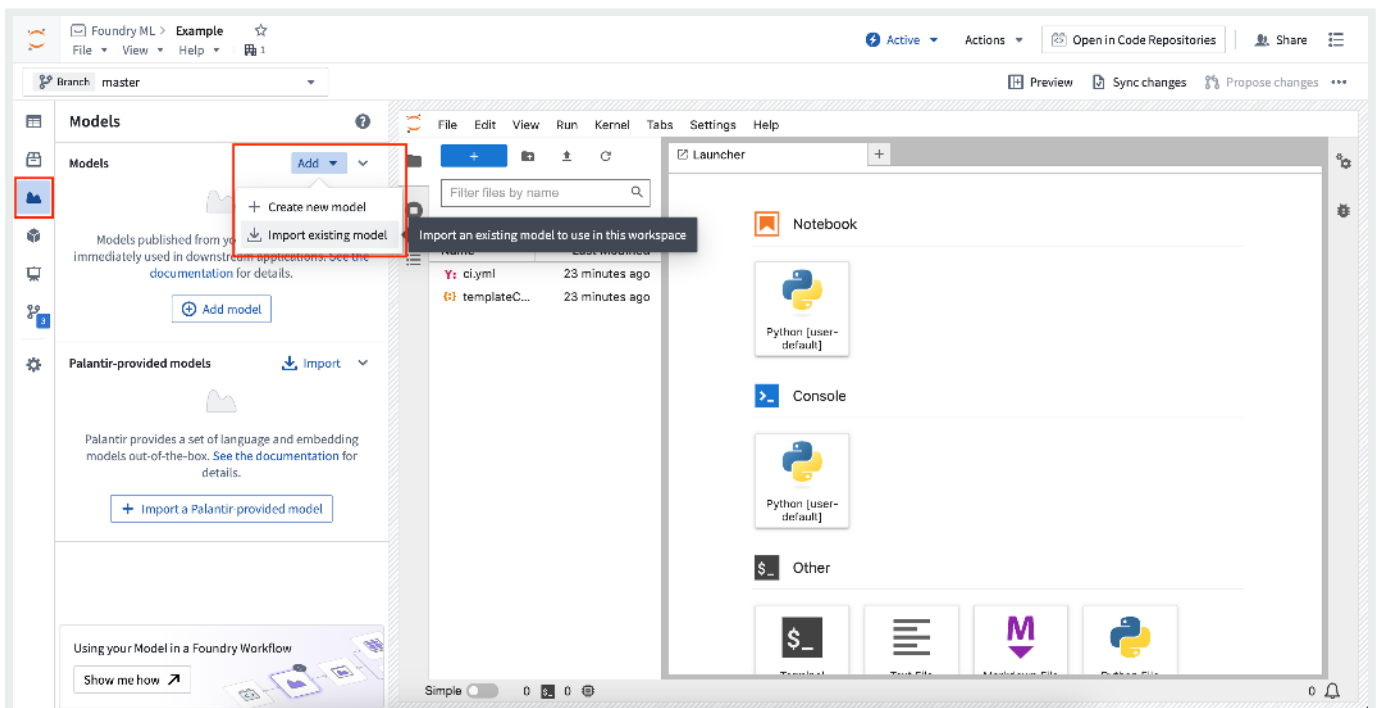
which must be implemented to publish the model back to Foundry.

API Reference ↗

Send feedback



To import an existing model, open the **Models** sidebar panel and select **Add > Import existing model**. This will walk you through importing the model and registering the alias, and will provide snippets for downloading the saved model state files.



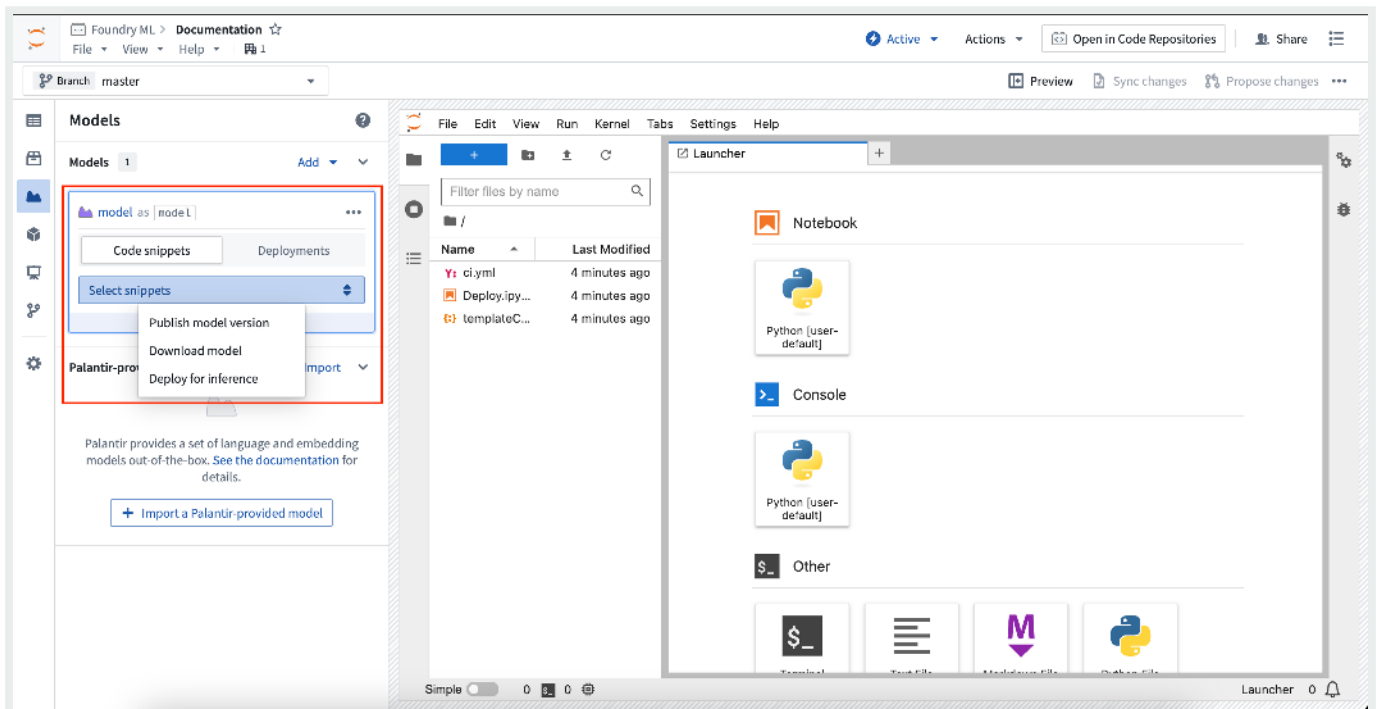
Imported models load the serialized model state from a given model version. Code Workspaces gives different controls over which model version is used when importing a model. [API Reference](#) to select the latest on the current workspace branch, the latest on a provided branch, or a specified model version.

[Send feedback](#)

Available snippets

The **Models** sidebar panel provides snippets to copy and paste into a notebook for common workflows. All snippets will instruct you to install `palantir_models`, which is the Palantir-provided library for developing and interacting with models.

Each model card in the sidebar will create unique snippets specific to that model. Selecting the model card will expand the card and reveal a selector to choose to show snippet for a specific task.



Publish model version

To publish a model version, you must first [develop a model adapter](#). For models created inside Code Workspaces, a skeleton adapter will be created for you to implement in a Python file named after the alias given to the model. The model card will provide in-depth instructions on [developing your model adapter](#) and [publishing your model back to Foundry](#).

The snippet below is for a model with the alias `linear_regression_model`. Assuming a model adapter has been written in `linear_regression_model_adapter.py`, the model p
API Reference ↗

Send feedback

```

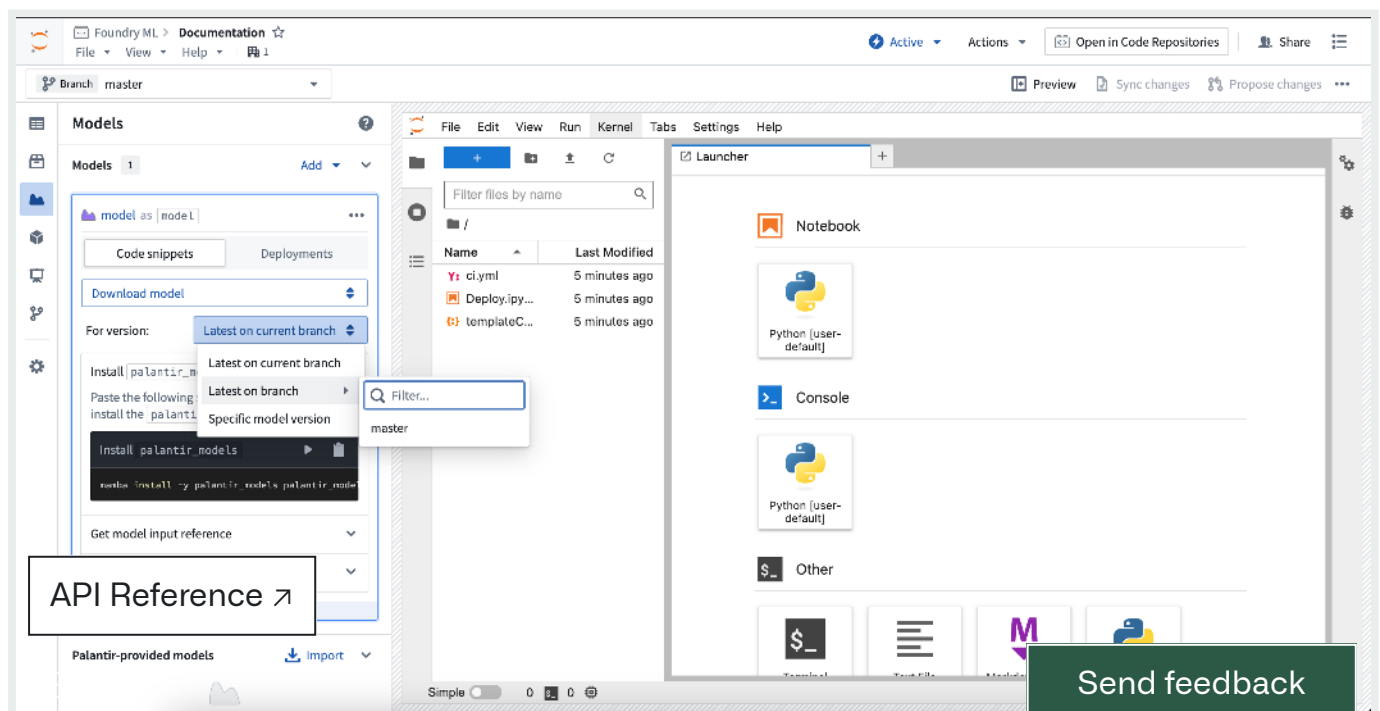
1 from palantir_models.code_workspaces import ModelOutput
2
3 # Model adapter api has been defined in linear_regression_model_adapter.py
4 from linear_regression_model_adapter import LinearRegressionModelAdapter
5
6 # sklearn_model is a model trained in another cell
7 linear_regression_model_adapter = LinearRegressionModelAdapter(sklearn_model)
8
9 model_output = ModelOutput("linear_regression_model")
10 model_output.publish(linear_regression_model_adapter)

```

Download model

To download a previously published model version, change the snippet selector to **Download model**. This will provide controls on resolving a model version, which will control which model version gets used for downloading the model files.

- **Latest on current branch:** The selected model version will be the latest on the current workspace branch.
- **Latest on branch:** The selected model version will be the latest on the selected branch.
- **Specific model version:** The selected model version will be pinned to the given model version RID.



The snippets will automatically update to support the selected model version resolution control. Review the [ModelInput](#) class documentation below to learn more.

Deploy model for inference

Models can be deployed in Code Workspaces for inference. These deployments are shared per model version, meaning a given model version can have at most one associated deployment, and these deployments are fully separate from existing [modeling objective live deployments](#) and [model deployments](#). Deployments are fully configurable from code and have a default scaling policy that will shut down the deployment after 20 minutes of no use.

To load the deployment for a given model version, use the **Select snippets** dropdown menu and change the option to **Deploy for inference**. This will load the snippets for deploying the model and will display the same selector as the **Download files** option for choosing a model version. The snippets will walk through deploying the model and running inference against it.

To monitor a deployment from within Code Workspaces, expand the model card for the model you are working with and select **Deployments**. In the deployments tab, a list of deployed model versions will be available. Selecting a model version will expand a new panel on the right that contains information about the deployment, such as resources used, logs, and more.

The screenshot displays the Foundry ML Code Workspaces interface. On the left, the 'Models' sidebar shows a list of models. The 'model as model' card is selected, and the 'Deployments' tab is active, showing a deployment for 'Thu, May 2, 2024, 1:41:23 PM'. A red box highlights this deployment entry. Below the sidebar, there is a section for 'Palantir-provided models' with an 'Import' button and a link to 'API Reference'.

The central pane shows a Jupyter Notebook with the following code:

```
[1]: from palantir_models.code_workspaces import ModelInput
model = ModelInput("model")

[2]: model_client = model.deploy(cpu=0.5, memory="4G", ma

[ ]: model_client.predict(10)
```

The right pane shows the 'Details' tab for the selected deployment, displaying the following information:

Model version	r.models.main.model-version.54fd1914-2caa-4442-8ef9-65daf08bc9f
CPU	0.5
Memory	4G
Max replicas	1

Below the details, the 'Logs' tab is active, showing a table of log entries:

level	time	message	par
INFO	2024-09-25 16:43:18.584	server starting	{ "lir
DEBUG	2024-09-25 16:43:18.584	error opening minid...	{ "err
INFO	2024-09-25 16:43:18.907	Started Python inter...	{ "ver
INFO	2024-09-25 16:43:18.907	Setup python logging...	{ "lir
INFO	2024-09-25 16:43:18.907	Testing normal non-w...	{ "lir
DEBUG	2024-09-25 16:43:18.907	Python interpreter s...	{ "lir
INFO	2024-09-25 16:43:23.932	Loaded palantir_model...	{ "ver
DEBUG	2024-09-25 16:43:23.933	could not open conta...	{ "lir
DEBUG	2024-09-25 16:43:23.933	Got model config	{ "cor

At the bottom right, there is a green button labeled 'Send feedback'.

See the [ModelInput](#) class documentation below to learn more.

Develop model adapters

Model adapters are used to instruct Foundry on how to save, load, and execute a model across the platform. When publishing a model, you must wrap the model with a model adapter.

In Code Workspaces, model adapters must be defined in a separate `.py` file. When a new model is added to the workspace, a file is generated based on the alias name given. For example, if the alias is `my_model`, a file named `my_model_adapter.py` is generated, with a skeleton model adapter class named `MyModelAdapter`.

The code snippets provided in the **Models** panel are generated under the assumption that the alias, file name, and class name match. If the file is moved or the adapter class is renamed, you may have to make manual edits to the generated snippets.

[Learn more about developing model adapters.](#)

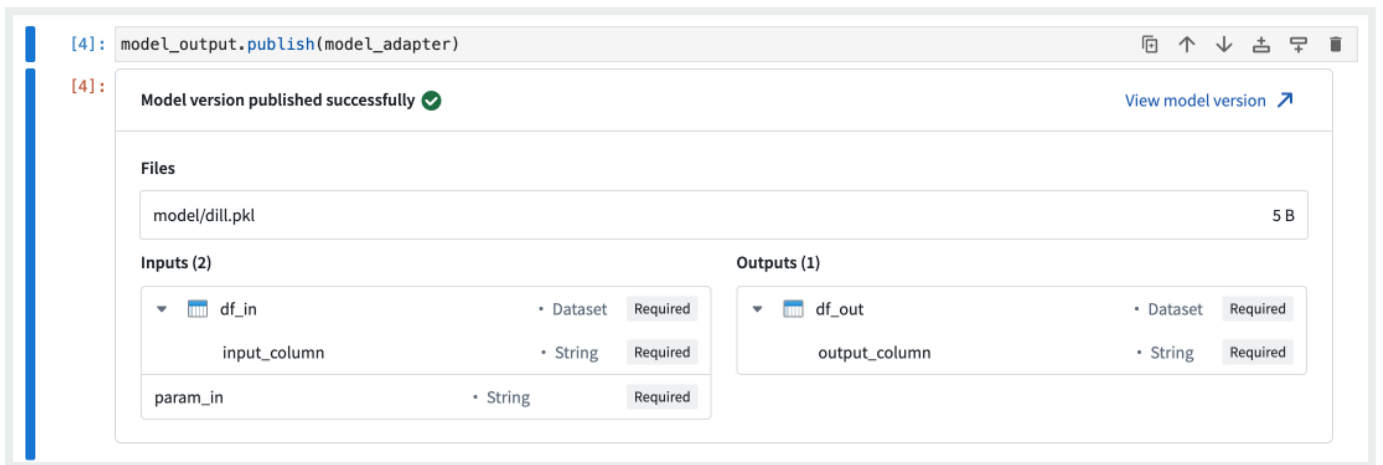
Model output class

The `ModelOutput` class imported from `palantir_models.code_workspaces` provides a writeable reference to your Foundry model. To publish your model, you must wrap the model in a model adapter and pass to the reference with the `publish` method.

Calling the `publish` method will save the model as a new model version. If the `model_output.publish` method call is the **last** line in any Jupyter® notebook cell, a preview window will be displayed confirming that the model has been published successfully.

API Reference ↗

Send feedback



Model input class

The `ModelInput` class imported from `palantir_models.code_workspaces` provides a lightweight readable reference to your Foundry model which allows reading the serialized state of a published models and deploying the model locally for inference. The code below shows an example of getting a readable reference to a model with the alias `my_model`, getting the latest model version on the `test` branch:

```
1 from palantir_models.code_workspaces import ModelInput
2 my_model_input = ModelInput("my_alias", branch="test")
```

Download models

`ModelInput` allows downloading the serialized model weights that are stored when the model is published.

```
1 from palantir_models.code_workspaces import ModelInput
2 # get the latest model version
3 my_model_input = ModelInput("my_alias")
4
5 # download the files
6 my_model_input.download_files()
```

[API Reference](#)

[Send feedback](#)

This returns a path to a temporary directory where the files are downloaded.

Initialize model adapters

For workflows involving model fine-tuning/republishing, it is often useful to be able to download the model files and load them into an adapter. If the workspace has a local copy of the adapter used to publish the model or the adapter comes from a shared adapter package and you have added it as a dependency in the workspace, you can load the adapter and use the adapter as you would anywhere else.

```
1 # in this example, the adapter is in a local python file `my_adapter_file`
2 from my_adapter_file import MyAdapter
3 from palantir_models.code_workspaces import ModelInput
4
5 my_model_input = ModelInput("my_alias", model_version="ri.models.main.model")
6
7 # initialize the adapter
8 initialized_adapter = my_model_input.initialize_adapter(MyAdapter)
9
10 # the initialized adapter can now be republished using `ModelOutput`, or
11 # for example, if there is a variable `pipeline` that holds a huggingface
12 # adapter definition.
13 pipeline = initialized_adapter.pipeline
```

Deploy models for inference

Models can also be deployed for inference within the code workspace. These deployments are shared per model version, so multiple concurrent sessions using the same model version will send requests to the same deployment.

To interact with a model deployment, you must get a deployment client for the model.

```
1 from palantir_models.code_workspaces import ModelInput
2
3 my_model_input = ModelInput("my_alias")
4
5 client = my_model_input.deploy(cpu=2, memory="8G", max_replicas=1)
```

API Reference ↗

Send feedback

The deployment client has a number of methods available for controlling the deployment.

```
1 # wait until the model is ready
2 client.wait_for_readiness()
3
4 # scale the deployment
5 client.scale(cpu=4)
6
7 # run inference
8 client.predict(...)
9
10 # manually shutdown (deployments will auto-shutdown after 20 minutes of r
11 client.disable()
```

Inference input size limit

There is a 40 MB limit on the size of a single inference request, as this approach is intended for quick iteration involving other models rather than production-scale inference. In practice, this often corresponds to an average dataset of around 100,000 rows, depending on row size.

If you need to run your model over larger datasets, you can enable *chunking*, which breaks down a large dataset into manageable pieces and sends them to the model one chunk at a time. You can enable chunking by setting the `client.use_chunking` method to `True` as follows:

```
1 client.use_chunking = True
2 client.predict(...)
```

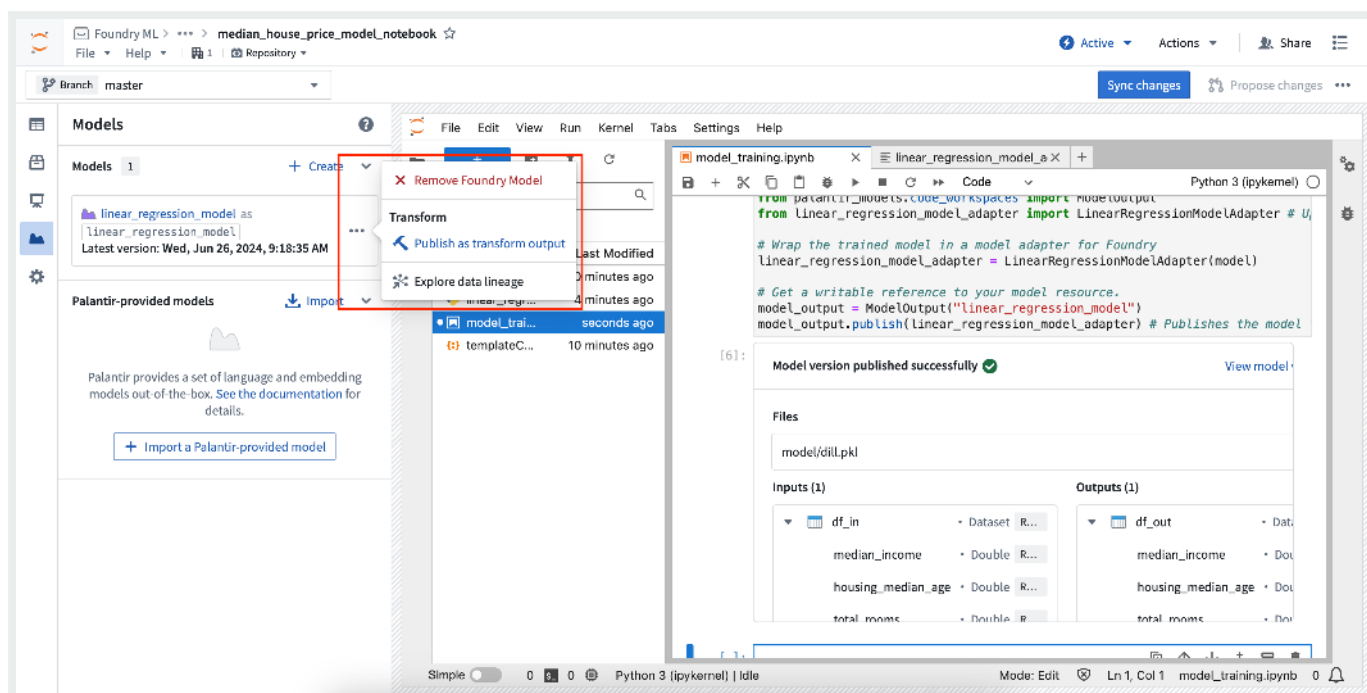
Note that chunking currently supports only one dataset as input, but you can include any number of other parameters. If your dataset exceeds these limits or the chunking approach is insufficient for your workflow, consider using Code Repositories and transforms to handle larger-scale data processing.

Transforms

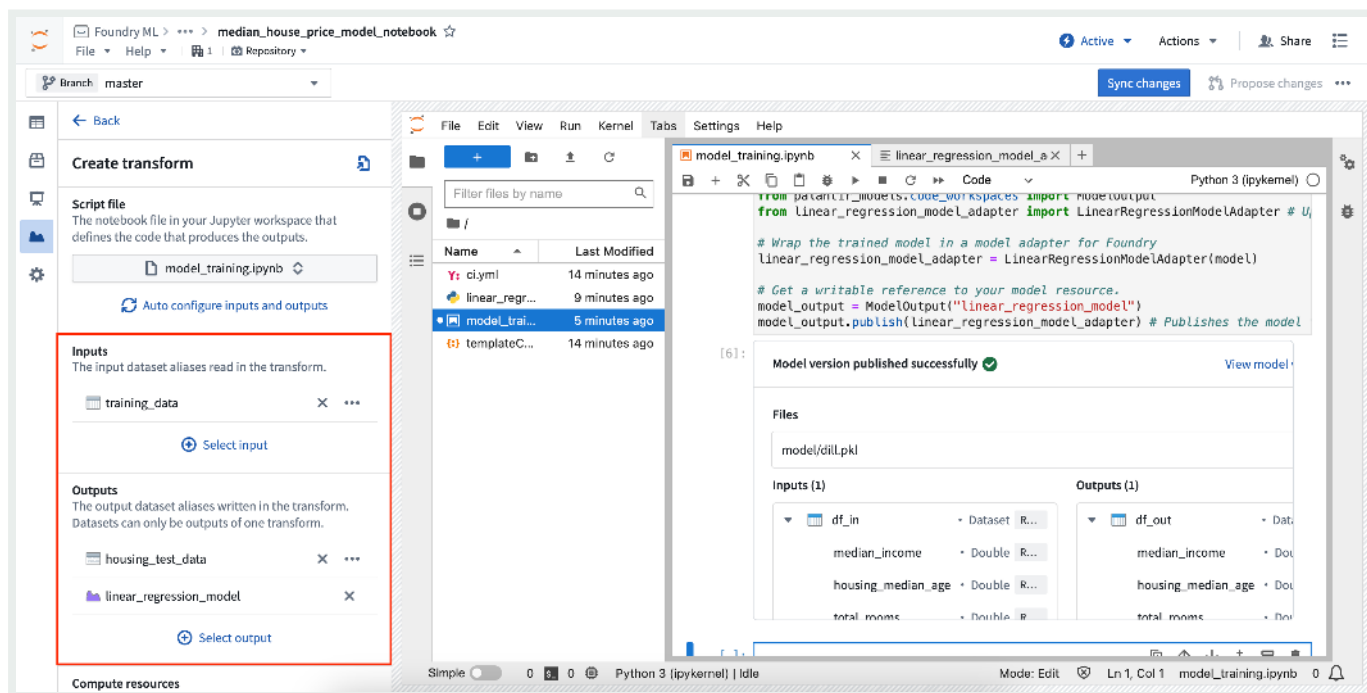
[API Reference ↗](#)

Jupyter® notebooks that produce model outputs can also be used in [Send feedback](#) from Code Workspaces. Follow the steps below to register a transform for a model output:

1. In the **Models** panel, find the model card for the model you want to output from a transform, then select **Publish as transform output** in the menu.



2. In the **Create transform** panel, you can configure the file to execute as well as the required inputs and outputs of your transform. In the example below, training data is used as an input and a test dataset and model are used as outputs.

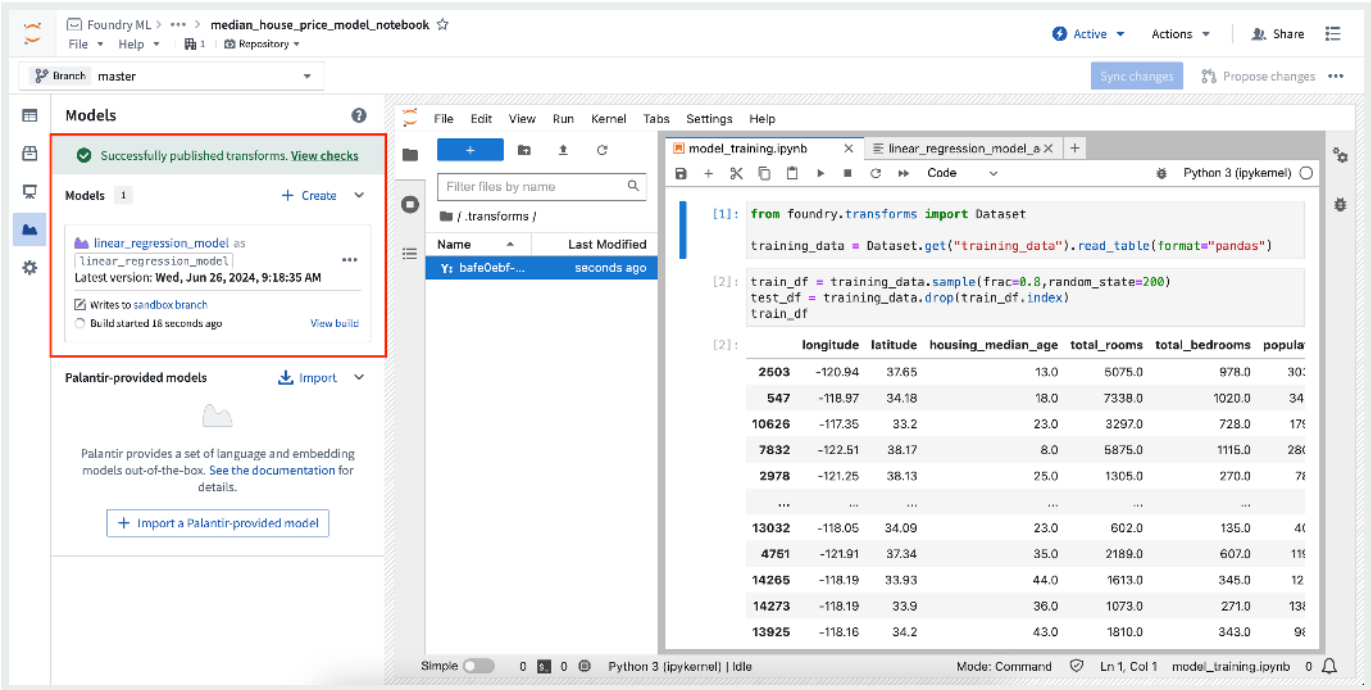


API Reference ↗

and outputs are configured, select **Sync and run** at the bottom of the configuration page to save your current code changes and create

Send feedback

transform will start running, and the status of the job will be displayed with the associated datasets and models in the **Models** panel.



You can then select **View build** to see the running transforms job. If you make a change to your notebook code and want to rebuild the model, select **Rebuild** in the model card to re-run the build. Models and datasets created with a transform can be scheduled just like transforms created elsewhere in the platform. [Learn more about scheduling transforms.](#)

Jupyter®, JupyterLab®, and the Jupyter® logos are trademarks or registered trademarks of NumFOCUS.

All third-party trademarks (including logos and icons) referenced remain the property of their respective owners. No affiliation or endorsement is implied.

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Developer toolchain / Code Workspaces / JupyterLab®

JupyterLab®

Code Workspaces enables you to use [JupyterLab® ↗](#) in Foundry. JupyterLab® in Code Workspaces supports:

- Reading and writing tabular datasets, [Iceberg tables](#), and [virtual tables](#).
- Downloading or uploading files from non-structured datasets.
- Importing Python libraries from conda or PyPI.
- Previewing Dash, Streamlit, or TensorBoard™ applications to facilitate iteration.
- Publishing Dash or Streamlit applications.
- Interacting with the Ontology.

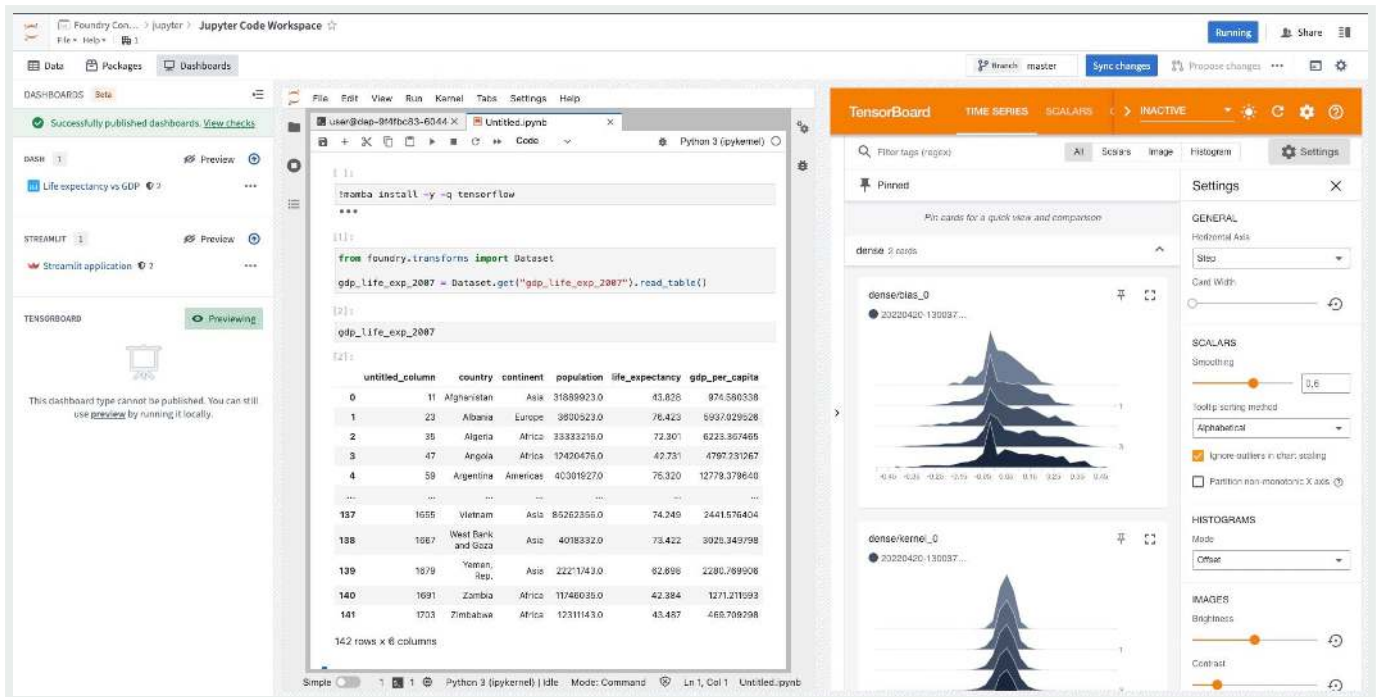
Code Workspaces currently supports [Dash ↗](#) and [Streamlit ↗](#) for Python applications. Users can create applications directly in Code Workspaces with Foundry's [version control](#), [branching](#), and [data governance](#) features built-in.

i Code Workspace applications support branching. If you [create a new Workspace branch](#), publishing a new application or synchronizing the changes will publish a new version of the application on that branch. This allows you to preview your application before exposing it to your users. To publish on the master branch, simply merge your branch into master.

API Reference ↗



Send feedback



Dash applications

⚠ Published Dash applications are subject to a 30-second timeout, meaning that the Dash server must start up within 30 seconds of executing your Dash application Python file. Otherwise, your Dash application will fail to start up.

To create a [Dash](#) application, follow the instructions below:

1. Open the **Applications** tab in a JupyterLab® workspace.
2. Select **Publish application**, choose a location in your **Files and Projects** for your new application, and provide a module name and variable name. A file will be automatically created at the path corresponding to the provided module name.
3. *Optional:* Configure advanced settings. By default, the application settings will match the settings of the Jupyter® workspace.
4. Select **Publish and sync** to register your new Dash application and sync the code to the backing code repository. After CI checks and publishing are complete, you can select the link in the Applications panel to view the published application.

API Reference ↗

i The module name of the Dash application should be entered as `relative to the root of the repository` (for example, `app`, or if your

Send feedback

folder, `folder.app`). The variable name is the Python variable within the module file that is declared for the Dash app server, which is usually called `server`.

To develop locally against an existing Dash application:

1. Open a terminal window and execute your newly created file, for example, `python app.py`.
2. Select **Preview** in the Dash section of the **Applications** tab to preview the application.

 You can ensure the conda/PyPI environment in your JupyterLab® workspace will be automatically restored in your published Dash application using [managed conda/PyPI environments in Jupyter® Notebooks](#).

Streamlit applications

 Published Streamlit applications are subject to a 30-second timeout, meaning that the Streamlit server must start up within 30 seconds of executing your Streamlit application Python file. Otherwise, your published Streamlit application will fail to start up.

To create a [Streamlit ↗](#) application, follow the instructions below:

1. Select **Publish application**, choose a location in your **Files and Projects** for your new application, and enter the name of your Python file relative to the root of the repository, for example, `app.py`.
2. *Optional:* Configure advanced settings. By default, the application settings will match the settings of the JupyterLab® workspace.
3. Select **Publish and sync** to register your new Streamlit application and sync the code to the backing code repository. After CI checks and publishing are complete, you can select the link in the Applications panel to view the published application.

[API Reference ↗](#)

To develop locally against an existing Streamlit application:

[Send feedback](#)

1. Open a terminal window and run the code with Streamlit using `streamlit run app-name.py`.
2. Open the **Applications** tab in a JupyterLab® code workspace and select **Preview** in the Streamlit section to preview the application you are developing locally.

i You can ensure the conda/PyPI environment in your JupyterLab® workspace will be automatically restored in your published Streamlit application using [managed conda/PyPI environments in Jupyter® Notebooks](#).

TensorBoard™

You can preview [TensorBoard™](#) directly from Code Workspaces. To do this, run `tensorboard --logdir=<$PATH_TO_LOGS_DIRECTORY>` from a terminal, filling in the path to the directory containing TensorFlow logs. Then, open the Applications tab in a Jupyter® Workspace and select **Preview** in the TensorBoard™ section.

The screenshot displays the JupyterLab interface. On the left, the 'Dashboards' sidebar shows 'Life expectancy vs GDP' and 'Streamlit application'. The main area is split: the left pane shows a Jupyter Notebook with code for installing TensorFlow and loading a dataset, and a table of data. The right pane shows the TensorBoard interface with two histograms.

untitled_column	country	continent	population	life_expectancy	gdp_per_capita	
0	11	Afghanistan	Asia	31829923.0	43.828	974.580328
1	23	Albania	Europe	3600523.0	76.423	5937.029526
2	39	Algeria	Africa	33333216.0	72.301	6223.387486
3	47	Angola	Africa	12420476.0	42.731	4797.231267
4	59	Argentina	Americas	40301027.0	75.320	12779.376640
...	...	...	...	...	...	...
137	1655	Vietnam	Asia	85262356.0	74.249	2441.676404
138	1667	West Bank and Gaza	Asia	4018332.0	73.422	3025.343798
139	1679	Yemen, Rep.	Asia	2221143.0	62.898	2280.769908
140	1691	Zambia	Africa	11746035.0	42.384	1271.211593
141	1703	Zimbabwe	Africa	12311143.0	43.487	469.709288

Jupyter® Notebook transforms

[API Reference](#)

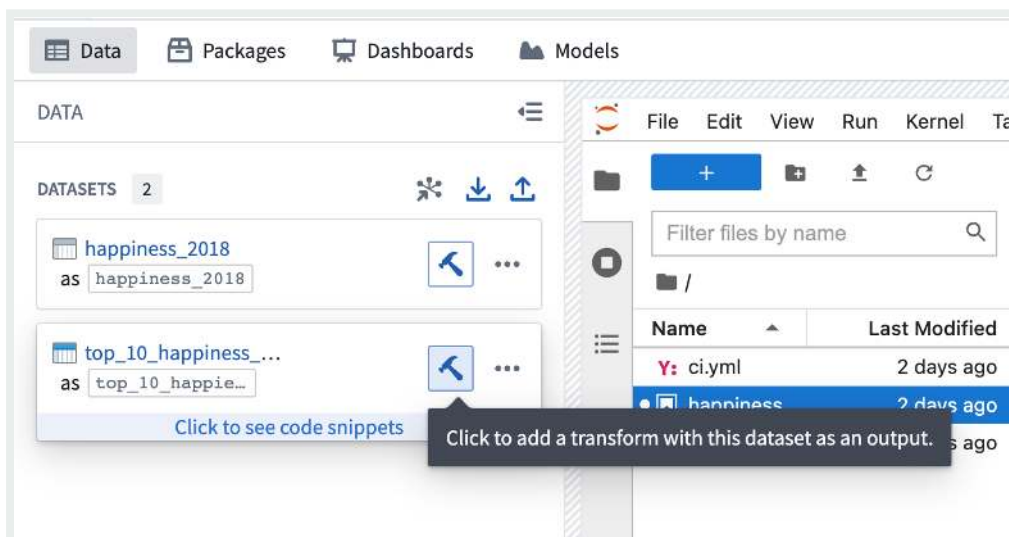
Send feedback

⚠ Jupyter Notebook transforms currently do not support making API calls to external services, even if the network policy was added in the code workspace. To run reproducible pipelines making API calls to external systems, you should use [external transforms](#) instead.

Code Workspaces can publish Jupyter® notebooks as data transformation pipelines and register transform JobSpecs for output datasets produced by the notebooks.

Follow the steps below to register a transform for an output dataset:

1. Save your Jupyter® notebook as a file in the workspace.
2. Select the blue **Build** icon next to the dataset in the **Data** panel.



3. Follow the interface to configure your transform. Code Workspaces will infer the Jupyter® notebook used along with the inputs added to produce the output dataset. If the inference is wrong, you can manually edit the inputs and outputs.

[API Reference ↗](#)

[Send feedback](#)

DATA

EDIT TRANSFORM

Beta

X

Script file

happiness_analysis.ipynb

Re-run alias configuration

Inputs

happiness_2018

X

...

Add dataset

Outputs

top_10_happiest_countries

X

...

Add dataset

CPU

0.5 CPU

MEDIUM

LARGE

X_LARGE

Memory

0.5 GB

MEDIUM

LARGE

X_LARGE

2X_LARGE

Cancel

Sync and run

4. Select **Sync and run** to save the transform configuration and trigger CI checks for publishing the transform and JobSpecs on the output dataset(s).

Once CI checks are completed, your transform is ready for build. You can use other Foundry [data integration](#) tools to manage your transform and the [data pipeline](#) to which it belongs.

i The transform does not automatically use the environment of the Code Workspace. You should install all the libraries needed at runtime at the top of the script file. The code snippet to install a library can be found in the **Libraries** panel. For example, to use `scikit-learn` and `scikit-image` in your transform, add this code snippet to

API Reference ↗ le:

```
1 !mamba install -y -q scikit-learn scikit-image
```

Send feedback

Managed conda/PyPI environments in Jupyter®

Notebooks

Jupyter® conda/PyPI environments can be consistently restored across Jupyter® workspaces, applications, transforms, and model outputs, using the managed environment feature backed by Maestro. Maestro is a Foundry library management tool that lets you install and manage libraries for Jupyter workspaces. It integrates with conda and pip, making library installation easier and helping maintain consistent environments. To convert commands to Maestro, refer to the instructions below for either [conda](#) or [PyPI](#).

Managed environments are consistent across Code Workspace workflows, eliminating the need to re-run the same `mamba` and `pip` commands every time you restart your workspace, start an application, or run a Jupyter® Notebook transform. With a managed environment, installed libraries in your working environment are tracked through files in your `/home/user/repo` folder, allowing the environment to be restored to its original state across workflows.

The following two files are used to track your managed environment:

1. `/home/user/repo/.envs/maestro/meta.yaml` : Known as the environment **manifest**, it contains the set of **requested** libraries and versions for your environment. [Learn more about the meta.yaml file.](#)
2. `/home/user/repo/.envs/maestro/hawk.lock` - Known as the environment **lock file**, it contains the set of **resolved** libraries and versions for your environment, solved from the constraints specified in the manifest. The `hawk.lock` file replaces `conda.lock`, which has been deprecated. [Learn more about the hawk.lock file.](#)

 We do not recommend modifying the `hawk.lock` lock file. Instead, rely on installation commands to update the files on your behalf. Unless there is a git conflict, you do not need to modify the `meta.yaml` file either; it will automatically update when libraries are interactively installed.

Your environment is installed in your `/home/user/envs/default` folder. This environment is restored before the following stages:

- Starting up your Jupyter® workspace

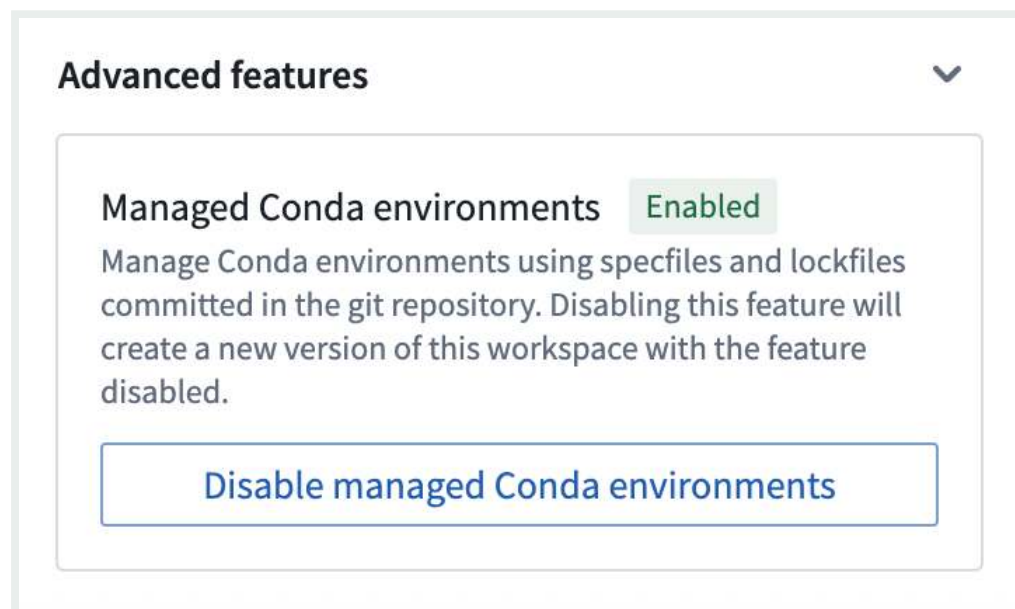
Send feedback

- Starting up applications produced from your workspace
- Running Jupyter® Notebook transforms produced from your workspace
- Running models

Usage

1. Enable managed environments

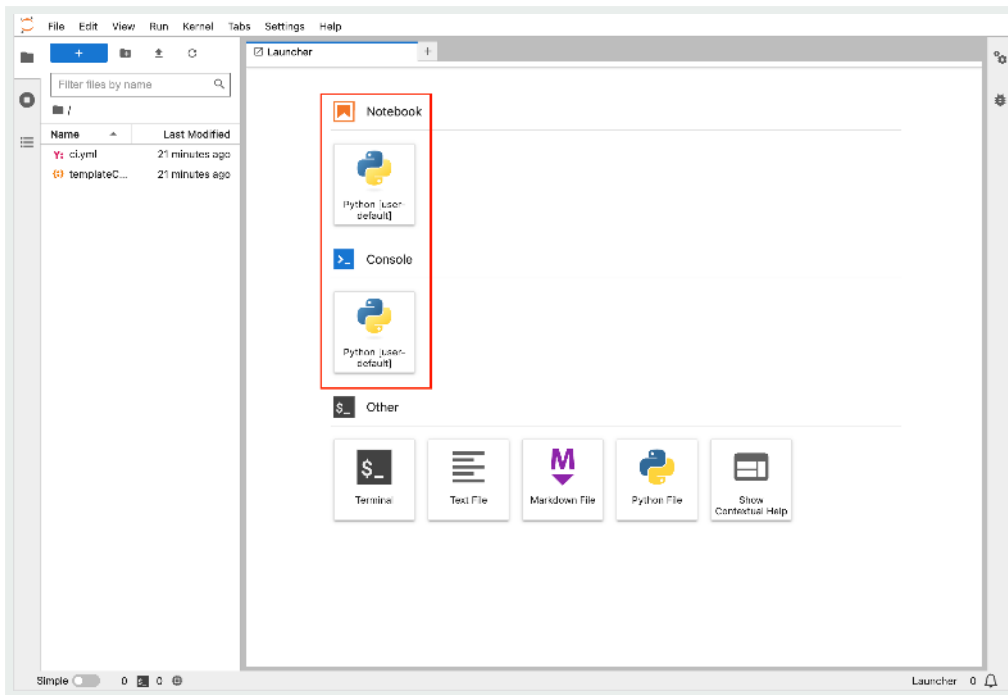
To enable this feature, toggle the **Managed Conda environments** advanced feature in your workspace settings, located on the left sidebar. Then, restart the workspace:



Once the feature is enabled, a managed environment named **default** will be available for you to use:

[API Reference ↗](#)

[Send feedback](#)



2. Update your managed environment

Follow the steps below to update your managed environment and manage libraries.

Manage conda libraries with Maestro

- **Install using the Libraries panel:** Search for the library name (for example, `numpy`) and install.
- **Install using the terminal:** To install the latest version of a library, run the following command in the terminal:

```
1 maestro env conda install <library_name>
```

To install a specific version, run the following command:

```
1 maestro env conda install <library_name>==<library_version>
```

[API Reference ↗](#)

Uninstall using the terminal: To uninstall a library, run the following command in the terminal:

[Send feedback](#)


```
1 maestro env uninstall --conda-dependencies <library_names>
```



Manage PyPI libraries with Maestro

- **Install using the Libraries panel:** Search for the library name (for example, `numpy`) and install.
- **Install using the terminal:** To install the latest version of a library, run the following command in the terminal:

```
1 maestro env pip install <library_name>
```



To install a specific version, run the following command:

```
1 maestro env pip install <library_name>==<library_version>
```



- **Uninstall using the terminal:** To uninstall a library, run the following command in the terminal:

```
1 maestro env uninstall --pip-dependencies <library_names>
```

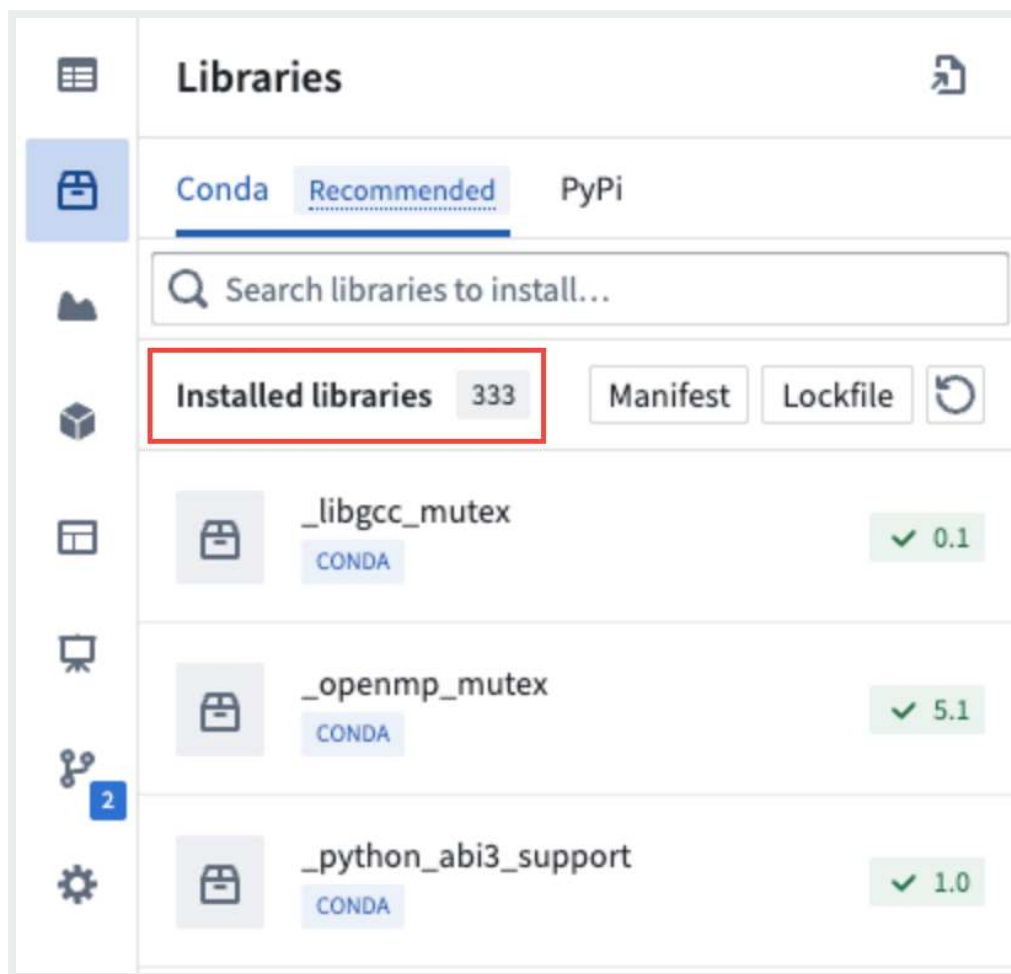


⚠ When installing a library through the **Libraries** tab, the install command is executed directly in the Jupyter® terminal, rather than executing it within your notebook or application code. Every time an install command is run, the `meta.yaml` manifest will be updated with your requested library, and the `hawk.lock` file will reflect the libraries installed in the current environment.

3. View currently installed libraries in your managed environment

In [API Reference ↗](#), visible in the **Libraries** tab:

Send feedback

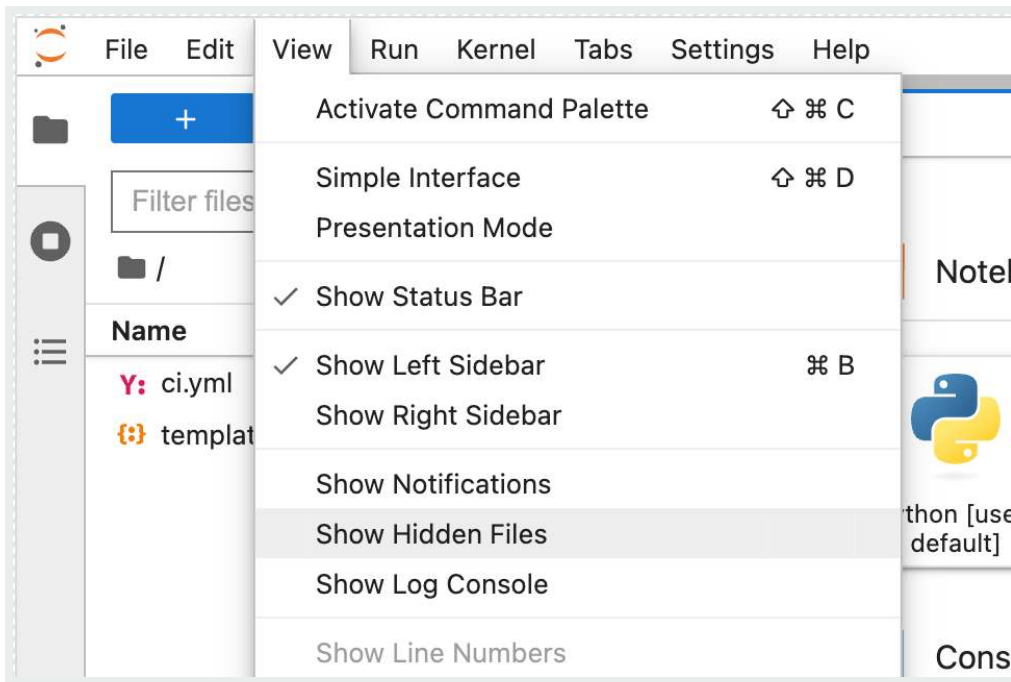


4. View environment files in the JupyterLab® interface

Navigate to the **View** dropdown menu in the workspace and select **Show Hidden Files**.

[API Reference ↗](#)

[Send feedback](#)



Then, navigate to the `.envs/maestro` folder to view the `meta.yaml` and `hawk.lock` files.

5. Synchronize your managed environment with changes to the `hawk.lock` file

After checking out a new branch or pulling new git changes, the contents of your `hawk.lock` file may change.

To synchronize your managed environment, execute the following command in your workspace terminal:

```
1 maestro env install
```



i `maestro` is the command line interface that updates managed environments. The above command re-resolves the lock file from the manifest and installs the environment into the `/home/user/envs/default` folder.

[API Reference ↗](#)

6. Resolve git conflicts to your environment files

[Send feedback](#)

To resolve git conflicts in your managed environment, first delete the `hawk.lock` file. Navigate to the `meta.yaml` file to resolve any conflicts, then merge your branch.

After merging, run the `maestro env install` command in your terminal to regenerate the `hawk.lock` file and reinstall your managed environment.

7. Upgrade your managed environment

To update your managed environment, first remove the `hawk.lock` file. Then, delete your `/home/user/envs/default` folder (`rm -rf /home/user/envs/default`). Finally, run the `maestro env install` command in your terminal to regenerate your `hawk.lock` file. Your environment will be re-installed with upgraded library versions.

Example: `meta.yaml` file

The `meta.yaml` file represents a specification of requested libraries. This file conforms to the [Conda metadata specification ↗](#), and can contain specifications for both conda and PyPI dependencies. Only libraries specified in the `requirements.run` and `requirements.pip` section are used for Jupyter® managed environments.

Below is an example of a `meta.yaml` file:

```
1 package:
2   name: '{{ PACKAGE_NAME }}'
3   version: '{{ PACKAGE_VERSION }}'
4 requirements:
5   build: []
6   run:
7     - ipykernel
8     - pip
9     - foundry-transforms-lib-python
10    - pandas
11    - polars
12    - dash
13    - streamlit
14    - tensorboard
```

API Reference ↗ -Testing

```
17   run_constrained: []
18 source:
19   path: ../src
```

Send feedback

```
20 build: null
21 test: null
22 about: null
```

Example: `hawk.lock` file

The `hawk.lock` file contains the resolved libraries, versions, and channels of a solved managed environment.

Below is an example of a `hawk.lock` file:

```
# This file is autogenerated. Though not necessary, it can be committed to the g
# To avoid build failures, we do not recommend changing this file,
```

```
[conda]
```

```
_libgcc_mutex=0.1=conda_forge @ 16fe1c80
_openmp_mutex=4.5=2_gnu @ 16fe1c80
absl-py=2.1.0=pyhd8ed1ab_0 @ c7211bb6
altair=5.3.0=pyhd8ed1ab_0 @ c7211bb6
asttokens=2.4.1=pyhd8ed1ab_0 @ c7211bb6
attrs=23.2.0=pyh71513ae_0 @ c7211bb6
aws-c-auth=0.7.22=h96bc93b_2 @ e1c11c02
aws-c-cal=0.6.14=h88a6e22_1 @ e1c11c02
aws-c-common=0.9.19=h4ab18f5_0 @ e1c11c02
aws-c-compression=0.2.18=h83b837d_6 @ e1c11c02
aws-c-event-stream=0.4.2=ha47c788_12 @ e1c11c02
aws-c-http=0.8.1=h29d6fba_17 @ e1c11c02
aws-c-io=0.14.8=h21d4f22_5 @ e1c11c02
aws-c-mqtt=0.10.4=h759edc4_4 @ e1c11c02
aws-c-s3=0.5.9=h594631b_3 @ e1c11c02
aws-c-sdkutils=0.1.16=h83b837d_2 @ e1c11c02
aws-checksums=0.1.18=h83b837d_6 @ e1c11c02
aws-crt-cpp=0.26.9=he3a8b3b_0 @ e1c11c02
aws-sdk-cpp=1.11.329=hba8bd5f_3 @ e1c11c02
brotli-python=1.1.0=py310hc6cd4ac_1 @ e1c11c02
bzip2=1.0.8=hd590300_5 @ e1c11c02
c-ares=1.28.1=hd590300_0 @ e1c11c02
```

API Reference ↗

Send feedback

ca-certificates=2024.7.4=hbcca054_0 @ e1c11c02
cachetools=5.3.3=pyhd8ed1ab_0 @ c7211bb6
certifi=2024.7.4=pyhd8ed1ab_0 @ c7211bb6
cffi=1.16.0=py310h2fee648_0 @ e1c11c02
charset-normalizer=3.3.2=pyhd8ed1ab_0 @ c7211bb6
click=8.1.7=unix_pyh707e725_0 @ c7211bb6
comm=0.2.2=pyhd8ed1ab_0 @ c7211bb6
conjure-python-client=2.8.0=pyhd8ed1ab_0 @ c7211bb6
dash=2.17.1=pyhd8ed1ab_0 @ c7211bb6
debugpy=1.8.2=py310h76e45a6_0 @ e1c11c02
decorator=5.1.1=pyhd8ed1ab_0 @ fe451c34
exceptiongroup=1.2.0=pyhd8ed1ab_2 @ c7211bb6
executing=2.0.1=pyhd8ed1ab_0 @ c7211bb6
flask=3.0.3=pyhd8ed1ab_0 @ c7211bb6
foundry-data-sidecar-api=0.626.0=py_0 @ fe451c34
foundry-transforms-lib-python=0.639.0=py_0 @ fe451c34
freetype=2.12.1=h267a509_2 @ e1c11c02
gflags=2.2.2=he1b5a44_1004 @ 16fe1c80
gitdb=4.0.11=pyhd8ed1ab_0 @ c7211bb6
gitpython=3.1.43=pyhd8ed1ab_0 @ c7211bb6
glog=0.7.1=hbabe93e_0 @ e1c11c02
grpcio=1.62.2=py310h1b8f574_0 @ e1c11c02
h2=4.1.0=pyhd8ed1ab_0 @ fe451c34
hpack=4.0.0=pyh9f0ad1d_0 @ fe451c34
hyperframe=6.0.1=pyhd8ed1ab_0 @ fe451c34
idna=3.7=pyhd8ed1ab_0 @ c7211bb6
importlib-metadata=7.2.1=pyha770c72_0 @ c7211bb6
importlib_metadata=7.2.1=hd8ed1ab_0 @ c7211bb6
importlib_resources=6.4.0=pyhd8ed1ab_0 @ c7211bb6
ipykernel=6.29.5=pyh3099207_0 @ c7211bb6
ipython=8.26.0=pyh707e725_0 @ c7211bb6
ipywidgets=8.1.3=pyhd8ed1ab_0 @ c7211bb6
itsdangerous=2.2.0=pyhd8ed1ab_0 @ c7211bb6
jedi=0.19.1=pyhd8ed1ab_0 @ c7211bb6
jinja2=3.1.4=pyhd8ed1ab_0 @ c7211bb6
jupyterlab=4.0.12=pyhd8ed1ab_0 @ c7211bb6
jupyterlab-specifications=2023.12.1=pyhd8ed1ab_0 @ c7211bb6
jupyter_client=8.6.2=pyhd8ed1ab_0 @ c7211bb6
jupyter_core=5.7.2=py310hff52083_0 @ e1c11c02

API Reference ↗

Send feedback

jupyterlab_widgets=3.0.11=pyhd8ed1ab_0 @ c7211bb6
keyutils=1.6.1=h166bdaf_0 @ 16fe1c80
krb5=1.21.3=h659f571_0 @ e1c11c02
lcms2=2.16=hb7c19ff_0 @ e1c11c02
ld_impl_linux-64=2.40=hf3520f5_7 @ e1c11c02
lerc=4.0.0=h27087fc_0 @ 16fe1c80
libabseil=20240116.2=cxx17_h59595ed_0 @ e1c11c02
libarrow=16.1.0=hcb6531f_6_cpu @ e1c11c02
libarrow-acero=16.1.0=hac33072_6_cpu @ e1c11c02
libarrow-dataset=16.1.0=hac33072_6_cpu @ e1c11c02
libarrow-substrait=16.1.0=h7e0c224_6_cpu @ e1c11c02
libblas=3.9.0=22_linux64_openblas @ e1c11c02
libbrotlicommon=1.1.0=hd590300_1 @ e1c11c02
libbrotlidec=1.1.0=hd590300_1 @ e1c11c02
libbrotlienc=1.1.0=hd590300_1 @ e1c11c02
libcbblas=3.9.0=22_linux64_openblas @ e1c11c02
libcrc32c=1.1.2=h9c3ff4c_0 @ 16fe1c80
libcurl=8.8.0=hca28451_1 @ e1c11c02
libdeflate=1.20=hd590300_0 @ e1c11c02
libedit=3.1.20191231=he28a2e2_2 @ 16fe1c80
libev=4.33=hd590300_2 @ e1c11c02
libevent=2.1.12=hf998b51_1 @ e1c11c02
libffi=3.4.2=h7f98852_5 @ 16fe1c80
libgcc-ng=14.1.0=h77fa898_0 @ e1c11c02
libgfortran-ng=14.1.0=h69a702a_0 @ e1c11c02
libgfortran5=14.1.0=hc5f4f2c_0 @ e1c11c02
libgomp=14.1.0=h77fa898_0 @ e1c11c02
libgoogle-cloud=2.24.0=h2736e30_0 @ e1c11c02
libgoogle-cloud-storage=2.24.0=h3d9a0c8_0 @ e1c11c02
libgrpc=1.62.2=h15f2491_0 @ e1c11c02
libjpeg-turbo=3.0.0=hd590300_1 @ e1c11c02
liblapack=3.9.0=22_linux64_openblas @ e1c11c02
libnghttp2=1.58.0=h47da74e_1 @ e1c11c02
libnsl=2.0.1=hd590300_0 @ e1c11c02
libopenblas=0.3.27=pthreads_hac2b453_1 @ e1c11c02
libpng=1.6.43=h2797004_0 @ e1c11c02
libprotobuf=4.25.3=h08a7969_0 @ e1c11c02
libre2-11=2023.09.01=h5a48ba9_2 @ e1c11c02

API Reference ↗

Send feedback

libsodium=1.0.18=h36c2ea0_1 @ 16fe1c80
libssh2=1.11.0=h0841786_0 @ e1c11c02
libstdcxx-ng=14.1.0=hc0a3c3a_0 @ e1c11c02
libthrift=0.19.0=hb90f79a_1 @ e1c11c02
libtiff=4.6.0=h1dd3fc0_3 @ e1c11c02
libutf8proc=2.8.0=h166bdaf_0 @ 16fe1c80
libuuid=2.38.1=h0b41bf4_0 @ e1c11c02
libwebp-base=1.4.0=hd590300_0 @ e1c11c02
libxcb=1.15=h0b41bf4_0 @ e1c11c02
libzlib=1.2.13=h4ab18f5_6 @ e1c11c02
lz4-c=1.9.4=hcb278e6_0 @ e1c11c02
markdown=3.6=pyhd8ed1ab_0 @ c7211bb6
markdown-it-py=3.0.0=pyhd8ed1ab_0 @ c7211bb6
markupsafe=2.1.5=py310h2372a71_0 @ e1c11c02
matplotlib-inline=0.1.7=pyhd8ed1ab_0 @ c7211bb6
mdurl=0.1.2=pyhd8ed1ab_0 @ c7211bb6
ncurses=6.2=h58526e2_4 @ 16fe1c80
nest-asyncio=1.6.0=pyhd8ed1ab_0 @ c7211bb6
numpy=1.26.4=py310hb13e2d6_0 @ e1c11c02
openjpeg=2.5.2=h488ebb8_0 @ e1c11c02
openssl=3.3.1=h4ab18f5_1 @ e1c11c02
orc=2.0.1=h17fec99_1 @ e1c11c02
packaging=24.1=pyhd8ed1ab_0 @ c7211bb6
pandas=2.2.2=py310hf9f9076_1 @ e1c11c02
parso=0.8.4=pyhd8ed1ab_0 @ c7211bb6
pexpect=4.9.0=pyhd8ed1ab_0 @ c7211bb6
pickleshare=0.7.5=py_1003 @ fe451c34
pillow=10.3.0=py310hf73ecf8_0 @ e1c11c02
pip=24.0=pyhd8ed1ab_0 @ c7211bb6
pkgutil-resolve-name=1.3.10=pyhd8ed1ab_1 @ c7211bb6
platformdirs=4.2.2=pyhd8ed1ab_0 @ c7211bb6
plotly=5.22.0=pyhd8ed1ab_0 @ c7211bb6
polars=1.1.0=py310h3af5803_0 @ e1c11c02
prompt-toolkit=3.0.47=pyha770c72_0 @ c7211bb6
protobuf=4.25.3=py310ha8c1f0e_0 @ e1c11c02
pycparser=2.21=py310h0c51659f_0 @ e1c11c02
pycups=2.0.1=h36c2ea0_1001 @ 16fe1c80
ptyprocess=0.7.0=pyhd3deb0d_0 @ fe451c34
pure_eval=0.2.2=pyhd8ed1ab_0 @ fe451c34

API Reference ↗

Send feedback

pyarrow=16.1.0=py310h17c5347_1 @ e1c11c02
pyarrow-core=16.1.0=py310h6f79a3a_1_cpu @ e1c11c02
pyparser=2.22=pyhd8ed1ab_0 @ c7211bb6
pydeck=0.8.0=pyhd8ed1ab_0 @ fe451c34
pygments=2.18.0=pyhd8ed1ab_0 @ c7211bb6
pysocks=1.7.1=pyha2e5f31_6 @ fe451c34
python=3.10.2=h543edf9_0_cpython @ 16fe1c80
python-dateutil=2.9.0=pyhd8ed1ab_0 @ c7211bb6
python-tzdata=2024.1=pyhd8ed1ab_0 @ c7211bb6
python_abi=3.10=4_cp310 @ e1c11c02
pytz=2024.1=pyhd8ed1ab_0 @ c7211bb6
pyyaml=6.0.1=py310h2372a71_1 @ e1c11c02
pymq=26.0.3=py310h6883aea_0 @ e1c11c02
re2=2023.09.01=h7f4b329_2 @ e1c11c02
readline=8.1=h46c0cb4_0 @ 16fe1c80
referencing=0.35.1=pyhd8ed1ab_0 @ c7211bb6
requests=2.32.3=pyhd8ed1ab_0 @ c7211bb6
retrying=1.3.3=py_2 @ fe451c34
rich=13.7.1=pyhd8ed1ab_0 @ c7211bb6
rpsd-py=0.19.0=py310h42e942d_0 @ e1c11c02
s2n=1.4.15=he19d79f_0 @ e1c11c02
setuptools=70.1.1=pyhd8ed1ab_0 @ c7211bb6
six=1.16.0=pyh6c4a22f_0 @ fe451c34
smmap=5.0.0=pyhd8ed1ab_0 @ fe451c34
snappy=1.2.1=ha2e4443_0 @ e1c11c02
sqlite=3.37.0=h9cd32fc_0 @ 16fe1c80
stack_data=0.6.2=pyhd8ed1ab_0 @ c7211bb6
streamlit=1.36.0=pyhd8ed1ab_0 @ c7211bb6
tenacity=8.5.0=pyhd8ed1ab_0 @ c7211bb6
tensorboard=2.17.0=pyhd8ed1ab_0 @ c7211bb6
tensorboard-data-server=0.7.0=py310h75e40e8_1 @ e1c11c02
tk=8.6.13=noxft_h4845f30_101 @ e1c11c02
toml=0.10.2=pyhd8ed1ab_0 @ fe451c34
toolz=0.12.1=pyhd8ed1ab_0 @ c7211bb6
tornado=6.4.1=py310hc51659f_0 @ e1c11c02
typing_extensions=4.12.2=pyhd8ed1ab_0 @ c7211bb6
typing_extensions=4.12.2=pyha770c72_0 @ c7211bb6
tzdata=2024a=h0c530f3_0 @ c7211bb6

API Reference ↗

Send feedback

```
tzlocal=5.2=py310hff52083_0 @ e1c11c02
urllib3=2.2.2=pyhd8ed1ab_1 @ c7211bb6
validators=0.31.0=pyhd8ed1ab_0 @ c7211bb6
watchdog=4.0.1=py310hff52083_0 @ e1c11c02
wcwidth=0.2.13=pyhd8ed1ab_0 @ c7211bb6
werkzeug=3.0.3=pyhd8ed1ab_0 @ c7211bb6
wheel=0.43.0=pyhd8ed1ab_1 @ c7211bb6
widgetsnbextension=4.0.11=pyhd8ed1ab_0 @ c7211bb6
xorg-libxau=1.0.11=hd590300_0 @ e1c11c02
xorg-libxdmcp=1.1.3=h7f98852_0 @ 16fe1c80
xz=5.2.6=h166bdaf_0 @ 16fe1c80
yaml=0.2.5=h7f98852_2 @ 16fe1c80
zeromq=4.3.5=h75354e8_4 @ e1c11c02
zipp=3.19.2=pyhd8ed1ab_0 @ c7211bb6
zlib=1.2.13=h4ab18f5_6 @ e1c11c02
zstandard=0.22.0=py310hab88d88_1 @ e1c11c02
zstd=1.5.6=ha6fb4c9_0 @ e1c11c02
```

[pip]

```
brotli==1.1.0
importlib-resources==6.4.0
jupyter-client==8.6.2
jupyter-core==5.7.2
jupyterlab-widgets==3.0.11
pure-eval==0.2.2
stack-data==0.6.2
```

[conda-metadata]

```
16fe1c80 - linux-64 @ tar.bz2 @ ri.artifacts.repository.bundle.jupyter-default-env
c7211bb6 - noarch @ conda @ ri.artifacts.repository.bundle.jupyter-default-env
e1c11c02 - linux-64 @ conda @ ri.artifacts.repository.bundle.jupyter-default-env
fe451c34 - noarch @ tar.bz2 @ ri.artifacts.repository.bundle.jupyter-default-env
```

[conda lock version]

```
v3 - 0454ce4598cd475c502df1fe8a15b86ddae33cd74f97c833cb35f122e7555dfe
```

[API Reference ↗](#)

[Send feedback](#)

Jupyter[®], JupyterLab[®], and the Jupyter[®] logos are trademarks or registered trademarks of NumFOCUS.

TensorBoard and any related marks are trademarks of Google Inc.

All third-party trademarks (including logos and icons) referenced remain the property of their respective owners. No affiliation or endorsement is implied.

PREVIOUS

← Train and interact with models

NEXT

RStudio[®] Workbench →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

RStudio® Workbench

i To use RStudio® Workbench, your platform administrator must first [configure its operational license in Control Panel](#). If you are a platform administrator and are unable to configure licenses in Control Panel, contact Palantir Support.

Code Workspaces enables you to use [RStudio® Workbench ↗](#) in Foundry. RStudio® Workbench in Code Workspaces supports:

- Reading and writing tabular datasets using the [Palantir R SDK ↗](#).
- Downloading or uploading files from non-structured datasets.
- Importing R libraries from CRAN, Posit™ library Manager, and Bioconductor.
- Publishing Shiny® applications.
- Publishing R transforms.

Code Workspaces currently supports [Shiny® ↗](#) for R applications. Users can create applications directly in Code Workspaces with Foundry's [version control](#), [branching](#), and [data governance](#) features built-in.

i Code Workspace applications support branching. If you [create a new Workspace branch](#), publishing a new application or synchronizing the changes will publish a new version of the application on that branch. This allows you to preview your application before exposing it to your users. To publish on the master branch, simply merge your branch into master.

API Reference ↗



Send feedback

With Code Workspaces, you can create a Shiny® application and preview it directly from the RStudio® UI.

 By default, RStudio® generates Shiny® applications in the home folder (`/home/user`) but Foundry requires applications to be in the git repository `/home/user/repo` for them to be persisted across sessions and published.

In Code Workspaces, select **Publish application**, select a location in your **Files and Projects** for your new application, and enter the name of the directory in which a new Shiny® `app.R` file should be automatically created, relative to the repository root `/home/user/repo`. This field can be left blank to create the application at the root of the repository. By default, the application settings will match the RStudio® workspace settings, and configuring advanced settings is optional.

 Published Shiny applications are subject to a 30-second timeout, meaning that the Shiny server must start up within 30 seconds of executing your Shiny application file. Otherwise, your Shiny application will fail to start up.

Select **Publish and sync** to register your new Shiny® application and sync the code to the backing code repository. After CI checks and publishing are complete, you can select the link in the Applications panel to view the published application.

To develop locally against a Shiny® application:

1. Open `app.R` in RStudio®.
2. Select **Run App**. A preview of your application will appear in a new window.

Configure Shiny® applications to automatically reconnect

By default, Shiny greys out if the websocket connection is closed, for instance if there is a network outage for over 15 seconds. To ensure your application can recover when the network is restored, we recommend updating the `server` function to set `session$allowReconnect("force")`:

[API Reference ↗](#)

Send feedback

```

1 server <- function(input, output, session) {
2   # Handle input changes
3   session$allowReconnect("force")
4 }

```

This will cause the frontend to trigger a new websocket when the network is available again, and to send the last input information that was set in the UI.

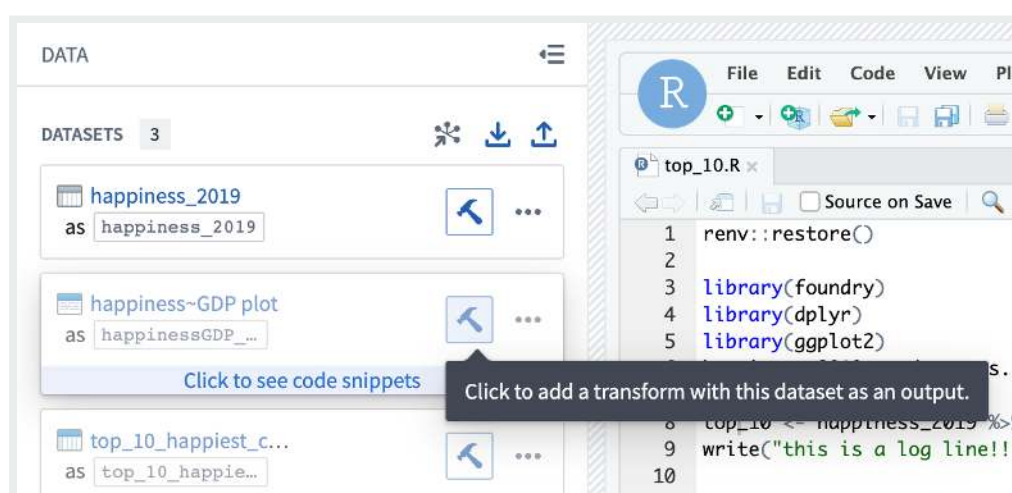
R Transform

⚠ R transforms currently do not support making API calls to external services, even if the network policy was added in the RStudio Code Workspace.

R scripts written in Code Workspaces can be registered as transforms for output datasets in Code Workspaces.

Follow the steps below to register a transform for an output dataset:

1. Select the blue **Build** icon next to the dataset in the **Data** panel.



[API Reference ↗](#)

2. Follow the interface to configure your transform. Code Workspaces infer the R script used along with the inputs added to produce the output dataset.

[Send feedback](#)

DATA

CREATE TRANSFORM

Beta

X

Script file

top_10.R

Re-run alias configuration

Inputs

happiness_2019

X

...

Add dataset

Outputs

happinessGDP_plot

X

...

top_10_happiest_countries

X

...

Add dataset

CPU

0.5 CPU

MEDIUM

LARGE

X_LARGE

Memory

0.5 GB

MEDIUM

LARGE

X_LARGE

2X_LARGE

Cancel

Sync and run

3. Select **Sync and run** to save the transform configuration and trigger CI checks for publishing the transform and JobSpecs on the output dataset(s).

Once CI checks are completed, your transform is ready for build. You can then use other Foundry [data integration](#) tools to manage your transform and the [data pipeline](#) to which it belongs.

i The transform does not automatically use the environment of the Code Workspace. You should install all the libraries needed at runtime at the top of the script file. The [API Reference](#) to install a library can be found in the **libraries** panel. For example, to use `uptyr` and `tidyr` in your transform, add this code snippet to the top of the file:

Send feedback

```
1 renv::install(c("dplyr", "tidyr"))
```



[Learn more about R transforms.](#)

RStudio® and Shiny® are trademarks of Posit™.

All third-party trademarks (including logos and icons) referenced remain the property of their respective owners. No affiliation or endorsement is implied.

Markdown Reports

R Markdown or Quarto files written in RStudio® workspaces can be shared with other Palantir platform users as reports. A report is a dataset built from a Markdown file using a transform that contains HTML produced by rendering the Markdown file with [Quarto](#).

Reports are built using transforms, so they can be configured to automatically update based on the latest changes to the Markdown file using a [schedule](#). This ensures that your analyses and visualizations are up to date and built with a reproducible, customizable workflow that integrates with the rest of the Palantir platform.

The screenshot displays the Palantir platform interface. On the left, the 'Applications' sidebar is visible, with 'Reports' selected. Below it, a list of reports is shown, including 'Airport Analysis' (airports.Rmd). A status message indicates 'Successfully published transforms. View checks'. A 'Report version' dropdown is set to 'Latest'. A callout box labeled 'API Reference' points to the 'Latest' button. On the right, the RStudio interface is shown, with the terminal window displaying the R version 4.1.3 output, including the R license and help information. A 'Send feedback' button is located in the bottom right corner.

Publish reports from an RStudio® workspace

A Markdown report can be published from an RStudio® workspace using the following steps:

1. Create the Markdown file (`.Rmd` or `.qmd`) you wish to use to create a report. This Markdown file can read from Foundry datasets that have been added to your workspace using the [data side panel](#).
2. Open the **Applications** side panel and select the **Reports** tab.
3. Select **Add** to open the report configuration dialog.
4. Provide a name and location; this determines where the dataset containing the report will be saved.
5. Select the Markdown file to build your report from. If your Markdown reads from Foundry datasets, add these datasets as report inputs. Select **Auto-configure inputs and outputs** to automatically parse your Markdown file and populate the form with any detected datasets.
6. Select **Create**. This will create a dataset at the specified location. The transform that builds the report will then run. Note that this can take several minutes to complete.
7. Once the report has been built, select the report name in the **Applications** side panel to open the report in a new tab. This link can be shared with other users.

To edit or delete an existing report, select ... on the right of the report card. This will open a menu with options to edit or delete the report, and an option to copy the link to the report.

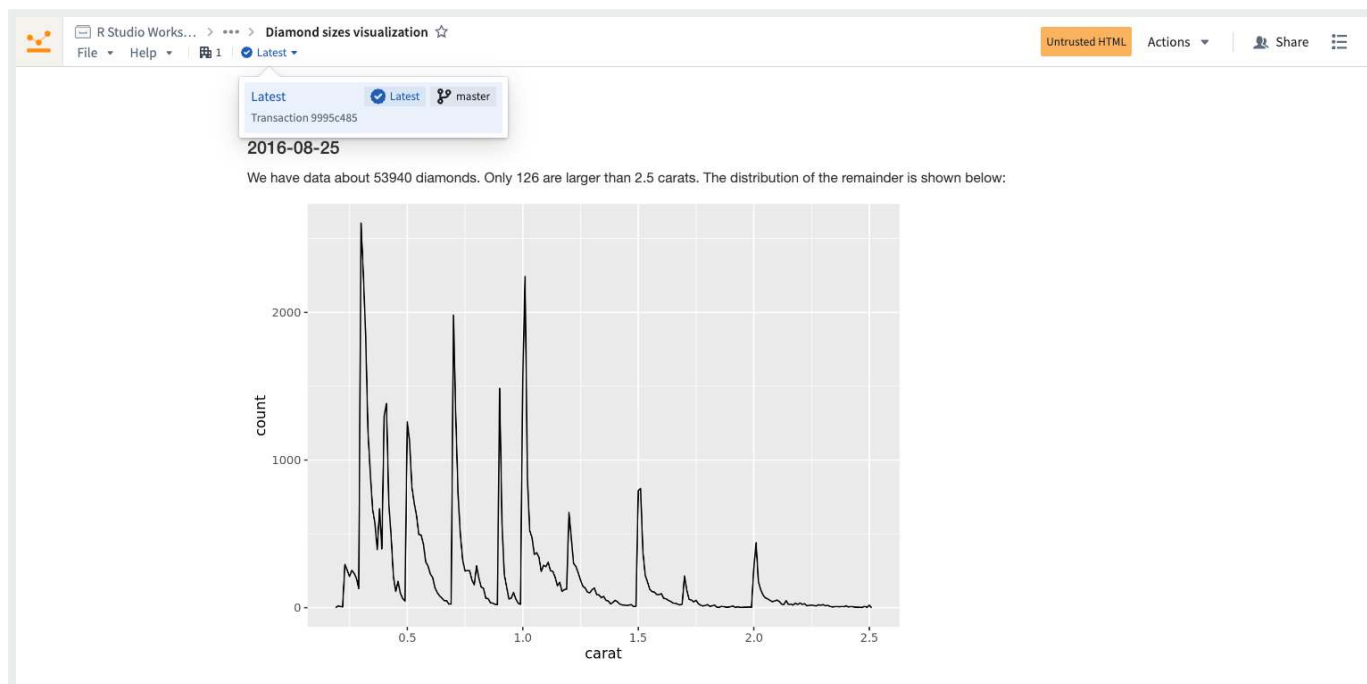
View previous report versions

Reports support full version history. To view a previous version of a report, expand the report card in the **Applications** side panel or use the version selector in the page header when viewing a report in a new tab.

Selecting a report version will update the link used to share the report, enabling you to share a specific historical version of a report that will not be updated if the underlying Markdown is modified.

[API Reference ↗](#)

[Send feedback](#)



The latest version of a report will be selected by default, so readers of your report automatically receive the latest changes.

Managed conda environments in RStudio®

RStudio® conda environments can be consistently restored across RStudio® workspaces, Shiny applications, transforms, and model outputs using **managed environments**. Managed environments are consistent across Code Workspace workflows, eliminating the need to re-run the same `renv::install` command every time you restart your workspace, start a Shiny application, or run an RStudio® transform.

- i** This feature now includes CRAN library management through Maestro, a Foundry library management tool that seamlessly integrates with conda and CRAN to efficiently handle library installations and maintain managed environments. Newly-installed CRAN libraries will be tracked in your environment files and consistently restored across RStudio® workspaces, Shiny applications, transforms, and model outputs, improving the reproducibility of your environments. See the [get started](#) section below to enable this feature.

[API Reference ↗](#)

To convert library install commands to Maestro, refer to the instructions below for either [conda](#) or [CRAN](#). With a managed environment, installed libraries in y

[Send feedback](#)

environment are tracked through the following files, allowing the environment to be restored to its original state across workflows:

1. `/home/user/repo/.envs/maestro/meta.yaml` : Known as the environment **manifest**, it contains the set of **requested** libraries and versions for your environment.
2. `/home/user/repo/.envs/maestro/hawk.lock` : Known as the environment **lock file**, it contains the set of **resolved** libraries and versions for your environment, solved from the constraints specified in the manifest. The `hawk.lock` file replaces `conda.lock`, which has been deprecated.

When you sync any new code in your conda environments, the files above will be synced as well. When you start your next session, the restored code will include the `maestro` files and your environment will be recreated.

 We do not recommend modifying the `hawk.lock` lock file. Instead, rely on installation commands to update the files on your behalf. Unless there is a git conflict, you do not need to modify the `meta.yaml` file either; it will automatically update when libraries are interactively installed. If you do modify these files, you risk restoring an outdated conda environment or failing the environment build.

Your managed environment is installed in your `/home/user/envs/default` folder. This environment will be restored before any of the following events:

- Startup of your RStudio® workspace
- Startup of Shiny applications produced from your workspace
- Runs of RStudio® transforms produced from your workspace
- Renderings of [Markdown reports](#)

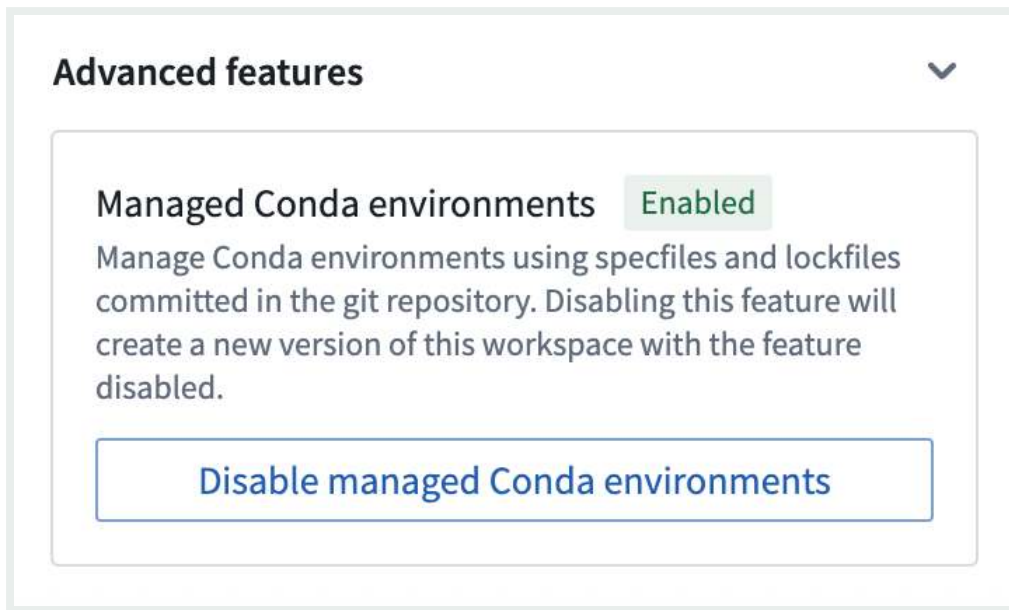
Get started

1. Enable managed environments

To enable managed environments in RStudio® workspaces, navigate to your workspace settings from the left sidebar, then toggle on **Managed Conda environments** under **Advanced features**. It may already be turned on by default.

[API Reference ↗](#) [Send feedback](#)

Then, restart the workspace:



To enable CRAN library management by Maestro in RStudio® workspaces, navigate to your workspace settings using the left sidebar, then toggle on **Managed CRAN environments** under **Experimental features**. This option may already be enabled by default.

Then, restart the workspace.

2. Update your managed environment

Follow the steps below to update your managed environment and manage new libraries:

Manage conda libraries with Maestro (recommended)

i Install R libraries with conda instead of CRAN to significantly reduce workspace startup time.

- **Install using the Libraries panel:**

API Reference ↗

libraries: Search for the library name with an `r-` prefix (for example, `r-ggplot2`) and install.

- For Bioconductor libraries: Search for the lowercase library name with a `bioconductor-` prefix (for example, `bioconductor-genomeinfodb`) and install.

Send feedback

- **Install using the terminal:** To install the latest version of a library, run the following command in the terminal:

```
1 maestro env conda install <library_name>
```

To install a specific version, run the following command:

```
1 maestro env conda install <library_name>==<library_version>
```

- **Uninstall using the terminal:** To uninstall a library, run the following command in the terminal:

```
1 maestro env uninstall --conda-dependencies <library_names>
```

Manage CRAN libraries with Maestro

- **Install using the Libraries panel:**
 - For standard libraries: Search for the library name (for example, `ggplot2`) and install.
 - For Bioconductor libraries: Search for the library name with initial capitalization format (for example, `GenomeInfoDb`) and install.
- **Install using the terminal:** To install the latest version of a library, run the following command in the terminal:

```
1 maestro env cran install <library_name>
```

To install a specific version, run the following command:

```
1 maestro env cran install <library_name>@<library_version>
```

API Reference ↗

- **Uninstall using the terminal:** To uninstall a library, run the following command in the terminal:

Send feedback

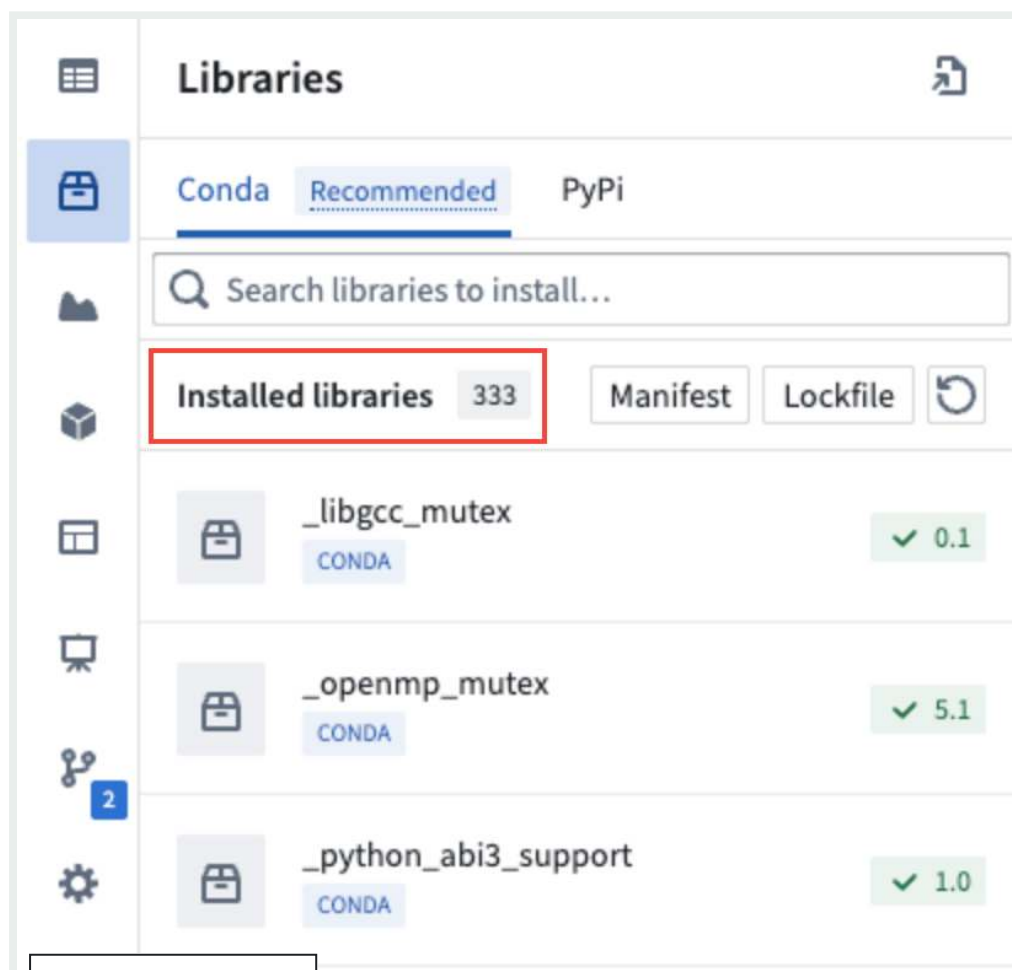
```
1 maestro env uninstall --cran-dependencies <library_names>
```



⚠ When installing a library through the **Libraries** tab, the install command is executed directly in the RStudio® console rather than executed within your script or application code. Every time an install command is run, the `meta.yaml` manifest will be updated with your requested library, and the `hawk.lock` file will reflect the libraries installed in the current environment.

3. View currently installed libraries in your managed environment

Your installed conda libraries are visible in the **Libraries** tab in the left sidebar:



[API Reference ↗](#)

You can also choose to view the `meta.yaml` and `hawk.lock` files in the

[Send feedback](#)

4. Synchronize your managed environment with changes to the `hawk.lock` file

After checking out a new branch or pulling new git changes, the contents of your `hawk.lock` file may change.

To synchronize your managed environment, execute the following command in your workspace terminal:

```
1 maestro env install
```



i `maestro` is the command line interface that updates managed environments. The above command re-resolves the lock file from the manifest and installs the environment into the `/home/user/envs/default` folder.

5. Resolve git conflicts to your environment files

To resolve git conflicts in your managed environment, first delete the `hawk.lock` file. Navigate to the `meta.yaml` file to resolve any conflicts, then merge your branch.

After merging, run the `maestro env install` command in your terminal to regenerate the `hawk.lock` file and reinstall your managed environment.

6. Upgrade your managed environment

To upgrade your managed environment to a more recent version, first remove the `hawk.lock` file. Then, delete your `/home/user/envs/default` folder (`rm -rf /home/user/envs/default`). Finally, run the `maestro env install` command in your terminal to regenerate your `hawk.lock` file. Your environment will be re-installed with upgraded library versions.

All third-party trademarks (including logos and icons) referenced remain the property of their respective owners. No affiliation or endorsement is implied.

PREVIOUS
← JupyterLab®

NEXT
Palantir-provided models →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Developer toolchain / Code Workspaces / Palantir-provided models

Use Palantir-provided language models within Jupyter[®] notebooks

Prerequisites

To use Palantir-provided language models, [AIP must first be enabled on your enrollment](#).

Palantir provides a [set of language and embedding models](#) which can be used within Jupyter[®] notebooks. The models can be used through the `palantir_models` library. This library provides a set of classes that provide bindings to interact with the models.

Palantir-provided model setup in Code Workspaces

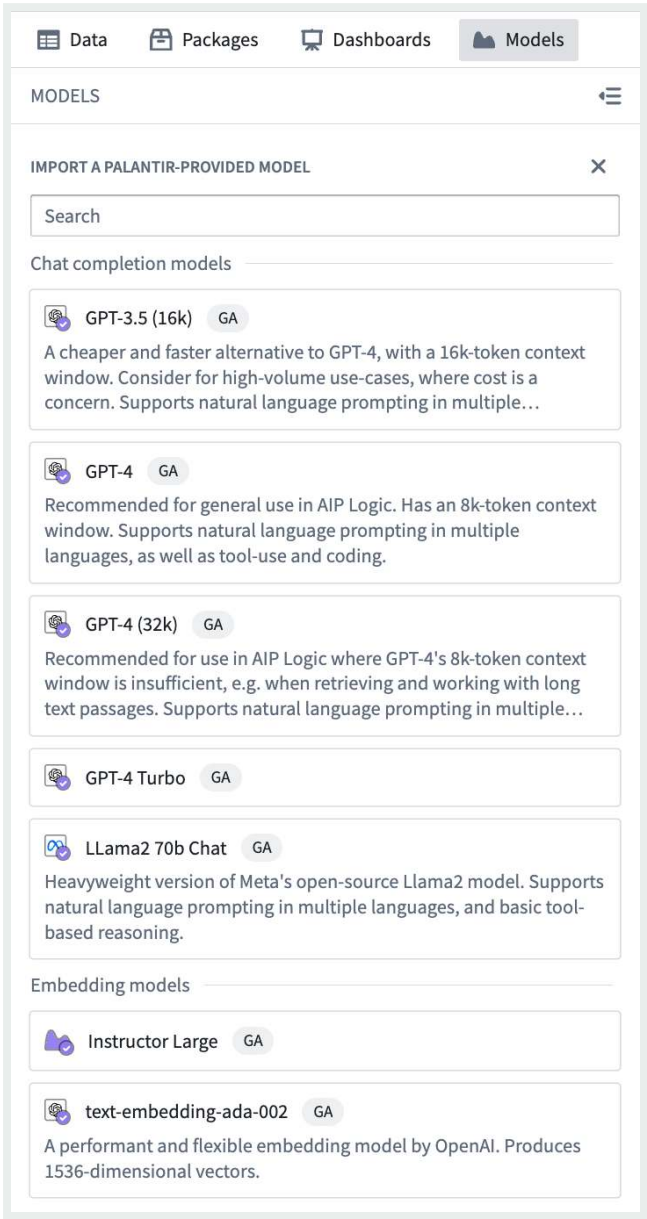
To add language model support to your notebook, open the packages search panel on the left side of your Code Workspace. Search for `palantir_models`, then choose **Latest**. This will copy an install command to your clipboard, which you can then paste into an empty cell and run.

Add a Palantir-provided model to your notebook

To add a language model to your notebook, open the **Models** panel on the left side of your Code Workspace. If you haven't already imported a model, click **Import a Palantir-provided model**. If you have already imported a model, you can import additional models by selecting the **API Reference** link at the top of the panel.

[Send feedback](#)

The panel will then show you a searchable list of models that are available to you. Models are listed in two categories: chat completion models and embedding models. Select the desired model to import it into your Code Workspace.



i Model availability may differ between customers. For more information, contact your Palantir representative.

After import, your model will appear in the **Models** panel. Selecting the model in the **Models** panel will display a code snippet demonstrating basic functionality for the model.

API Reference ↗

Send feedback

MODELS

MODELS

Chat completion models

 GPT-4 (32k) GA

Recommended for use in AIP Logic where GPT-4's 8k-token context window is insufficient, e.g. when retrieving and working with long text passages. Supports natural language prompting i...

```
from language_model_service_api.language_model_service_api_completion_v3 import GptChatCompletionRequest
from language_model_service_api.language_model_service_api import ChatMessage, ChatMessageRole
from palantir_models.models import OpenAiGptChatLanguageModel

model = OpenAiGptChatLanguageModel.get("gpt_v4__32k")
response = model.create_chat_completion(GptChatCompletionRequest([ChatMessage(ChatMessageRole.USER, "why is the sky blue?")]))
```

To get started with the model, click the snippet to copy the code and paste into any cell in your notebook.

Using the language model to generate completions

In this example, we will use an OpenAI model to answer a question. Assuming you have already imported a model, you can copy the code snippet below into any cell to proceed.

API Reference ↗

```
from language_model_service_api.language_model_service_api_completion_v3 import GptChatCompletionRequest
from language_model_service_api.language_model_service_api import ChatMessage, ChatMessageRole
from palantir_models.models import OpenAiGptChatLanguageModel

model = OpenAiGptChatLanguageModel.get("gpt_v4__32k")
response = model.create_chat_completion(GptChatCompletionRequest([ChatMessage(ChatMessageRole.USER, "why is the sky blue?")]))
```

Send feedback

```
5 model = OpenAiGptChatLanguageModel.get("gpt_v4")
6 response = model.create_chat_completion(GptChatCompletionRequest([ChatMess
```

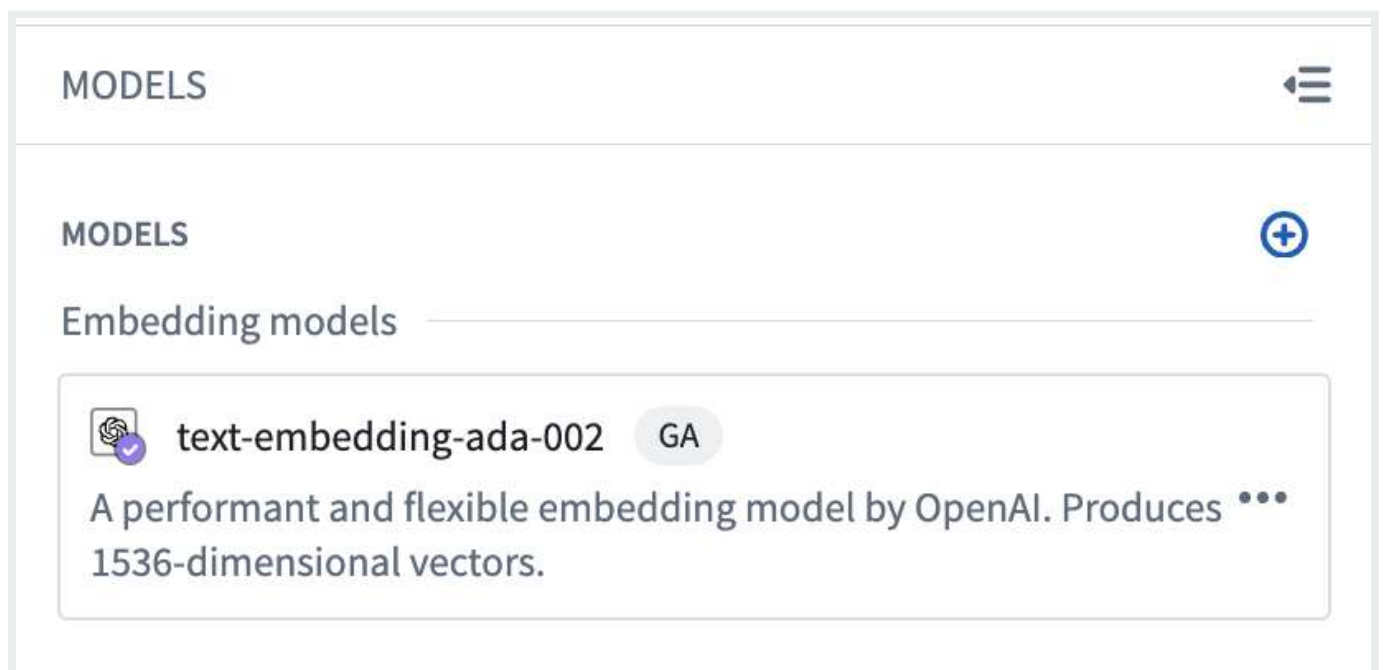
Embeddings

Along with generative language models, Palantir also provides embedding models. The following example shows how we can use an embedding model to calculate embeddings for a list of words, and plot the embeddings to visualize them. Each code block below should be treated as its own cell.

First, add the dependencies needed for this example:

```
1 !mamba install -y "palantir_models>=0.1795.0" matplotlib numpy scikit-learn
```

Then make sure you have imported an embedding model in the **Models** panel. In this example, we will use OpenAI's `text-embedding-ada-002` model.



To create embeddings, we start with a copy of the model snippet and make
s [API Reference ↗](#) as shown below:

[Send feedback](#)

```

1 from language_model_service_api.language_model_service_api_embeddings_v3 import
2 from palantir_models.models import GenericEmbeddingModel
3
4 fruits = [
5     "apple", "banana", "orange", "melon", "kiwi", "pear", "grape", "straw
6     "blueberry", "berry", "mango", "watermelon"
7 ]
8 animals = [
9     "dog", "cat", "cow", "eagle", "mouse", "horse", "squirrel", "lion", "
10 ]
11 words = fruits + animals
12
13 model = GenericEmbeddingModel.get("text-embedding-ada-002")
14 embeddings = model.create_embeddings(GenericEmbeddingsRequest(inputs=words))

```

Finally, we can use scikit-learn and Matplotlib to visualize our embeddings:

```

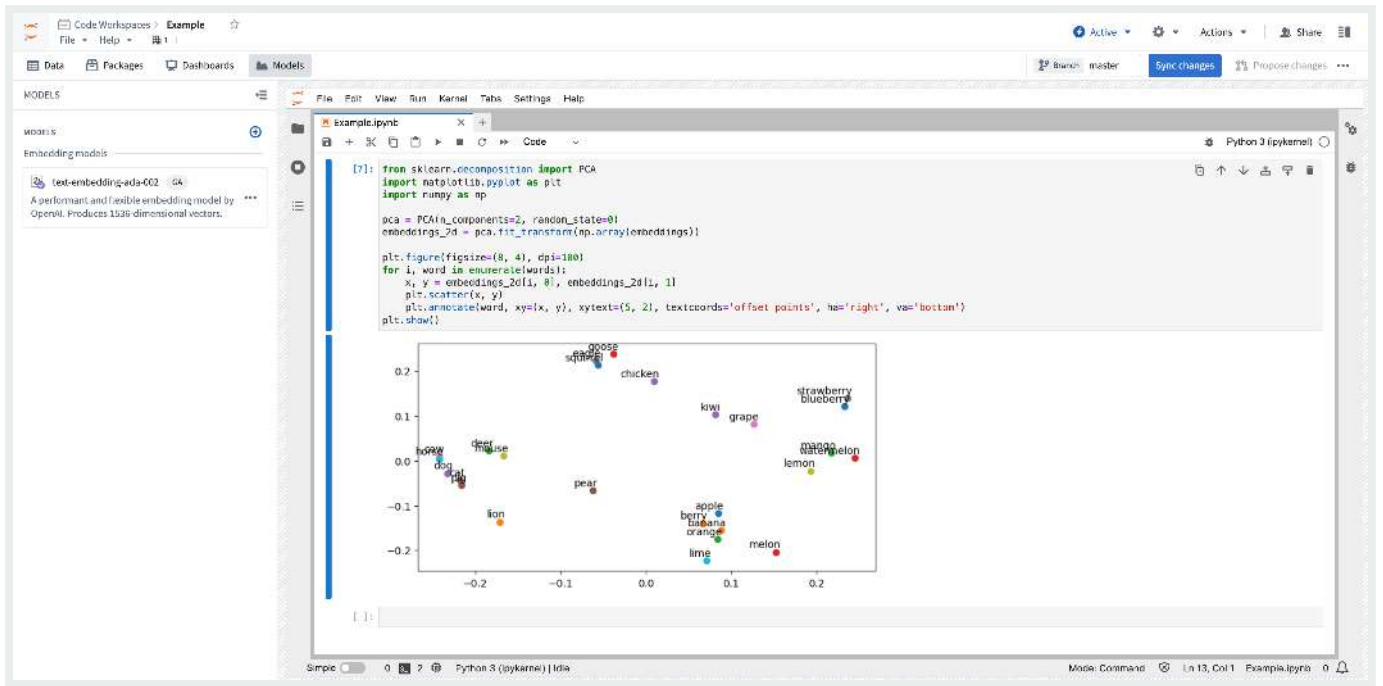
1 from sklearn.decomposition import PCA
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 pca = PCA(n_components=2, random_state=0)
6 embeddings_2d = pca.fit_transform(np.array(embeddings))
7
8 plt.figure(figsize=(8, 4), dpi=100)
9 for i, word in enumerate(words):
10     x, y = embeddings_2d[i, 0], embeddings_2d[i, 1]
11     plt.scatter(x, y)
12     plt.annotate(word, xy=(x, y), xytext=(5, 2), textcoords='offset point
13 plt.show()

```

After running the notebook, you will be presented with a graph of the embeddings:

[API Reference ↗](#)

[Send feedback](#)



Jupyter®, JupyterLab®, and the Jupyter® logos are trademarks or registered trademarks of NumFOCUS.

All third-party trademarks (including logos and icons) referenced remain the property of their respective owners. No affiliation or endorsement is implied.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

API Reference ↗

Send feedback

Security

Code Workspaces ensures that Foundry's security and permissions model is applied to the third-party IDEs connected to Foundry. This layer of security on top of the third-party applications served by Code Workspaces provides a number of benefits:

- Data loaded in Code Workspaces is tracked by Foundry. Data downloads and uploads are restricted in Code Workspaces other than [methods governed by Foundry's data governance and access controls](#).
- Every request to Code Workspaces is validated against Foundry's governance framework. This means that if a user's access to a Code Workspace is revoked, or access to the security markings of any data imported in the workspace, that user will immediately lose access to the application.
- Data produced from Code Workspaces is tracked by Foundry, so access to output data will be restricted if a user loses access to data which may have been used to produce the output data.
- Users are fully isolated. Each user opening a given Jupyter® or RStudio® Code Workspace will get their own isolated environment.
- R and Python packages can only be loaded from Foundry Artifacts channels backing the repository, which enables control over the Conda, PyPI, or CRAN packages that can be used in a specific Code Workspace.
- External API calls can only be made to URLs configured as Network Policies which have been added to the Code Workspace.

Restricted outputs mode

API Reference ↗

R Restricted outputs mode provides enhanced safeguards when working with sensitive data.

R Restricted outputs mode is a "read-only" mode that restricts write op

Send feedback

source connections to ensure that data cannot be exported from the code workspace. In particular, it allows you to load [restricted views](#) into a workspace.

When enabled, restricted outputs mode:

- **Prevents write operations** to all Foundry outputs, including datasets, models, and tables.
- **Disables telemetry collection and logs**, as well as data checkpoint uploads, while still preserving [code checkpoints](#).
- **Disables network policies and sources** that have been added to the workspace. These resources remain in the workspace but are inactivated while in restricted outputs mode.

As described in the documentation on [code checkpoints](#), it is your responsibility to ensure that code files you write while in restricted outputs mode do not contain data.

To enable or disable restricted outputs mode:

1. Open the **Settings** side panel in your code workspace.
2. Locate the **Restricted outputs mode** toggle.
3. Toggle the setting on or off as needed.
4. Save your changes and restart your workspace when prompted.

i After enabling restricted outputs mode, your workspace will prompt you to restart. Your setting will persist across future workspace restarts until you explicitly disable it.

RStudio® and Shiny® are trademarks of Posit™.

Jupyter®, JupyterLab®, and the Jupyter® logos are trademarks or registered trademarks of NumFOCUS.

All third-party trademarks (including logos and icons) referenced remain the property of their respective owners. No affiliation or endorsement is implied.

[API Reference ↗](#)

[Send feedback](#)

[PREVIOUS](#)

[NEXT](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

FAQ

Can I use PySpark in Code Workspaces?

Yes, you can use PySpark in Jupyter® Code Workspaces. However, it runs in local mode inside your workspace container and uses only the available CPUs and memory; it does not run on a distributed Spark cluster. We recommend Foundry applications like [Pipeline Builder](#) and [Code Repositories](#) to run distributed Spark workloads.

To install PySpark in your JupyterLab® Notebook, navigate to the **Libraries** tab and install both `pyspark` and `openjdk`. The following OpenJDK versions are supported:

- **PySpark < 4.0.0:** OpenJDK `8.x`, `11.x`, or `17.x`
- **PySpark 4.0.0 and above:** OpenJDK `17.x` or `21.x`

i If you encounter an error such as `java.lang.UnsupportedOperationException: getSubject is supported only if a security manager is allowed`, ensure that OpenJDK is pinned to a supported version for your PySpark release.

Use the following code to create a Spark session, read raw parquet files into a Spark dataframe, and then apply transformations. This approach is useful for drafting PySpark transformations that you want to apply in a Code Repositories PySpark transform.

```
1 from foundry.transforms import Dataset
```

```
2 from foundry.sql import SparkSession
```

```
3 import pyspark.sql.functions as F
```

```
4 # Create the Spark context. The example below shows a local mode workspace
```

```
5 # Create the Spark session
```

```
6 spark = (SparkSession.builder
```

API Reference ↗



Send feedback

```

7     .appName("ParquetExample")
8     .master("local[2]") # Set to the CPUs available.
9     .config("spark.driver.memory", "10g") # Define the driver memory, an
10    .config("spark.driver.memoryOverhead", "4g") # Define the overhead m
11    .config("spark.sql.shuffle.partitions", "6") # Optionally define par
12    .getOrCreate())
13    sc = spark.sparkContext
14
15    # Download a Foundry dataset of parquet files into the notebook.
16    parquet_foundry = Dataset.get("parquet_foundry_dataset")
17    files = parquet_foundry.files().download()
18
19    # Read the files into a Spark dataframe.
20    spark_df = spark.read.parquet(*list(files.values()))
21
22    # Apply PySpark transformations.
23    df_processed = spark_df.filter(F.col("col1") == 1)
24
25    # Write your output dataset.
26    spark_output = Dataset.get("spark_output")
27    spark_output.write_table(df_processed.toArrow())

```

Can I make API calls in Code Workspaces?

Yes, you can make API calls in Code Workspaces using [external systems](#), which have succeeded network policies as the preferred method of egress from a workspace. For [CBAC-enabled environments](#) you should use network policies configured in [Control Panel](#).

Which IDEs are supported by Code Workspaces?

Code Workspaces currently supports JupyterLab® and RStudio® Workbench.

Which packages are not supported by Code Workspaces?

Due to security reasons, the following Python packages are not supported:

API Reference ↗

- folium
- pandasgui

Send feedback

Contact your Palantir representative if you have any concerns about the packages above.

Why does the code in my repository have JSON formatting?

The Code Repositories application receives code from associated Code Workspaces in an IPython format, which renders the code at a cell-by-cell level in JSON format.

Can I use my own packages?

Yes; see the documentation on [importing packages](#). If your package is hosted on an organizational Conda/PyPI/CRAN channel, it is possible for Foundry to proxy the channel and make it available to your projects. Contact your Palantir representative for more information.

Why does the Code Repository backing my Code Workspace not have a **Libraries** tab?

To import libraries into your Code Workspace, [use the **Packages** tab](#) located in the left panel of your workspace.

Can I write or edit code in Code Repositories that does not come from my Code Workspace?

Yes, you can edit code directly in Code Repositories when the code originates in Code Workspaces. Once committed, you can use the **Sync** or **Reset changes** functionality in the Code Workspace to pick up the remote changes in the Workspace.

Conceptually, you can think of Code Repositories as the version control manager for Code Workspaces, handling pull requests, conflict resolution, and administration, while code development can occur in Code Workspaces.

[API Reference ↗](#)

[Send feedback](#)

Why do my colleagues see a different view compared to mine?

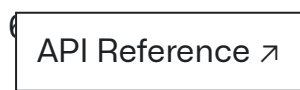
For security purposes, users are isolated when working in JupyterLab® or RStudio®. This means each user accessing the same Code Workspace will have their own environment. Collaboration happens through git workflows: if you wish to make your latest code available to colleagues, select **Sync Changes** to synchronize your changes with the backing code repository and the changes will become available to your colleagues when they select **Sync** or **Reset changes**. When multiple users work on the same workspace, we recommend they work on independent branches.

Note that we ignore some files by default using `.gitignore` to ensure that no data is synchronized with the git repository, and to limit the size of the git repository. We also remove all outputs from JupyterLab® `.ipynb` files.

Can I import extra LaTeX packages to render my R Markdown file?

Yes. By default, Code Workspaces only pre-installs the requirements to render common R Markdown files (as defined in [TinyTex-1 ↗](#)), but you can install other LaTeX packages on your own:

1. Download or create the TDS archive. For instance, you can download a `.tds.zip` file from [CTAN ↗](#).
2. Upload the TDS archives to a new or existing Foundry dataset. You can drag and drop files in a folder to upload them to a new Foundry dataset.
3. Select **Upload to a dataset without a schema**. You may upload multiple TDS archives simultaneously to the same dataset.
4. In Code Workspaces, go to the **Data** tab, then **Read existing datasets**. Select the dataset containing the TDS archives.
5. Optionally, update the dataset alias. In this example, we assume you named your dataset alias `my_latex_packages`.

 API Reference ↗ snippet, paste it in your `.Rprofile`, and update it to also extract the files
ation. You will need to replace `my_latex_packages` with your dataset
alias in the below snippet:

Send feedback

```
library(foundry)
my_latex_packages_files <- datasets.list_files("my_latex_packages")
my_latex_packages_local_files <- datasets.download_files("my_latex_packages", my
texmflocal <- system("kpsewhich --var-value TEXMFLOCAL", intern = TRUE)
sapply(my_latex_packages_local_files, function(tds_file) { unzip(tds_file, exdir
```

7. You can now include your package in the R Markdown header to use it:

```
headers-include:
- \usepackage{my_package}
```

Can I use custom fonts in my R Markdown file?

Yes.

1. Download or create the TTF files you want to use.
2. Upload the TTF files to a new or existing Foundry dataset. You can drag and drop files in a folder to upload them to a new Foundry dataset.
3. Select **Upload to a dataset without a schema**. You may upload multiple TTF files simultaneously to the same dataset.
4. In Code Workspaces, go to the **Data** tab, then **Read existing datasets**. Select the dataset containing the TTF files.
5. Optionally, update the dataset alias. In this example, we assume you named your dataset alias `my_fonts`.
6. Copy the code snippet, run it in the R Console, and copy the files to your repo so they can be tracked by the version control. You will need to replace `my_fonts` with your dataset alias in the below snippet:

```
my_fonts_files <- datasets.list_files("my_fonts")
my_fonts_local_files <- datasets.download_files("my_fonts", my_fonts_files)
dir.create("fonts")
API Reference ↗ (my_fonts_local_files), "fonts")
```

Send feedback

7. You can now reference the fonts in the `/home/user/repo/fonts` directory so they can be used in your HTML or PDF outputs following the R Markdown recommendations.

To change the font for HTML, set the `font-family` in a [custom CSS ↗](#).

To change the font for PDFs, you can load the font in LaTeX with `\newfontfamily` using [custom LaTeX code ↗](#) in the preamble.

How can I prevent my Jupyter® workspace from pausing while my code is running?

You can leverage the `%%keep_alive` cell magic to prevent Code Workspaces from pausing a Jupyter® notebook while the code in the cell is running.

```
1 %%keep_alive
2
3 long_running_process()
```

This will keep the cell alive for up to 24 hours while the cell code is running. If your browser tab is closed or left idle for too long, JupyterLab® will stop displaying and persisting the cell's output. If you would like to view the cell's output later, you may use the `%%capture` cell magic to store the output in a variable:

```
1 %%keep_alive
2 %%capture cell_output
3
4 long_running_process()
```

In a different cell, write the output of the above variable to a file so it can be viewed later:

```
API Reference ↗ cell_output.txt', 'w+') as f:
    cell_output.stdout
    f.write(cell_output.stderr)
```

Send feedback

How can I download files from my Jupyter or RStudio Code Workspace?

The IDE native file downloads are disabled in the Palantir platform to ensure that download restrictions configured by administrators are enforced.

In order to download files, write the files back to a Foundry dataset. To do so, in the Code Workspaces sidebar, navigate to the **Data** tab, then **Write data to new dataset**, and enter a name for the new dataset. After confirming the alias for the dataset, select **Non-tabular dataset** as dataset type, enter the location of the files or directory you wish to upload and run the generated code snippet.

Once the files have been uploaded to the Foundry dataset, you can navigate to the dataset preview by selecting on the dataset link and download the files from there. Remember that your administrator may require a justification before exporting data from the platform or limit the amount of data exported.

RStudio® and Shiny® are trademarks of Posit™.

Jupyter®, JupyterLab®, and the Jupyter® logos are trademarks or registered trademarks of NumFOCUS.

All third-party trademarks (including logos and icons) referenced remain the property of their respective owners. No affiliation or endorsement is implied.

Troubleshooting

This page contains troubleshooting tips for errors that you may encounter while using Code Workspaces. If you have other issues or are unable to resolve your issue with this guide, contact your Palantir representative.

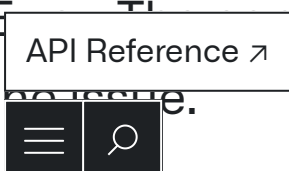
Error messages and suggested resolution

Error: Unauthorized – Session is no longer valid. It may have timed out. Please refresh the page.

If you encounter the “Unauthorized” error stating that your session may have timed out, follow these steps for remediation:

1. Refresh the page.
2. Ensure that you are using Google Chrome to access Code Workspaces. The Safari browser is **not** supported.
3. If data with security markings was previously loaded in this Code Workspace and you lose access to the marked data, you may receive this error. If this is the case, select **Running** and then **Restart workspace** to get a fresh session with no data.
4. If this is your first time using Code Workspaces, you may need to contact an IT administrator to ensure that Code Workspaces is properly whitelisted.

Error: Content is blocked. Contact the site owner to fix the issue.



Send feedback

Foundry applies a strict Content Security Policy to ensure the content rendered in Code Workspaces is properly controlled. For instance, Code Workspaces does not allow loading arbitrary JavaScript from CDNs, as is the case with some Python packages like `folium`.

If you receive this error for a critical workflow, contact your Palantir representative to discuss potential support for the workflow. Some workflows are already supported, but may require you to run a slightly different command:

- [Building Dash applications](#)
- [Building Streamlit applications](#)
- [Previewing TensorBoard™](#)

Error: package or namespace load failed

You may encounter a package or namespace load failure error as below:

```
> library("hdf5r")
Error: package or namespace load failed for 'hdf5r' in dyn.load(file, DLLpath =
  unable to load shared object '/home/user/libs/r/hdf5r/libs/hdf5r.so':
    libhdf5_serial_hl.so.100: cannot open shared object file: No such file or dire
```

This can occur since Foundry limits the number of Linux packages that are pre-installed on the server in order to reduce the exposure to potential vulnerabilities. However, some R libraries rely on specific system Linux libraries which may not be pre-installed on the server.

Note that some packages are [not supported by Code Workspaces](#)

Contact your Palantir representative to discuss the availability of required Linux package(s).

Error: Failed building wheel when installing a pip package

Pip may expect some system dependencies to be pre-installed, while conda bundles all required system dependencies. For more information, see the [API Reference](#) for the Python package to run. The general recommendation is to:

1. Always install from conda instead of pip if possible.
2. If possible, conda-build the pip package and publish it to an Artifacts repository.

[Send feedback](#)

3. Otherwise, try installing from pip using the following command:

```
1 maestro env pip install <package>
```

4. If the pip package fails to install, the error will generally indicate which system dependency or file is missing. First find the conda package that supplies the required dependency, and install it using the following command:

```
1 maestro env conda install <package>
```

5. Repeat steps 3 and 4 until the installation succeeds

Example

The `octis` package exists in Pypi but not in conda. When running `maestro env pip install octis`, we encounter the following error:

```
...
ERROR      g++ -pthread -B /home/user/envs/default/compiler_compat -Wno-unused-
ERROR      error: command 'g++' failed: No such file or directory
ERROR      [end of output]
ERROR
ERROR      note: This error originates from a subprocess, and is likely not a problem
ERROR      ERROR: Failed building wheel for libsvm
INFO       Running setup.py clean for libsvm
INFO       Failed to build libsvm
ERROR ERROR: ERROR: Failed to build installable wheels for some pyproject.toml b
INFO       Installing pip environment
ERROR      ❌ Failed to install environment, exit status: "1"
```

In [API Reference ↗](#) dependency (`libsvm`) seems to require a compiler to build the wheel, so we will install the compiler via the corresponding conda package.

Send feedback

The above error indicates that the missing dependency is `g++`, therefore we should install the corresponding conda package (in this case, `gxx_linux-64`), using maestro:

```
1 maestro env conda install gxx_linux-64
```

We can now try installing the pip package by running this command again:

```
1 maestro env pip install octis
```

This time the command should succeed.

Error: Attempting to sync data to Code Repository failed

This error occurs when the git commit that is being synced is too large. This can happen if the commit contains non-code files, such as output data. Large files are not supported by version control within Code Workspaces. Instead, [data can be interactively written back to Foundry](#), enabling you to easily use and share the outputs of your code workspace.

To find out which files are causing this error, open a terminal within your workspace and execute the following command:

```
1 git rev-list --objects --all |
2   git cat-file --batch-check='%<objecttype> %<objectname> %<objectsize> %
3   sed -n 's/^blob //p' |
4   awk '{sum[$3]+=$2} END {for (i in sum) print sum[i], i}' |
5   sort --numeric-sort --key=1 |
6   $(command -v gnumfmt || echo numfmt) --field=1 --to=iec-i --suffix=B --
```

To resolve this error, remove any large files from the git history. For instance, to delete all files larger than 10 MB you can run the terminal command: `git filter-repo --strip-blobs-`
b API Reference ↗ Alternatively, to delete a specific file from the git history, use `git filter-repo --path path/to/file.ext --invert-paths`.

Send feedback

To prevent this error happening again, update the `.gitignore` file with the paths or extensions of any large files that should not be synced. For example, to prevent any output CSV files from being synced, add `*.csv`. Note that you may need to ensure that hidden files are visible in your IDE to view the `.gitignore` file.

To apply these changes, run `git push --force` in a terminal. After following these steps, the **Sync changes** process should now work as expected.

File download or upload errors

For security reasons, Foundry generally restricts file downloads and uploads to ensure that these activities occur in a proper data governance framework. This means users cannot add a “download” button or equivalent in a Shiny® or Streamlit application.

To download data from a Jupyter® or RStudio® Workspace, write the data to an output dataset in Foundry first. You can download data from the output dataset using Foundry’s built-in processes, which ensure the application of appropriate data governance restrictions.

File load errors

A file load error can occur if a user enforced limits on the amount of data that can be downloaded or rendered in the browser. In such scenarios, try running the command `jupyter nbconvert --clear-output --inplace notebook.ipynb`, replacing `notebook.ipynb` with the name of the file shown in the error message:



However, note that this command will clear all the cell outputs, including charts.

RStudio® and Shiny® are trademarks of Posit™.

Jupyter®, JupyterLab®, and the Jupyter® logos are trademarks or registered trademarks of NumFOCUS.

TensorBoard and any related marks are trademarks of Google Inc.

All third-party trademarks (including logos and icons) referenced remain the property of their respective owners. No affiliation or endorsement is implied.

PREVIOUS
← [FAQ](#)

NEXT
[Compute usage](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Compute usage with Code Workspaces

With [Code Workspaces](#), users can securely connect to existing internal systems and build analyses, models, applications, or entire workflows on data with Foundry’s access controls and data permissioning.

Code workspaces spin up dedicated Foundry computation modules for execution, which utilizes Foundry compute-seconds while in use.

Pricing

Code workspace usage is tracked as Foundry compute-seconds (see [usage types](#)). Compute seconds are measured as long as the workspace is starting or active. There are a few factors to determine compute seconds used:

- The number of virtual CPUs (vCPUs).
- The gibibits (GiB) of RAM.
- The workspace being used.
- The number and the type of GPUs.

When paying for Foundry usage, the default rates are as follows:

Workspace	GPU Type	Usage Rate
API Reference (profile)	No GPU	0
	No GPU	0.1
JupyterLab®	No GPU	0.1

Send feedback

Workspace	GPU Type	Usage Rate
RStudio®	No GPU	0.1
Dash	No GPU	0.1
Streamlit	No GPU	0.1
Shiny®	No GPU	0.1
Any workspace	T4 GPU	1.2
Any workspace	A100 GPU	1.3
Any workspace	A10G GPU	1.5
Any workspace	L4 GPU	2.1
Any workspace	V100 GPU	3.0
Any workspace	H100 GPU	4.7

If you have an enterprise contract with Palantir, contact your Palantir representative before proceeding with compute usage calculations.

```
vcpu_compute_seconds = max(vCPUs, GiB_RAM/ 7.5) * vcpu_usage_rate * time_active_in_seconds
```

The following formula measures GPU compute seconds:

```
gpu_compute_seconds = GPUs * gpu_usage_rate * time_active_in_seconds
```

i Each code workspace comes with a Foundry sidecar that introduces a small overhead of 0.25 vCPUs and 3 GiB of RAM

Workspace	Compute Profile	Duration	Incurred Usage
VS Code	Default profile (0.75 CPUs/6 GiB)	1 hour	0 compute-seconds
API Reference ↗	4 CPUs/32 GiB	1 hour	1680 compute-seconds
JupyterLab®	1 CPU/8 GiB	1 hour	528 compute-seconds
Any workspace	1 T4 GPU	1 hour	4320 compute-seconds

Send feedback

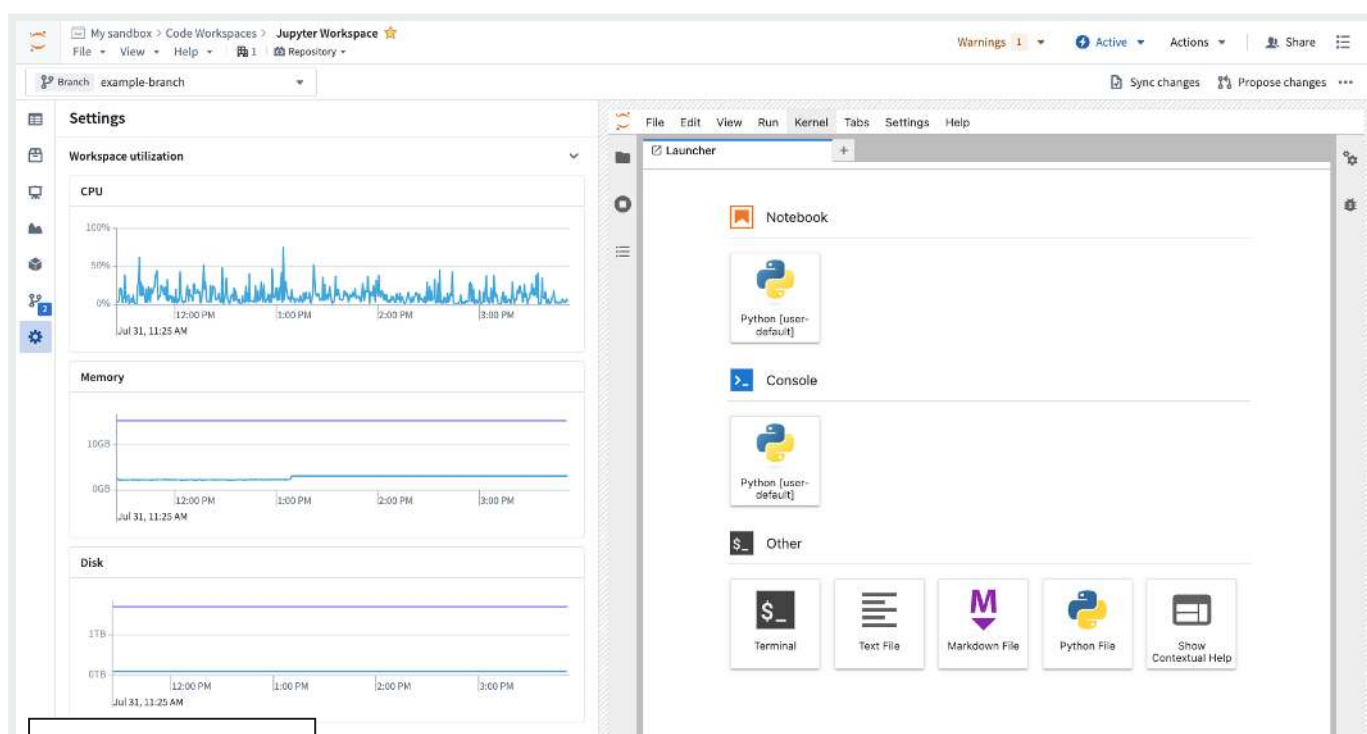
Workspace	Compute Profile	Duration	Incurred Usage
Any workspace	1 A10G GPU	1 hour	5400 compute-seconds
Any workspace	1 V100 GPU	1 hour	10800 compute-seconds

VS Code Usage Pricing

By default, new VS Code workspaces are provisioned with 0.75 CPU and 6 GB memory. This default profile is free of charge and not subject to usage-based pricing (UBP). You can adjust these resources when creating a workspace using the **Advanced** configuration section, or later by using the **Compute resources** slider in the settings side panel. If you increase memory, CPU, or add GPUs to your workspace, standard UBP pricing will apply for those upgrades.

View workspace utilization

Live workspace utilization metrics are available for CPU, memory and disk usage. To view these metrics, expand the **Workspace utilization** section of the **Settings** side panel.



[API Reference ↗](#)

Measuring Foundry compute

[Send feedback](#)

When a Code Workspace is opened, Foundry will launch a dedicated compute session for the workspace. The session's status is `STARTING` until the session is available, becomes `INITIALIZING`, and then `RUNNING`. When the user manually stops the session, or when no user interaction is detected for longer than the auto-shutdown timeout, the session will become `STOPPING` and then `IDLE`. Foundry compute seconds are only used when the session is `INITIALIZING` or `RUNNING`.

All the possible states for a workspace are listed below. The states when the code workspace uses compute are indicated with right arrows (→).

- `STARTING` : a new session was requested
- → `INITIALIZING` : the session is available and is being config
- → `RUNNING` : the session is available and usable
- `STOPPING` : the session is stopping
- `IDLE` : the session is stopped
- `FAILED` : an error occurred
- `TERMINATING` : the session is being deleted permanently
- `NOT_STARTED` : no session available for this workspace

Code Workspaces measure compute seconds in the same manner as other scaling compute in the platform. Review the general [Foundry Compute documentation](#) for a description of how it is measured.

Investigating Foundry compute usage with Code Workspaces

Foundry Code Workspaces attribute all associated compute to the workspace resource in the filesystem. You can always view the usage of all sessions in the [Resource Management Application](#).

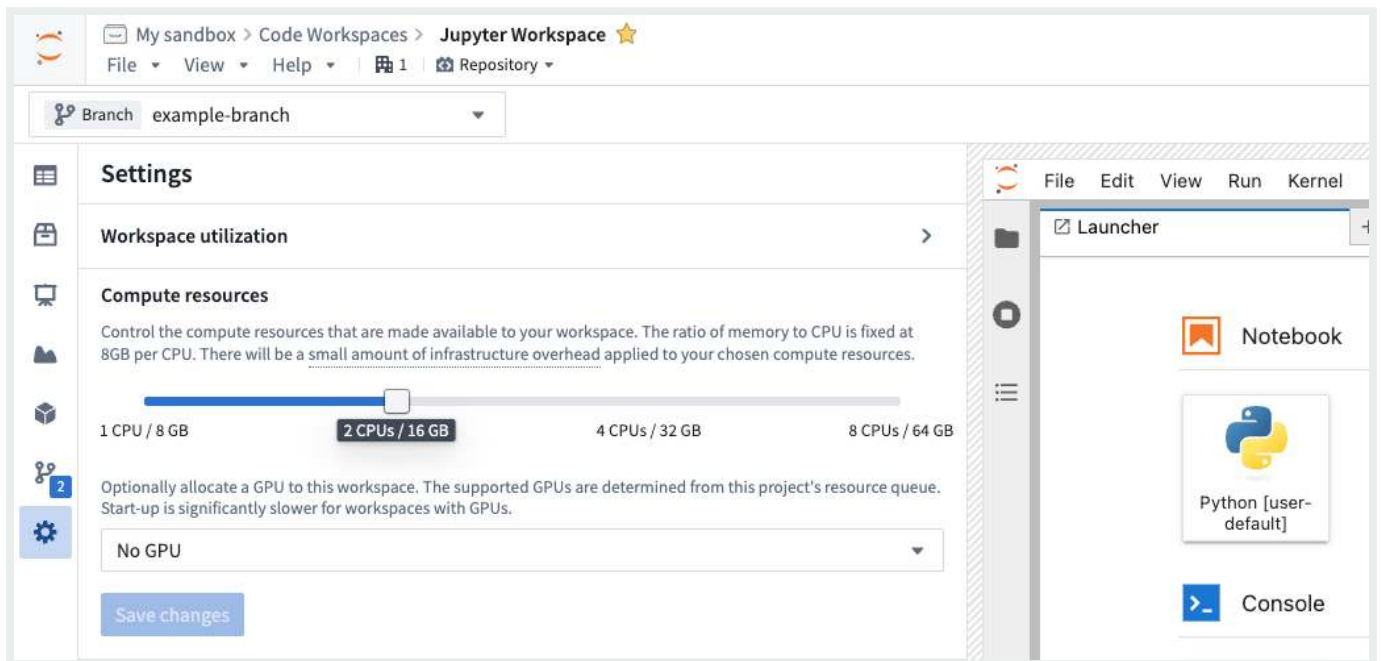
Understanding drivers of Foundry compute usage in Code Workspaces

[API Reference ↗](#)

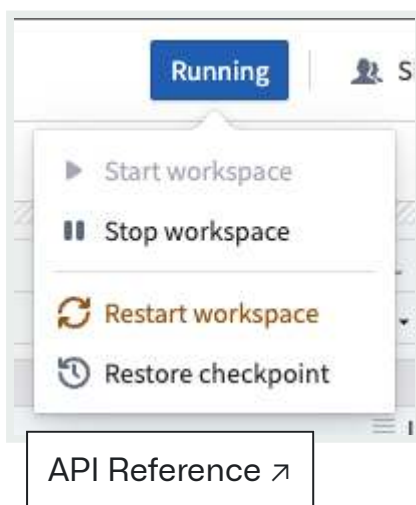
[Send feedback](#)

The compute usage of a Code Workspace session is directly proportional to the dedicated computational resources available to session and the length of the session.

To manage the hardware size of your session, go to **Settings > Compute Resources**. You can choose from a variety of different session sizes. Refer to the screenshot below for an indication of how you can choose sizes and optionally allocate GPUs to your workspace. The resource queue in the project containing your workspace determines available GPUs. Learn more about [using GPUs in projects](#) and [leveraging GPUs for model training](#).



Long-running sessions utilize more compute than short-running sessions. Be sure to tune your auto-shutdown time to be consistent with your working style to not use more compute than necessary. You should manually stop sessions when you know you are done.



© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)